Handbook de Programación Competitiva



McDuck Team

November 1, 2026

Contents

1 Base de competencia **9**

1.1 Template C++17 . . . 9
1.2 Includes estándar sin bits/stdc++.h . . . 9
1.3 Matemática cotidiana en C++ . . . 9
1.4 Template Python 3 . . . 15
1.5 Imports y utilidades de Python . . . 16
1.6 Tipos, límites y overflow . . . 17
1.7 Sintaxis C++ más usada . . . 17
1.8 STL esencial . . . 18
1.9 Contenedores comunes . . . 18
1.10 Algoritmos STL comunes . . . 19
1.11 Comparadores, bits y aleatoriedad . . . 19
1.12 Entrada y salida . . . 20
1.13 Parsing seguro . . . 21
1.14 Letras a posiciones . . . 21

2 Técnicas **23**

2.1 Búsqueda binaria . . . 23
2.2 Sumas parciales . . . 23
2.3 Two pointers y sliding window . . . 23
2.4 Máscaras de bits . . . 24
2.5 Conversión a base arbitraria . . . 24
2.6 Compresión de coordenadas . . . 25
2.7 Descomposición por raíz: frecuentes y poco frecuentes . . . 25
2.8 Meet-in-the-middle: suma de subconjuntos . . . 26
2.9 Constant intervals of a monotone function . . . 26
2.10 Union and subtraction of intervals . . . 27
2.11 Minimum interval cover . . . 27
2.12 Discrete ternary search . . . 27

3 Estructuras de datos **29**

3.1 Disjoint Set Union (DSU) . . . 29
3.2 Fenwick Tree (BIT) . . . 29
3.3 Segment Tree iterativo . . . 30
3.4 Lazy Segment Tree . . . 30
3.5 Segment Tree persistente . . . 31

3.6 Sparse Table . . . 32
3.7 Disjoint Sparse Table . . . 32
3.8 Stack y queue monótonas . . . 33
3.9 Ordered Set de PBDS . . . 33
3.10 Fenwick 2D offline . . . 33
3.11 Wavelet Tree . . . 34
3.12 Treap implícito . . . 35
3.13 Convex Hull Trick y Li Chao Tree . . . 37
3.14 Intervalos disjuntos (ODT) . . . 38
3.15 KACTL source conventions . . . 38
3.16 DSU rollback . . . 38
3.17 Range assignment, addition and maximum . . . 39
3.18 2D prefix sums . . . 39
3.19 Mo queries on tree paths . . . 40

4 Grafos **41**
4.1 BFS, DFS y bipartición . . . 41
4.2 BFS multiorigen . . . 41
4.3 Orden topológico . . . 41
4.4 Componentes fuertemente conexas (Tarjan) . . . 42
4.5 2-SAT . . . 42
4.6 Dijkstra . . . 43
4.7 0–1 BFS . . . 44
4.8 Bellman–Ford . . . 44
4.9 Floyd–Warshall . . . 45
4.10 Árbol de expansión mínima (Kruskal) . . . 46
4.11 Puentes y puntos de articulación . . . 46
4.12 Componentes biconexas por vértices . . . 47
4.13 Ancestro común más bajo (LCA) . . . 48
4.14 LCA con binary lifting . . . 49
4.15 LCA $O(1)$ con Euler tour y RMQ . . . 50
4.16 Virtual Tree . . . 50
4.17 Heavy-Light Decomposition . . . 51
4.18 Descomposición por centroides . . . 52
4.19 Rerooting DP . . . 53
4.20 Diámetro de un àrbol y camino . . . 54
4.21 Camino euleriano . . . 54
4.22 Flujo máximo (Ford–Fulkerson) . . . 55

4.23 Flujo máximo (Dinic) . . . 56
4.24 Flujo de costo mínimo . . . 57
4.25 Matching bipartito: Hopcroft–Karp . . . 58
4.26 Asignación de costo mínimo: Húngaro . . . 59
4.27 Cobertura mínima bipartita: König . . . 59
4.28 Bellman–Ford: affected destinations . . . 60
4.29 Biconnected edge blocks . . . 60
4.30 Recovering a minimum s-t cut . . . 61
4.31 Global minimum cut . . . 61
4.32 Gomory–Hu tree . . . 62
4.33 Matching in general graphs . . . 62
4.34 Edge coloring . . . 63
4.35 Enumerating maximal cliques . . . 63
4.36 Maximum clique . . . 63
4.37 Maximum independent set . . . 64
4.38 Dynamic forests: link-cut tree . . . 64
4.39 Directed minimum spanning arborescence . . . 65

5 Strings **67**
5.1 Rolling hash . . . 67
5.2 Prefix function, KMP . . . 67
5.3 Z-function . . . 68
5.4 Trie . . . 68
5.5 Manacher . . . 69
5.6 Aho–Corasick . . . 69
5.7 Suffix Array y LCP . . . 70
5.8 Minimum string rotation . . . 71
5.9 Online suffix tree and longest common substring . . . 71

6 Matemática **73**
6.1 Teoremas e identidades para reconocer soluciones . . . 73
6.2 GCD, LCM y Euclides extendido . . . 74
6.3 Aritmética modular . . . 75
6.4 Teorema chino del resto para dos congruencias . . . 75
6.5 Combinatoria módulo primo . . . 76
6.6 Teorema de Lucas . . . 76
6.7 Criba lineal y menor factor primo . . . 76
6.8 Criba segmentada: primos en un intervalo . . . 77
6.9 Factorización por división por tentativa . . . 78

6.10 Listar divisores . . . 78
6.11 Pollard–Rho con Brent . . . 78
6.12 Pollard–Rho con Brent optimizado . . . 79
6.13 Exponenciación de matrices . . . 81
6.14 Eliminación gaussiana y determinante . . . 81
6.15 Base XOR de enteros de 64 bits . . . 82
6.16 Dependencia XOR de bitsets con reconstrucción . . . 83
6.17 NTT y convolución . . . 83
6.18 Continued fractions . . . 84
6.19 Binary search on fractions . . . 85
6.20 Table of modular inverses . . . 85
6.21 Discrete logarithm . . . 86
6.22 Modular square root . . . 86
6.23 Sums of floors and modular progressions . . . 87
6.24 Totient sieve . . . 87
6.25 Recovering a linear recurrence . . . 87
6.26 Distant term of a linear recurrence . . . 88
6.27 Polynomial operations . . . 88
6.28 Polynomial interpolation . . . 89
6.29 Real polynomial roots . . . 89
6.30 Real convolution with FFT . . . 89
6.31 Convolution modulo arbitrary integers . . . 90
6.32 AND, OR and XOR convolutions . . . 90
6.33 Golden-section search . . . 91
6.34 Two-dimensional local optimization . . . 91
6.35 Adaptive numerical integration . . . 92
6.36 Determinant modulo a composite integer . . . 92
6.37 Real matrix inverse . . . 92
6.38 Matrix inverse modulo a prime . . . 93
6.39 Uniquely determined variables in singular systems . . . 93
6.40 Linear systems over F2 . . . 94
6.41 Linear programming: simplex . . . 94
6.42 Permutation indexing . . . 95

7 Geometría **97**
7.1 Geometría 2D entera . . . 97
7.2 Área y convex hull . . . 97
7.3 Rectas con punto flotante . . . 98

7.4 Círculos . . . 98
7.5 Punto en polígono . . . 99
7.6 Rotating calipers: diámetro convexo . . . 99
7.7 Geometría 3D y planos . . . 100
7.8 Camino mínimo rodeando un polígono convexo . . . 100
7.9 Angular ordering . . . 100
7.10 Distance to a segment . . . 101
7.11 Intersection points of segments . . . 101
7.12 Reflection across a line . . . 102
7.13 Similarity between oriented segments . . . 102
7.14 Circle–line intersection . . . 103
7.15 Common circle tangents . . . 103
7.16 Circle–polygon intersection area . . . 103
7.17 Circumcircle . . . 104
7.18 Minimum enclosing circle . . . 104
7.19 Polygon centroid . . . 104
7.20 Polygon clipping . . . 105
7.21 Polygon union area . . . 105
7.22 Line–convex hull intersection . . . 106
7.23 Closest pair of points . . . 106
7.24 Nearest-neighbor queries: KD-tree . . . 107
7.25 Manhattan MST candidate edges . . . 107
7.26 Delaunay triangulation . . . 108
7.27 Three-dimensional convex hull . . . 109
7.28 Polyhedron volume . . . 110
7.29 Spherical distance . . . 110

8 Programación dinámica **111**
8.1 Longest Increasing Subsequence . . . 111
8.2 LCS con reconstrucción . . . 111
8.3 Distancia de edición con operaciones . . . 111
8.4 Distancia al palíndromo más cercano . . . 112
8.5 Knapsack 0/1 . . . 112
8.6 Bitmask DP: camino hamiltoniano mínimo . . . 113
8.7 Mochila ilimitada . . . 113
8.8 Suma exacta 0/1 con mínima cantidad de objetos . . . 114
8.9 Subset sum con bitset . . . 114
8.10 Digit DP: contar enteros por sus dígitos . . . 114

8.11 Divide and Conquer DP Optimization . . . 115
8.12 DP $O(n\sqrt{n})$ por cota triangular . . . 116
8.13 Knuth optimization . . . 116

9 Algoritmos offline **117**
9.1 Mo's Algorithm . . . 117

1. Base de competencia

1.1 Template C++17

Base común para los snippets del handbook. Incluye el template clásico del equipo; las macros son opcionales, pero permiten pegar código viejo sin traducir nombres durante una competencia.

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define fst first
#define snd second
#define fore(i,a,b) for (int i = (a), ThxDem = (b); i < ThxDem; ++i)
#define SZ(x) ((int)(x).size())
#define ALL(x) (x).begin(), (x).end()
#define mset(a,v) memset((a), (v), sizeof(a))
#define FIN ios::sync_with_stdio(false); cin.tie(nullptr); cout.tie(nullptr)
using namespace std;

using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;
using u128 = __uint128_t;
using ii = pair<int,int>;
using pii = pair<int,int>;
using pll = pair<ll,ll>;

const int INF = numeric_limits<int>::max() / 4;
const ll LINF = numeric_limits<ll>::max() / 4;

int main() {FIN;

    return 0;
}
```

Para llenar memoria byte a byte, usar `memset` solamente con valores seguros como cero o -1. Para otros valores, usar `fill`.

Cuidado: `fore` guarda el límite una sola vez: no usarlo si ese límite debe cambiar dentro del loop. (Es decir, no se recalcula con cada iteración como si lo haría i $<$ SZ(v) `pb/fst/snd/ALL/SZ` son abreviaturas; ante un error de tipos, expandirlas mentalmente antes de depurar.

1.2 Includes estándar sin bits/stdc++.h

`bits/stdc++.h` es la opción rápida con GNU C++; no es parte del estándar. Si el juez usa Clang/MSVC o exige portabilidad, incluir uno por uno:

Necesidad	Header
entrada/salida	`<iostream>`, `<iomanip>`, `<sstream>`
vector/arreglos/tuplas	`<vector>`, `<array>`, `<tuple>`, `<utility>`
strings	`<string>`, `<cstring>`
sort/búsquedas/numéricos	`<algorithm>`, `<numeric>`
stack/colas	`<stack>`, `<queue>`, `<deque>`
set/map/hash	`<set>`, `<map>`, `<unordered_set>`, `<unordered_map>`
matemática	`<cmath>`, `<complex>`, `<limits>`, `<bitset>`
random/tiempo	`<random>`, `<chrono>`
funciones/opcionales	`<functional>`, `<optional>`
assert/tipos enteros	`<cassert>`, `<cstdint>`

Template portable mínimo:

```cpp
#include <algorithm>
#include <cassert>
#include <cmath>
#include <iostream>
#include <limits>
#include <map>
#include <numeric>
#include <queue>
#include <set>
#include <string>
#include <tuple>
#include <unordered_map>
#include <utility>
#include <vector>
#include <climits>
```

Agregar headers sólo cuando se usen. Para ICPC con GCC, `bits/stdc++.h` sigue siendo la alternativa más práctica.

1.3 Matemática cotidiana en C++

1.3.1 Raíces, potencias y logaritmos

Operación	C++
$\sqrt{x}$ real	`sqrt(x)`, `sqrtl(x)`
x^y real	`pow(x,y)`, `powl(x,y)`
$e^x, \ln x$	`exp(x)`, `log(x)`
$\log_2 x, \log_{10} x$	`log2(x)`, `log10(x)`
redondeos	`floor`, `ceil`, `round`, `trunc`
hipotenusa estable	`hypot(x,y)`
valor absoluto	`abs(x)`, `fabs(x)`

Cuidado: No usar `pow(a,b)` para calcular potencias enteras exactas, ya que trabaja con punto flotante y puede introducir errores de redondeo. En su lugar, usar exponenciación binaria.

Tampoco confiar en convertir directamente `sqrt(n)` a entero cuando `n` está cerca de cuadrados perfectos grandes, debido a posibles errores de precisión.

Para calcular raíces cuadradas enteras, se recomienda utilizar la función `integer_sqrt`.

Para realizar una división entera redondeando el resultado hacia arriba, utilizar `ceil_division`.

Para potencias enteras usar `binary_power`: requiere exponente no negativo y que la respuesta entre en `long long`. La sobrecarga real admite exponentes negativos. Usar `2LL` o `2.0L` para elegir explícitamente la versión entera o real.

```cpp
ll integer_sqrt(ll value) {
    assert(value >= 0);
    ll root = sqrtl((long double)value);
    while ((i128)(root + 1) * (root + 1) <= value) ++root;
    while ((i128)root * root > value) --root;
    return root; // floor(sqrt(value))
}

ll ceil_division(ll a, ll b) {
    assert(b && !(a == numeric_limits<ll>::min() && b == -1));
    ll q = a / b, r = a % b;
    return q + (r != 0 && ((r > 0) == (b > 0)));
}

ll floor_division(ll a, ll b) {
    assert(b && !(a == numeric_limits<ll>::min() && b == -1));
    ll q = a / b, r = a % b;
    return q - (r != 0 && ((r > 0) != (b > 0)));
}

ll binary_power(ll base, ll exponent) {
    assert(exponent >= 0);
    ll result = 1;
    for (; exponent; exponent >>= 1) {
        if (exponent & 1) result *= base;
        if (exponent > 1) base *= base; // Evita el cuadrado final innecesario.
    }
    return result;
}

long double binary_power(long double base, ll exponent) {
    assert(base != 0.0L || exponent >= 0);
    // Convierte |exponent| sin desbordar cuando exponent == LLONG_MIN.
    unsigned long long e = exponent < 0 ? -(unsigned long long)exponent
                                        : (unsigned long long)exponent;
    long double result = 1.0L;
    for (; e; e >>= 1) {
        if (e & 1) result *= base;
        if (e > 1) base *= base;
    }
    return exponent < 0 ? 1.0L / result : result;
}

ll modular_power(ll base, ll exponent, ll modulus) {
    assert(exponent >= 0 && modulus > 0);
    base %= modulus;
    if (base < 0) base += modulus;
    ll result = 1 % modulus;
    for (; exponent; exponent >>= 1) {
        if (exponent & 1) result = (i128)result * base % modulus;
        if (exponent > 1) base = (i128)base * base % modulus;
    }
    return result;
}
```

`integer_sqrt` ya usa una aproximación rápida con `sqrtl`; las correcciones enteras conservan exactitud cerca de cuadrados grandes. Las divisiones requieren divisor no nulo y cociente representable: excluir `LLONG_MIN / -1`. Las potencias usan $O(\log e)$ multiplicaciones y $O(1)$ memoria; para $e = 0$ devuelven la identidad.

Potencia de 2

Para potencias de dos, usar desplazamientos. **Cuidado:** El tipo de la izquierda se determina antes de asignar: usar `1LL << k` para `long long`, con $0 \le k \le 62$ para un resultado positivo. Para `unsigned long long`, usar `1ULL << k` con $0 \le k \le 63$. Nunca desplazar por un valor negativo ni por el ancho del tipo o más.

```cpp
    // Potencia 2 elevado a la k; requiere 0 <= k <= 62.
    int k = 40;
    ll x = 1LL << k;

    // Maxima potencia positiva para tipos de 32, 64 y 128 bits.
    int p_int = 1 << 30;
    unsigned int p_uint = 1U << 31;
    ll p_ll = 1LL << 62;
    unsigned long long p_ull = 1ULL << 63;
    i128 p_i128 = (i128)1 << 126;
    u128 p_u128 = (u128)1 << 127;
```

1.3.2 Punto flotante

Comparar con error absoluto y relativo: Ej: $0.1 + 02 == 03$?

```cpp
bool almost_equal(long double a, long double b,
                  long double epsilon = 1e-12L) {
    return fabsl(a - b) <= epsilon * max({1.0L, fabsl(a), fabsl(b)});
}
```

- `double`: unas 15 cifras decimales; `long double`: más en GNU/Linux x86, pero no necesariamente en todas las plataformas.
- Para $a \cdot b$ antes de módulo, convertir primero a `__int128` si puede exceder `long long`.
- Para $\lceil a/b \rceil$ con positivos sirve `(a+b-1)/b`; con negativos usar la función anterior.

1.3.3 Trabajo con ángulos y trigonometría

Usar cuando aparecen triángulos, rotaciones, pendientes, direcciones o coordenadas polares. C++ recibe y devuelve ángulos en **radianes**.

Para un triángulo rectángulo:

$$\sin(\theta) = \frac{CO}{H}, \qquad \cos(\theta) = \frac{CA}{H}, \qquad \tan(\theta) = \frac{CO}{CA} = \frac{\sin(\theta)}{\cos(\theta)}.$$

El signo depende del cuadrante:

Cuadrante	rango	x	y	signos positivos
I	$0 < \theta < \pi/2$	$+$	$+$	$\sin, \cos, \tan$
II	$\pi/2 < \theta < \pi$	$-$	$+$	$\sin$
III	$\pi < \theta < 3\pi/2$	$-$	$-$	$\tan$
IV	$3\pi/2 < \theta < 2\pi$	$+$	$-$	$\cos$

Ángulos notables

Grados	Radianes	sin	cos	tan
$0°$	0	0	1	0
$30°$	$\pi/6$	$1/2$	$\sqrt{3}/2$	$1/\sqrt{3}$
$45°$	$\pi/4$	$\sqrt{2}/2$	$\sqrt{2}/2$	1
$60°$	$\pi/3$	$\sqrt{3}/2$	$1/2$	$\sqrt{3}$
$90°$	$\pi/2$	1	0	no definida
$180°$	π	0	-1	0
$270°$	$3\pi/2$	-1	0	no definida
$360°$	2π	0	1	0

Identidades frecuentes

$$\begin{aligned}
\sin^2\theta + \cos^2\theta &= 1, \\
\sin(-\theta) &= -\sin\theta, \qquad & \cos(-\theta) &= \cos\theta, \\
\sin(a+b) &= \sin a\cos b + \cos a\sin b, \\
\cos(a+b) &= \cos a\cos b - \sin a\sin b, \\
\sin(2a) &= 2\sin a\cos a, \\
\cos(2a) &= \cos^2 a - \sin^2 a = 1 - 2\sin^2 a.
\end{aligned}$$

La periodicidad es 2π para seno y coseno, y π para tangente.

Operación	`double`	`long double`
seno, coseno, tangente	`sin/cos/tan`	`sinl/cosl/tanl`
inversas	`asin/acos/atan`	`asinl/acosl/atanl`
ángulo de (x, y)	`atan2(y,x)`	`atan2l(y,x)`
hipotenusa	`hypot(x,y)`	`hypotl(x,y)`

```cpp
const long double PI = acosl(-1.0L);

long double degrees_to_radians(long double degrees) {
    return degrees * PI / 180.0L;
}

long double radians_to_degrees(long double radians) {
    return radians * 180.0L / PI;
}

long double normalize_angle(long double angle) { // [0, 2*pi)
    angle = fmodl(angle, 2.0L * PI);
    if (angle < 0) angle += 2.0L * PI;
    return angle;
}

long double signed_angle(long double angle) { // [-pi, pi)
    angle = normalize_angle(angle);
    if (angle >= PI) angle -= 2.0L * PI;
    return angle;
}

long double safe_acos(long double value) {
    return acosl(clamp(value, -1.0L, 1.0L));
}

long double safe_asin(long double value) {
    return asinl(clamp(value, -1.0L, 1.0L));
```

```
}

void trigonometry_example(long double degrees) {
    long double angle = degrees_to_radians(degrees);
    long double sine = sinl(angle);
    long double cosine = cosl(angle);
    long double tangent = tanl(angle);
    long double direction = atan2l(sine, cosine); // primero y, luego x
    (void)tangent; (void)direction;
}
```

Conversión entre grados y radianes

Una vuelta completa cumple

$$360^\circ = 2\pi \text{ rad}, \qquad 180^\circ = \pi \text{ rad}.$$

Por lo tanto:

$$rad = grados\frac{\pi}{180}, \qquad grados = rad\frac{180}{\pi}.$$

En C++17 no existe `std::numbers::pi`; la forma portable es `acosl(-1.0L)`. `M_PI` no es estándar y puede no estar definido.

Normalización y distancia angular

El mismo rayo puede representarse con infinitos valores: θ, $\theta+2\pi$, $\theta-4\pi$, etc. Normalizar evita errores al comparar direcciones. Para saber cuánto girar, no restar ángulos sin considerar el salto entre 0 y 2π.

```
// Giro antihorario desde 'from' hasta 'to', en [0, 2*pi).
long double counterclockwise_angle(long double from, long double to) {
    return normalize_angle(to - from);
}

// Menor distancia entre dos direcciones, en [0, pi].
long double angle_distance(long double first, long double second) {
    return fabsl(signed_angle(second - first));
}

// Giro firmado mas corto desde 'from' hasta 'to', en [-pi, pi).
long double signed_angle_difference(long double from, long double to) {
    return signed_angle(to - from);
}

// target pertenece al arco antihorario cerrado start -> finish.
bool angle_between_ccw(long double start, long double target,
                       long double finish, long double epsilon = 1e-12L) {
    return counterclockwise_angle(start, target)
        <= counterclockwise_angle(start, finish) + epsilon;
}
```

Ejemplo: entre 350° y 10° hay solamente 20°, no 340°.

`atan2`: dirección y cuadrantes

Para un vector (x, y):

$$r = \sqrt{x^2 + y^2}, \qquad \theta = \text{atan2}(y, x).$$

Siempre escribir `atan2(y,x)`: el argumento vertical va primero. `atan(y/x)` pierde el cuadrante, divide por cero cuando $x = 0$ y no puede distinguir $(1, 1)$ de $(-1, -1)$.

Vector	`atan2(y,x)`	grados
$(1, 0)$	0	0°
$(0, 1)$	$\pi/2$	90°
$(-1, 0)$	π	180°
$(0, -1)$	$-\pi/2$	-90°

Cuidado: La dirección del vector $(0, 0)$ no está definida. Algunas implementaciones devuelven cero para `atan2(0,0)`, pero no debe usarse como un ángulo geométrico válido.

Coordenadas polares y rotaciones

Conversión polar a cartesiana:

$$x = r\cos\theta, \qquad y = r\sin\theta.$$

Rotar (x, y) un ángulo θ en sentido antihorario:

$$x' = x\cos\theta - y\sin\theta, \qquad y' = x\sin\theta + y\cos\theta.$$

```
pair<long double,long double> polar_to_cartesian(
        long double radius, long double angle) {
    return {radius * cosl(angle), radius * sinl(angle)};
}

pair<long double,long double> rotate_coordinates(
        long double x, long double y, long double angle) {
    long double cosine = cosl(angle), sine = sinl(angle);
    return {x * cosine - y * sine,
            x * sine + y * cosine};
}

```

```cpp
pair<long double,long double> cartesian_to_polar(
        long double x, long double y) {
    return {hypotl(x, y), atan2l(y, x)}; // {radio, angulo}
}
```

Para rotar alrededor de un centro C, trasladar $P-C$, rotar y volver a sumar C. Varias rotaciones suman sus ángulos; una rotación no cambia distancias.

Ángulo entre vectores

Para vectores u y v no nulos:

$$\cos\theta = \frac{u\cdot v}{|u||v|}, \qquad \theta = \operatorname{atan2}(|u\times v|, u\cdot v).$$

La versión con `atan2` suele ser más estable y conserva mejor ángulos cercanos a 0 o π.

```cpp
long double angle_between_vectors(long double ux, long double uy,
                                  long double vx, long double vy) {
    long double dot = ux * vx + uy * vy;
    long double cross_value = ux * vy - uy * vx;
    assert(hypotl(ux, uy) > 0 && hypotl(vx, vy) > 0);
    return atan2l(fabsl(cross_value), dot); // [0, pi]
}

// Angulo orientado u -> v: positivo antihorario, en [-pi, pi].
long double oriented_angle(long double ux, long double uy,
                           long double vx, long double vy) {
    return atan2l(ux * vy - uy * vx, ux * vx + uy * vy);
}
```

Si sólo importa comparar ángulos o saber si son agudos/obtusos, evitar la trigonometría: $u\cdot v>0$ indica ángulo agudo, $=0$ recto y <0 obtuso.

Triángulos cualesquiera

Si los lados a, b, c están opuestos a los ángulos A, B, C:

Ley de cosenos:

$$c^2 = a^2+b^2-2ab\cos C, \qquad \cos C = \frac{a^2+b^2-c^2}{2ab}.$$

Ley de senos:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R,$$

donde R es el radio de la circunferencia circunscrita.

Área:

$$s = \frac{a+b+c}{2}, \qquad K = \sqrt{s(s-a)(s-b)(s-c)}.$$

```cpp
long double angle_opposite_side(long double a, long double b,
                                long double opposite) {
    assert(a > 0 && b > 0);
    long double cosine = (a * a + b * b - opposite * opposite)
                         / (2.0L * a * b);
    return safe_acos(cosine);
}

long double opposite_side(long double a, long double b,
                          long double included_angle) {
    long double squared = a * a + b * b
                          - 2.0L * a * b * cosl(included_angle);
    return sqrtl(max(0.0L, squared));
}

bool valid_triangle(long double a, long double b, long double c,
                    long double epsilon = 1e-12L) {
    return a > 0 && b > 0 && c > 0
        && a + b > c + epsilon
        && a + c > b + epsilon
        && b + c > a + epsilon;
}

long double triangle_area_heron(long double a, long double b,
                                long double c) {
    assert(valid_triangle(a, b, c));
    long double semiperimeter = (a + b + c) / 2.0L;
    long double product = semiperimeter * (semiperimeter - a)
                          * (semiperimeter - b) * (semiperimeter - c);
    return sqrtl(max(0.0L, product));
}
```

Cuidado: La ley de senos puede tener dos soluciones cuando se conocen dos lados y un ángulo no comprendido. La ley de cosenos determina directamente el lado opuesto al ángulo comprendido. Herón puede perder precisión en triángulos casi degenerados; con coordenadas, preferir producto cruzado.

Circunferencias, arcos y sectores

Con radio r y ángulo central θ en radianes:

$$\text{arco} = r\theta, \qquad \text{sector} = \frac{1}{2}r^2\theta, \qquad \text{cuerda} = 2r\sin(\theta/2).$$

```cpp
long double arc_length(long double radius, long double angle) {
    return radius * fabsl(angle);
}
```

```cpp

long double sector_area(long double radius, long double angle) {
    return radius * radius * fabsl(angle) / 2.0L;
}

long double chord_length(long double radius, long double angle) {
    return 2.0L * radius * sinl(fabsl(angle) / 2.0L);
}
```

Polígonos regulares

Para un polígono regular de $n \geq 3$ lados:

$$\sum \text{interiores} = (n-2)\pi, \quad \text{interior} = \frac{(n-2)\pi}{n}, \quad \text{exterior} = \frac{2\pi}{n}.$$

Si R es el circunradio y s el lado:

$$s = 2R\sin(\pi/n), \qquad \text{área} = \frac{nR^2\sin(2\pi/n)}{2}.$$

Convenciones: matemática, brújula y pantalla

Antes de usar un ángulo, identificar la convención del problema:

- matemática: 0 apunta a $+x$ y crece antihorario;
- rumbo de brújula: 0 apunta al norte y crece horario;
- matrices/pantallas: la fila o coordenada y suele crecer hacia abajo, invirtiendo visualmente la orientación.

```cpp
long double bearing_to_math_angle(long double bearing) {
    return normalize_angle(PI / 2.0L - bearing);
}

long double math_angle_to_bearing(long double angle) {
    return normalize_angle(PI / 2.0L - angle);
}

// Direccion en una grilla donde row crece hacia abajo.
long double screen_direction(long double delta_column,
                             long double delta_row) {
    return atan2l(-delta_row, delta_column);
}

// Una recta no orientada es igual tras girar pi: resultado [0, pi).
long double normalize_line_angle(long double angle) {
    angle = fmodl(angle, PI);
    if (angle < 0) angle += PI;
    return angle;
}
```

Orden polar exacto sin funciones trigonométricas

Para ordenar muchos vectores por ángulo, calcular `atan2` para cada uno es más lento y agrega error flotante. Se puede separar el plano en dos mitades y comparar con producto cruzado. El siguiente orden comienza en $+x$ y avanza antihorario; los colineales se ordenan por longitud.

```cpp
int polar_half(ll x, ll y) {
    return (y < 0 || (y == 0 && x < 0)) ? 1 : 0;
}

bool polar_less(pair<ll,ll> first, pair<ll,ll> second) {
    auto [ax, ay] = first;
    auto [bx, by] = second;
    int half_a = polar_half(ax, ay), half_b = polar_half(bx, by);
    if (half_a != half_b) return half_a < half_b;
    i128 cross_value = (i128)ax * by - (i128)ay * bx;
    if (cross_value != 0) return cross_value > 0;
    i128 length_a = (i128)ax * ax + (i128)ay * ay;
    i128 length_b = (i128)bx * bx + (i128)by * by;
    return length_a < length_b;
}
```

Cuidado: Excluir el vector $(0,0)$ o decidir explícitamente dónde ubicarlo: no posee dirección. El comparador debe ser estricto; no introducir epsilon en un comparador de `sort`, `set` o `map`.

Proyección, reflexión y bisectriz

La proyección del vector v sobre una dirección no nula d es

$$\text{proj}_d(v) = d\frac{v \cdot d}{d \cdot d}.$$

Si n es una normal de una recta/espejo, la reflexión es

$$\text{reflect}(v) = v - 2\,\text{proj}_n(v).$$

La bisectriz interna de dos vectores no opuestos apunta en la suma de sus versiones unitarias.

```cpp
pair<long double,long double> project_vector(
        long double vx, long double vy,
        long double dx, long double dy) {
```

```cpp
    long double norm_squared = dx * dx + dy * dy;
    assert(norm_squared > 0);
    long double scale = (vx * dx + vy * dy) / norm_squared;
    return {dx * scale, dy * scale};
}

pair<long double,long double> reflect_vector(
        long double vx, long double vy,
        long double normal_x, long double normal_y) {
    auto projection = project_vector(vx, vy, normal_x, normal_y);
    return {vx - 2.0L * projection.first,
            vy - 2.0L * projection.second};
}

optional<pair<long double,long double>> internal_angle_bisector(
        long double ax, long double ay,
        long double bx, long double by) {
    long double norm_a = hypotl(ax, ay), norm_b = hypotl(bx, by);
    if (norm_a == 0 || norm_b == 0) return nullopt;
    long double x = ax / norm_a + bx / norm_b;
    long double y = ay / norm_a + by / norm_b;
    long double norm = hypotl(x, y);
    if (norm < 1e-18L) return nullopt; // direcciones opuestas
    return pair<long double,long double>{x / norm, y / norm};
}
```

Para reflejar un punto respecto de una recta, proyectar el vector desde un punto de la recta hasta el punto original y usar la misma identidad. Para la bisectriz externa, restar los vectores unitarios en lugar de sumarlos.

Funciones relacionadas de `<cmath>`

- `sinh/cosh/tanh`: funciones hiperbólicas; no describen ángulos circulares y aparecen ocasionalmente en fórmulas analíticas.
- `fmod(a,b)` conserva el signo de a; para normalizar ángulos negativos hay que sumar el período, como hace `normalize_angle`.
- `remainder(a,b)` devuelve el residuo respecto del múltiplo más cercano y puede ser útil para obtener un ángulo firmado, aunque su tratamiento de empates suele ser menos intuitivo en competencia.
- `copysign(magnitude,sign)` copia el signo sin bifurcar; `isfinite/isnan/isinf` permiten detectar resultados inválidos.

Rangos de retorno:

- `asin`: $[-\pi/2, \pi/2]$; `acos`: $[0, \pi]$.
- `atan`: $(-\pi/2, \pi/2)$; `atan2(y,x)`: $[-\pi, \pi]$.
- `atan2` distingue los cuatro cuadrantes y funciona cuando $x = 0$; para direcciones casi siempre es mejor que `atan(y/x)`.

Cuidado: Por errores de redondeo, un coseno calculado puede quedar apenas fuera de $[-1, 1]$ y hacer que `acos` devuelva NaN; aplicar `clamp`. La tangente no está definida cuando $\cos(\theta) = 0$.

Checklist de competencia:

- confirmar si la entrada está en grados o radianes;
- usar `atan2(y,x)`, no `atan(y/x)`;
- normalizar antes de comparar direcciones que cruzan $0/2\pi$;
- aplicar `clamp` antes de `asin/acos`;
- evitar trigonometría si dot/cross permiten decidir exactamente;
- usar `long double` y un epsilon acorde a las coordenadas;
- no comparar resultados flotantes con `==`.

Cada función trigonométrica y cada helper de esta sección cuesta $O(1)$ tiempo y $O(1)$ memoria.

1.4 Template Python 3

Usar Python cuando la complejidad tiene margen, predominan enteros grandes, strings, diccionarios o implementación corta. En problemas ajustados de $O(n \log n)$ pesado, grafos enormes o DP con millones de estados, preferir C++.

```python
import sys
from collections import Counter, defaultdict, deque
from functools import lru_cache
from heapq import heappop, heappush
from bisect import bisect_left, bisect_right
from math import gcd, isqrt, lcm, inf

input = sys.stdin.buffer.readline

def ints():
    return map(int, input().split())

def solve():
    n = int(input())
    values = list(ints())
    assert len(values) == n
    print(sum(values))

if __name__ == "__main__":
    solve()
```

Atajos importantes:

- `math.isqrt(n)` da $\lfloor \sqrt{n} \rfloor$ exactamente.
- `pow(a,b,mod)` hace potencia modular y `pow(a,-1,mod)` calcula el inverso si existe.
- `sys.setrecursionlimit(...)` sólo amplía el límite lógico; para DFS muy profundo suele ser más seguro usar una pila iterativa.

- Evitar `pop(0)` en listas; usar `deque.popleft()`.
- Usar input().decode() si se toma como b antes del input

1.5 Imports y utilidades de Python

```
import sys
from collections import Counter, defaultdict, deque
from functools import lru_cache
from heapq import heappop, heappush
from bisect import bisect_left, bisect_right
from math import gcd, isqrt, lcm, inf
# sys.setrecursionlimit(1_000_000) no se limite
input = sys.stdin.buffer.readline
```

1.5.1 Entrada rapida

```
n = int(input())
values = list(map(int, input().split()))
```

Lee una linea en $O(L)$, donde L es su cantidad de bytes.

1.5.2 Counter

Cuenta frecuencias.

```
frequency = Counter([2, 1, 2, 3, 2])
print(frequency[2])       # 3
print(frequency.most_common(1)) # [(2, 3)]
```

Construirlo cuesta $O(n)$; consultar una frecuencia cuesta $O(1)$ promedio.

1.5.3 defaultdict

Crea un valor inicial automaticamente para claves nuevas.

```
graph = defaultdict(list)
graph[1].append(4)
graph[1].append(7)
print(graph[1]) # [4, 7]

count = defaultdict(int)
count[5] += 1
print(count[5]) # 1
```

Acceso e insercion: $O(1)$ promedio.

1.5.4 deque

Cola doble: insertar o quitar en ambos extremos.

```
q = deque([2, 3])
q.append(4)
q.appendleft(1)
print(q.popleft()) # 1
print(q.pop())     # 4
```

`append`, `appendleft`, `pop` y `popleft` cuestan $O(1)$.

1.5.5 lru_cache

Memoiza resultados de una funcion recursiva.

```
@lru_cache(None) # None almacena y si pongo nro ultimos x
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(10)) # 55
```

Cada estado se calcula una vez. En este ejemplo: $O(n)$ tiempo y $O(n)$ memoria.

1.5.6 heapq

Heap minimo: permite obtener siempre el menor elemento.

```
heap = []
heappush(heap, 5)
heappush(heap, 2)
heappush(heap, 7)

print(heappop(heap)) # 2
print(heappop(heap)) # 5
```

`heappush` y `heappop` cuestan $O(\log n)$. Para un max-heap, guardar los valores negados.

1.5.7 bisect

Busca posiciones en un vector ordenado.

```
values = [1, 2, 2, 2, 5]

print(bisect_left(values, 2))  # 1
print(bisect_right(values, 2)) # 4
print(bisect_left(values, 4))  # 4
```

`bisect_left` busca la primera posicion posible; `bisect_right`, la posterior a las repeticiones. Ambos cuestan $O(\log n)$. Insertar en una lista sigue costando $O(n)$.

1.5.8 Matematica

```python
print(gcd(18, 24)) # 6
print(lcm(6, 8))   # 24
print(isqrt(20))   # 4
print(inf > 10**100) # True
a = int("ABCDEF", 16)
b = int("123456", 16)

result = a * b
print(format(result, "X"))
```

`gcd` cuesta $O(\log(\min(a,b)))$; `lcm` tiene costo equivalente. `isqrt(n)` devuelve $\lfloor\sqrt{n}\rfloor$ sin errores de punto flotante. `inf` representa infinito para inicializar minimos o distancias.

1.6 Tipos, límites y overflow

- `int`: aproximadamente $[-2\cdot10^9, 2\cdot10^9]$.
- `long long`: aproximadamente $[-9\cdot10^{18}, 9\cdot10^{18}]$.
- `double`: unas 15 cifras decimales de precisión.
- `long double`: precisión dependiente de la plataforma.
- `__int128`: entero no estándar disponible en GCC/Clang.

Promover *antes* de operar:

```cpp
ll promoted_product(int a, int b) { return 1LL * a * b; }
bool square_leq(ll x, ll n) { return x <= n / x; } // x > 0

void print_i128(i128 x) {
    if (x == 0) { cout << 0; return; }
    bool negative = x < 0;
    u128 value = negative ? (u128)(-(x + 1)) + 1 : (u128)x;
    if (negative) cout << '-';
    string s;
    while (value) {
        s.push_back(char('0' + value % 10));
        value /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}
```

El overflow de enteros con signo es comportamiento indefinido en C++.

1.7 Sintaxis C++ más usada

1.7.1 Funciones, referencias y lambdas

Pasar objetos grandes por referencia constante si no se modifican; usar referencia normal cuando la función debe modificarlos.

```cpp
ll sum_values(const vector<int> &values) {
    ll result = 0;
    for (int value : values) result += value;
    return result;
}

void normalize(vector<int> &values) {
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
}

int apply_operation(int a, int b, const function<int(int,int)> &operation) {
    return operation(a, b);
}

void lambda_example() {
    int offset = 5;
    auto add_offset = [offset](int value) { return value + offset; };
    assert(add_offset(3) == 8);
}
```

1.7.2 Pair, tuple, array y optional

```cpp
void aggregate_types_example() {
    pair<int,string> person{20, "Ada"};
    auto [age, name] = person;

    tuple<int,int,ll> edge{0, 1, 15};
    auto [from, to, weight] = edge;

    array<int, 4> fixed{1, 2, 3, 4};
    optional<int> answer;
    if (!fixed.empty()) answer = fixed.front();
    if (answer) cout << *answer << '\n';

    (void)age; (void)name; (void)from; (void)to; (void)weight;
}
```

Cuidado: Comparar `int` con `size()` mezcla tipos firmados y sin signo. En competencia suele ser más simple convertir el tamaño una vez a `int`.

1.8 STL esencial

Estructura	Operaciones principales	Complejidad
`vector`	acceso / push al final	$O(1)$ / amortizado $O(1)$
`deque`	push/pop en ambos extremos	$O(1)$
`stack, queue`	push/pop/top/front	$O(1)$
`priority_queue`	push/pop/top	$O(\log n)$ / $O(1)$
`set, map`	find/insert/erase por clave	$O(\log n)$
`unordered_*`	find/insert/erase	promedio $O(1)$

```cpp
void stl_examples(vector<ll> &v, ll x) {
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());

    auto lo = lower_bound(v.begin(), v.end(), x); // primero >= x
    auto hi = upper_bound(v.begin(), v.end(), x); // primero > x
    if (lo != v.end()) { ll value = *lo; (void)value; }
    (void)hi;

    priority_queue<int> max_heap;
    priority_queue<int, vector<int>, greater<int>> min_heap;

    ll total = accumulate(v.begin(), v.end(), 0LL);
    (void)max_heap; (void)min_heap; (void)total;
}
```

Cuidado: `reserve(n)` reserva capacidad pero no crea elementos. Un arreglo local suele vivir en el stack; uno global o `static`, no.

1.9 Contenedores comunes

1.9.1 Vector y string

```cpp
void vector_and_string_examples() {
    vector<int> values(5, 0);       // cinco ceros
    values.push_back(7);
    values.pop_back();
    values.resize(10, -1);          // crea elementos
    values.reserve(100);            // solo reserva capacidad
    values.insert(values.begin() + 2, 9); // O(n)
    values.erase(values.begin() + 2);     // O(n)

    string text = "competitive";
    text += " programming";
    string part = text.substr(0, 11);
    size_t position = text.find("program");
    if (position != string::npos) text.replace(position, 7, "contest");
    (void)part;
}
```

1.9.2 Set, multiset, map y unordered

```cpp
void associative_container_examples() {
    set<int> ordered{5, 1, 5}; // {1,5}
    ordered.insert(3);
    bool exists = ordered.count(5) != 0;
    auto first_at_least = ordered.lower_bound(4);
    if (first_at_least != ordered.end()) ordered.erase(first_at_least);

    multiset<int> repeated{2, 2, 3};
    auto occurrence = repeated.find(2);
    if (occurrence != repeated.end()) repeated.erase(occurrence); // una copia
    repeated.erase(2); // todas las copias restantes de 2

    map<string,int> frequency;
    ++frequency["duck"];
    auto found = frequency.find("duck");
    if (found != frequency.end()) cout << found->second << '\n';

    unordered_map<int,int> fast_frequency;
    fast_frequency.reserve(1000);
    fast_frequency.max_load_factor(0.7);
    ++fast_frequency[10];
    (void)exists;
}
```

1.9.3 Stack, queue, deque y priority queue

```cpp
void queue_examples() {
    stack<int> stack_values;
    stack_values.push(1); stack_values.push(2);
    int top = stack_values.top(); stack_values.pop();

    queue<int> pending;
    pending.push(1); pending.push(2);
    int front = pending.front(); pending.pop();

    deque<int> double_ended;
    double_ended.push_front(1); double_ended.push_back(2);
    double_ended.pop_front(); double_ended.pop_back();

    priority_queue<int> maximum_heap;
    priority_queue<int, vector<int>, greater<int>> minimum_heap;
    maximum_heap.push(5); minimum_heap.push(5);
    (void)top; (void)front;
```

```
}
```

Contenedor	Acceso/búsqueda	Inserción/borrado
`vector` final	$O(1)$	amortizado $O(1)$
`deque` extremos	$O(1)$	$O(1)$
`set/map`	$O(\log n)$	$O(\log n)$
`unordered_*`	promedio $O(1)$	promedio $O(1)$
`priority_queue` top	$O(1)$	$O(\log n)$

1.10 Algoritmos STL comunes

```
void common_algorithms(vector<int> values) {
    sort(values.begin(), values.end());
    sort(values.rbegin(), values.rend());
    stable_sort(values.begin(), values.end()); // conserva el orden relativo de
    elementos que empatan (noo mas rapido si mas memo)

    int minimum = *min_element(values.begin(), values.end());
    int maximum = *max_element(values.begin(), values.end());
    auto [minimum_it, maximum_it] = minmax_element(values.begin(), values.end())
    ;

    ll sum = accumulate(values.begin(), values.end(), 0LL);
    int amount = count(values.begin(), values.end(), 5); // Cuantas veces
    aparece 5
    int positives = count_if(values.begin(), values.end(),
                             [](int value) { return value > 0; });
    bool all_positive = all_of(values.begin(), values.end(),
                               [](int value) { return value > 0; });

    reverse(values.begin(), values.end());
    rotate(values.begin(), values.begin() + values.size() / 2, values.end()); //
     {1, 2, 3, 4, 5, 6} -> {4, 5, 6, 1, 2, 3}
    (void)minimum; (void)maximum; (void)minimum_it; (void)maximum_it;
    (void)sum; (void)amount; (void)positives; (void)all_positive; //: no la uso;
     es intencional
}
```

1.10.1 Bounds, duplicados y selección

```
void ordered_range_algorithms(vector<int> values, int target) {
    sort(values.begin(), values.end());
    auto first_equal = lower_bound(values.begin(), values.end(), target);
    auto after_equal = upper_bound(values.begin(), values.end(), target);
    int occurrences = after_equal - first_equal;
    bool exists = binary_search(values.begin(), values.end(), target);

    values.erase(unique(values.begin(), values.end()), values.end());
    if (!values.empty()) {
        int middle = values.size() / 2;
        nth_element(values.begin(), values.begin() + middle, values.end());
        int median = values[middle];
        (void)median;
    }
    (void)occurrences; (void)exists;
}
```

1.10.2 Permutaciones y sumas acumuladas

```
void sequence_algorithms() {
    vector<int> permutation{1, 2, 3};
    do {
        // procesar permutation
    } while (next_permutation(permutation.begin(), permutation.end()));

    // partial_sum acumula con el tipo de entrada, no con el de prefix.
    // Usar ll; cada suma parcial debe entrar en ese tipo.
    vector<ll> values(5);
    iota(values.begin(), values.end(), 10LL); // 10,11,12,13,14
    vector<ll> prefix(values.size());
    partial_sum(values.begin(), values.end(), prefix.begin());
}
```

Cuidado: `lower_bound`, `upper_bound` y `binary_search` requieren el rango ordenado con el mismo comparador.

1.11 Comparadores, bits y aleatoriedad

1.11.1 Orden personalizado

El comparador debe ser estricto: nunca puede decir que $a < a$. No usar una tolerancia EPS como comparador directo de `set` o `sort`.

```
struct BySecondThenFirst {
    bool operator()(const pair<int,int> &a, const pair<int,int> &b) const {
        return tie(a.second, a.first) < tie(b.second, b.first);
    }
};

void comparator_example(vector<pair<int,int>> &items) {
    sort(items.begin(), items.end(), BySecondThenFirst{});
    sort(items.begin(), items.end(), [](const auto &a, const auto &b) {
        if (a.first != b.first) return a.first > b.first;
        return a.second < b.second;
```

```
    });
}
```

1.11.2 Operaciones de bits

Consulta	oper	NEUT
XOR	^	0
OR	|	0
AND	&	(~0)

```
int count_bits(unsigned long long value) {
    return __builtin_popcountll(value);
} //cuenta cuantos bits estan en 1

int least_significant_bit_index(unsigned long long value) {
    assert(value != 0);
    return __builtin_ctzll(value);
} // devuelve cuantos ceros hay al final, o sea el indice del bit 1 mas bajo

int most_significant_bit_index(unsigned long long value) {
    assert(value != 0);
    return 63 - __builtin_clzll(value);
} // obtiene el indice del bit 1 mas alto

unsigned long long lowest_set_bit(unsigned long long value) {
    return value & (~value + 1); // tambien value & -value para unsigned
} // devuelve solo el bit 1 mas bajo

void bitset_example() {
    bitset<64> bits; // requiere <bitset>; inicialmente todos en 0
    bits.set(5);    // pone el bit 5 en 1
    bits.flip(7);   // invierte el bit 7
    bits.reset(5);  // pone el bit 5 en 0
    bool seventh = bits.test(7); // consulta el bit de indice 7
    size_t active = bits.count(); // cuenta bits en 1
    cout << seventh << '\n'; // 1
    cout << active << '\n';  // 1
}
```

Cuidado: `__builtin_clz(0)` y `__builtin_ctz(0)` son indefinidos. Para `long long`, usar las versiones terminadas en `ll`.

1.11.3 Random y shuffle

```
mt19937_64& random_engine() {
    static mt19937_64 engine(chrono::steady_clock::now()
                             .time_since_epoch().count());
    return engine;
}

ll random_integer(ll left, ll right) {
    return uniform_int_distribution<ll>(left, right)(random_engine());
}

void shuffle_values(vector<int> &values) {
    shuffle(values.begin(), values.end(), random_engine());
}
```

1.11.4 Errores frecuentes

- `map[key]` y `unordered_map[key]` insertan la clave si no existe; usar `find` para consultar sin modificar.
- No desreferenciar `end()`, ni llamar `front/back/top` sobre un contenedor vacío.
- `erase(value)` en `multiset` borra todas las copias; `erase(find(value))` borra una.
- Un comparador de `sort/set/map` debe cumplir orden estricto.
- Evitar `endl` salvo que se necesite forzar el flush; usar `'\n'`.
- En expresiones modulares, normalizar negativos con $((x \bmod m) + m) \bmod m$.

1.12 Entrada y salida

```
void read_test_cases() {
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        vector<ll> a(n);
        for (ll &x : a) cin >> x;
    }
}

void read_until_eof() {
    int a, b;
    while (cin >> a >> b) cout << a + b << '\n';
}

vector<string> read_grid() {
    int n, m; cin >> n >> m;
    vector<string> grid(n);
    for (string &row : grid) cin >> row;
    return grid;
}

string read_line_after_token() {
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string line;
    getline(cin, line);
```

```
    return line;
}
```

Para salida decimal:

```
void print_decimal(long double answer) {
    cout << fixed << setprecision(10) << answer << '\n';
}
```

1.13 Parsing seguro

```
bool parse_int(const string &s, int &value) {
    if (s.empty()) return false;
    const char *first = s.data();
    const char *last = first + s.size();
    auto [ptr, ec] = from_chars(first, last, value);
    return ec == errc{} && ptr == last;
}

bool is_digit(char c) { return '0' <= c && c <= '9'; }
```

`from_chars` detecta valores fuera del rango y no acepta espacios sobrantes. Para dividir una línea en tokens puede usarse `istringstream`.

1.14 Letras a posiciones

```
char c = 'C';

int zero_indexed = c - 'A';     // A -> 0, B -> 1, ..., Z -> 25
int one_indexed = c - 'A' + 1; // A -> 1, B -> 2, ..., Z -> 26

char lower = 'c';
int lower_indexed = lower - 'a'; // a -> 0, ..., z -> 25
```

```
c = 'C'
zero_indexed = ord(c) - ord('A')      # A -> 0
one_indexed = ord(c) - ord('A') + 1   # A -> 1

lower = 'c'
lower_indexed = ord(lower) - ord('a') # a -> 0
```

2. Técnicas

2.1 Búsqueda binaria

Cuándo usar: Existe una condición monótona falso–verdadero o verdadero–falso y se busca el primer/último valor que la satisface, incluso sobre la respuesta.

Busca la primera posición donde una condición monótona pasa de falsa a verdadera.

Complejidad: Dominio discreto de n candidatos: $O(\log(n+1))$ evaluaciones; si cada chequeo cuesta C, tiempo $O(C\log(n+1))$. Para reales con ancho inicial W y tolerancia ε, $O(\log(1+W/\varepsilon))$ evaluaciones hasta alcanzar la tolerancia o el límite de precisión. Memoria adicional $O(1)$ aparte del chequeo.

```
ll l = start; // start y end son int; los limites pueden salir de ese dominio.
ll r = end;
while (l <= r) {
    ll m = l + (r - l) / 2;
    if (can((int)m)) {
        // La condicion se cumple en m, ajustar el limite superior
        r = m - 1;
    } else {
        // La condicion no se cumple en m, ajustar el limite inferior
       l = m + 1;
    }
}
// Esto es en caso de que la condicion inicialmente sea falsa y luego verdadera.
     Si la condicion es verdadero--falso, se debe invertir los contenidos del if
    -else
```

El algoritmo anterior permite usar la busqueda binaria unicamente en un dominio discreto. En este caso r nos da el ultimo valor que no cumple la condicion y l el menor valor que si la cumple. El dominio inicial es de `int`, pero los límites se guardan en `long long`: esto permite calcular el ancho completo y representar `end+1` o `start-1` sin desbordar. Si no hay ningún verdadero, $l =$ `end` $+1$; si todos son verdaderos, $r =$ `start` -1. Para reales, mantener un intervalo con `can(l)==false` y `can(r)==true`. Usar la tolerancia pedida por el enunciado y cortar si el punto medio coincide con un extremo para evitar ciclos por redondeo.

```
double l = start;
double r = end;
double eps = 1e-6; // Ajustar al error permitido por el enunciado.
while (r - l > eps) {
    double m = l + (r - l) / 2.0;
    if (m == l || m == r) break; // Limite de precision de double.
    if (can(m)) {
        // La condicion se cumple para m, ajustamos el limite superior
        r = m;
    } else {
       // La condicion no se cumple para m, ajustamos el limite inferior
       l = m;
    }
}
// r cumple la condicion; l no la cumple.
```

El ancho se divide por dos en cada iteración. La precisión final también está limitada por la representación de `double`: el corte por redondeo puede detener la búsqueda antes de alcanzar `eps`. Si se permite error relativo, adaptar la tolerancia a la escala del resultado.

2.2 Sumas parciales

Cuándo usar: Arreglo estático con muchas sumas de rango; también para transformar restricciones de subarreglos y construir diferencias.

Usar un vector de tamaño $n+1$ elimina el caso especial cuando $l = 0$. Los rangos son semiabiertos $[l, r)$.

Complejidad: construcción $O(n)$; consulta $O(1)$; memoria $O(n)$.

```
struct PrefixSum {
    vector<ll> pref;

    explicit PrefixSum(const vector<ll> &a) : pref(a.size() + 1, 0) {
        fore (i,0,SZ(a))
            pref[i + 1] = pref[i] + a[i];
    }

    ll query(int l, int r) const { // suma de [l, r)
        return pref[r] - pref[l];
    }
};
```

En 2D: $P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]$.

2.3 Two pointers y sliding window

Cuándo usar: Subarreglos/pares ordenados con una propiedad monótona al mover los extremos, pudiendo agregar y quitar elementos en $O(1)$.

Si al avanzar el extremo derecho la validez cambia monótonamente, cada puntero avanza a lo sumo n veces.

Con valores no negativos se usa sliding window. Si hay negativos, la segunda variante usa máximos prefijos y búsqueda binaria: una ventana deslizante común no garantiza encontrar la respuesta.

Complejidad: Ventana con valores no negativos y pares sobre un arreglo ordenado: $O(n)$ tiempo y $O(1)$ memoria adicional. Variante con negativos: $O(n\log n)$ tiempo y $O(n)$ memoria adicional.

```cpp
// Longitud maxima con suma <= limit; requiere a[i] >= 0.
// Tiempo O(n), memoria adicional O(1).
int longest_window_nonnegative(const vector<ll> &a, ll limit) {
    ll sum = 0;
    int ans = 0;
    for (int l = 0, r = 0; r < (int)a.size(); ++r) {
        sum += a[r];
        while (l <= r && sum > limit) sum -= a[l++];
        ans = max(ans, r - l + 1);
    }
    return ans;
}

// Pares con suma target en un arreglo ordenado.
bool has_pair_sum(const vector<ll> &a, ll target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        i128 sum = (i128)a[l] + a[r];
        if (sum == target) return true;
        if (sum < target) ++l;
        else --r;
    }
    return false;
}

// Longitud maxima con suma <= limit; admite valores negativos.
// Prefijos y busqueda binaria: tiempo O(n log n), memoria O(n).
int longest_window_signed(const vector<ll> &a, ll limit) {
    vector<ll> maximum_prefix{0};
    vector<int> first_position{0};
    ll prefix = 0;
    int answer = 0;
    for (int r = 0; r < (int)a.size(); ++r) {
        prefix += a[r];
        // Buscamos el menor i con prefix[i] >= prefix[r+1] - limit.
        ll required = prefix - limit;
        auto it = lower_bound(
            maximum_prefix.begin(), maximum_prefix.end(), required
        );
        if (it != maximum_prefix.end()) {
            int k = it - maximum_prefix.begin();
            answer = max(answer, r + 1 - first_position[k]);
        }
        // Guardamos solamente nuevos maximos prefijos.
        if (prefix > maximum_prefix.back()) {
            maximum_prefix.push_back(prefix);
            first_position.push_back(r + 1);
        }
    }
    return answer;
}
```

2.4 Máscaras de bits

Cuándo usar: Representar subconjuntos pequeños, enumerar submáscaras o guardar flags compactos; suele combinarse con DP exponencial.

Complejidad: operaciones de bits $O(1)$; enumerar todos los subconjuntos $O(2^n)$ y todas las submáscaras de todas las máscaras $O(3^n)$; memoria $O(1)$.

Para evitar desplazamientos indefinidos, usar tipos sin signo y verificar que $0 \le k < 64$.

```cpp
using ull = unsigned long long;

// Se debe asegurar que k este en el rango [0,63]
bool has_bit(ull mask, int k) { return (mask >> k) & 1ULL; }
void add_bit(ull &mask, int k) { mask |= 1ULL << k; }
void remove_bit(ull &mask, int k) { mask &= ~(1ULL << k); }
void toggle_bit(ull &mask, int k) { mask ^= 1ULL << k; }
// Las funciones anteriores funcionan con un unico bit
// Sin embargo tambien se puede trabajar con varios bits al mismo tiempo usando
    lo siguiente
bool has_bits(ull mask, ull bits) { return (ull)(bits & mask) == bits; }
void add_bits(ull &mask, ull bits) { mask |= bits; }
void remove_bits(ull &mask, ull bits) { mask &= ~bits; }
void toggle_bits(ull &mask, ull bits) { mask ^= bits; }

vector<ull> nonempty_submasks(ull mask) {
    vector<ull> answer;
    for (ull sub = mask; sub; sub = (sub - 1) & mask)
        answer.push_back(sub);
    return answer;
}
```

Intersección: `a & b`; unión: `a | b`; diferencia: `a & ~b`; diferencia simétrica: `a ^ b`.

2.5 Conversión a base arbitraria

Cuándo usar: Extraer dígitos, trabajar con representaciones posicionales o convertir un entero no negativo a una base entre 2 y 36.

Complejidad: $O(\log_b n)$ tiempo y memoria para producir los dígitos.

Los dígitos se devuelven como enteros, por lo que admite bases mayores que 36. El signo se maneja por separado.

```cpp
pair<bool, vector<int>> to_base(ll value, int base) {
    if (base < 2) throw invalid_argument("base < 2");
    bool negative = value < 0;
```

```
    unsigned long long x = negative
        ? (unsigned long long)(-(value + 1)) + 1
        : (unsigned long long)value;

    vector<int> digits;
    do {
        digits.push_back((int)(x % base));
        x /= base;
    } while (x);
    reverse(digits.begin(), digits.end());
    return {negative, digits};
}
```

```
char digit_to_char(int digit) {
    if (digit < 10) return char('0' + digit);
    return char('A' + digit - 10);
}

string to_base_string(ll value, int base) {
    if (base < 2 || base > 36)
        throw invalid_argument("base fuera de [2,36]");

    auto [negative, digits] = to_base(value, base);

    string result;
    if (negative) result += '-';
    for (int digit : digits)
        result += digit_to_char(digit);

    return result;
}
```

```
int char_to_digit(char c) {
    if ('0' <= c && c <= '9') return c - '0';
    if ('A' <= c && c <= 'Z') return c - 'A' + 10;
    if ('a' <= c && c <= 'z') return c - 'a' + 10;
    return -1;
}

ll from_base(const string& text, int base) {
    ll value = 0;
    for (char c : text) {
        int digit = char_to_digit(c);
        if (digit < 0 || digit >= base)
            throw invalid_argument("digito invalido");
        value = value * base + digit;
    }
    return value;
}
```

2.6 Compresión de coordenadas

Cuándo usar: Los valores son enormes pero sólo importa su orden/igualdad; mapearlos a $[0, k)$ para usar Fenwick, Segment Tree o arreglos.

Reemplaza valores grandes por sus posiciones ordenadas conservando igualdad y orden relativo.

Complejidad: $O(n \log n)$ de construcción; $O(\log n)$ por consulta; memoria $O(n)$.

```
template<class T>
struct Compressor {
    vector<T> values;

    explicit Compressor(vector<T> a) : values(move(a)) {
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
    }

    int id(const T &x) const {
        return lower_bound(values.begin(), values.end(), x) - values.begin();
    }

    const T& value(int id) const { return values[id]; }
    int size() const { return (int)values.size(); }
};
```

2.7 Descomposición por raíz: frecuentes y poco frecuentes

Cuándo usar: Objetos con frecuencias muy distintas donde procesar uno cuesta $O(f)$ o $O(n/f)$; separar pesados y livianos balancea ambas cotas.

Usar cuando objetos con frecuencia f cuestan $O(f)$ o $O(n/f)$ por consulta. Elegir un umbral $B \approx \sqrt{n}$: procesar directamente los objetos con $f < B$ y precomputar respuestas para los que tienen $f \geq B$. Hay como máximo n/B objetos frecuentes.

Complejidad: típicamente preproceso $O(n\sqrt{n})$, consulta $O(\sqrt{n})$ y memoria entre $O(n)$ y $O(n\sqrt{n})$, según lo precomputado.

```
int sqrt_threshold(int total_items) {
    return max(1, (int)sqrt((long double)total_items) + 1);
}

vector<int> frequent_values(const vector<int> &frequency) {
    int total = accumulate(frequency.begin(), frequency.end(), 0);
    int block = sqrt_threshold(total);
```

```
    vector<int> result;
    for (int value = 0; value < (int)frequency.size(); ++value)
        if (frequency[value] >= block) result.push_back(value);
    return result;
}
```

Cuidado: No existe una plantilla universal: escribir primero las dos cotas y verificar que ambas quedan balanceadas en el umbral elegido.

2.8 Meet-in-the-middle: suma de subconjuntos

Cuándo usar: Unos 40 elementos hacen inviable 2^n, pero se pueden enumerar dos mitades y combinar sus sumas. Admite números negativos.

Complejidad: Tiempo $O(n2^{n/2})$ por enumeración, ordenamiento y búsquedas; memoria $O(2^{n/2})$. Con $n = 40$, las dos listas suelen ocupar unos 32 MiB.

```
optional<vector<int>> subset_sum_mitm(const vector<ll> &a, ll target) {
    int n = a.size(), middle = n / 2;
    assert(n <= 40);
    auto enumerate = [&](int begin, int length) {
        vector<pair<ll,int>> sums{{0, 0}};
        sums.reserve(1 << length);
        for (int i = 0; i < length; ++i) {
            int size = sums.size();
            for (int mask = 0; mask < size; ++mask)
                sums.push_back({sums[mask].first + a[begin + i],
                                sums[mask].second | (1 << i)});
        }
        return sums;
    };
    auto left = enumerate(0, middle), right = enumerate(middle, n - middle);
    sort(right.begin(), right.end());
    for (auto [sum, mask] : left) {
        i128 raw = (i128)target - sum;
        if (raw < numeric_limits<ll>::min() || raw > numeric_limits<ll>::max())
    continue;
        ll wanted = (ll)raw;
        auto it = lower_bound(right.begin(), right.end(), pair<ll,int>{wanted,
    0});
        if (it == right.end() || it->first != wanted) continue;
        vector<int> chosen;
        for (int i = 0; i < middle; ++i) if (mask >> i & 1) chosen.push_back(i);
        for (int i = 0; i < n - middle; ++i)
            if (it->second >> i & 1) chosen.push_back(middle + i);
        return chosen;
    }
    return nullopt;
}
```

```
void mitm_example() {
    auto chosen = subset_sum_mitm({7,-3,10,5}, 12);
    if (chosen) for (int i : *chosen) cout << i << ' '; // Una solucion: 0 3.
}
```

Cuidado: Todas las sumas de subconjuntos deben entrar en `long long`. Con $n = 50$ estas dos listas ocuparían aproximadamente 1 GiB: esta plantilla no escala a miles de elementos sin una reducción adicional.

2.9 Constant intervals of a monotone function

```
/**
 * Author: Simon Lindholm
 * Date: 2015-03-20
 * License: CC0
 * Source: me
 * Description: Split a monotone function on [from, to) into a minimal set of
    half-open intervals on which it has the same value.
 *  Runs a callback g for each such interval.
 * Usage: constantIntervals(0, sz(v), [\&](int x){return v[x];}, [\&](int lo,
    int hi, T val){...});
 * Time: O(k\log\frac{n}{k})
 * Status: tested
 */
#pragma once

template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to <= from) return;
  int i = from; auto p = f(i), q = f(to-1);
  rec(from, to-1, f, g, i, p, q);
  g(i, to, q);
}
```

2.10 Union and subtraction of intervals

```
/**
 * Author: Simon Lindholm
 * License: CC0
 * Description: Add and remove intervals from a set of disjoint intervals.
 * Will merge the added interval with any overlapping intervals in the set when
    adding.
 * Intervals are [inclusive, exclusive).
 * Time: O(\log N)
 * Status: stress-tested
 */
#pragma once

set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

2.11 Minimum interval cover

```
/**
 * Author: Johan Sannemo
 * License: CC0
 * Description: Compute indices of smallest set of intervals covering another
    interval.
 * Intervals should be [inclusive, exclusive). To support [inclusive, inclusive
    ],
 * change (A) to add \texttt{|| R.empty()}. Returns empty set on failure (or if
    G is empty).
 * Time: O(N \log N)
 * Status: Tested on kattis:intervalcover
 */
#pragma once

template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

2.12 Discrete ternary search

```
/**
 * Author: Simon Lindholm
 * Date: 2015-05-12
 * License: CC0
 * Source: own work
 * Description:
 * Find the smallest i in $[a,b]$ that maximizes $f(i)$, assuming that $f(a) < \
    dots < f(i) \ge \dots \ge f(b)$.
 * To reverse which of the sides allows non-strict inequalities, change the <
    marked with (A) to <=, and reverse the loop at (B).
 * To minimize $f$, change it to >, also at (B).
 * Usage:
  int ind = ternSearch(0,n-1,[\&](int i){return a[i];});
 * Time: O(\log(b-a))
 * Status: tested
 */
#pragma once

```

```cpp
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

3. Estructuras de datos

3.1 Disjoint Set Union (DSU)

Cuándo usar: Mantener componentes conexas mientras sólo se agregan conexiones; Kruskal, consultas de conectividad y agrupamiento por equivalencia.

Mantiene componentes bajo uniones. Usa compresión de caminos y unión por tamaño.

Complejidad: $O(\alpha(n))$ amortizado por operación; memoria $O(n)$.

```cpp
struct DSU {
    vector<int> parent; // raiz: -tamano

    explicit DSU(int n) : parent(n, -1) {}

    int find(int x) {
        return parent[x] < 0 ? x : parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (parent[a] > parent[b]) swap(a, b);
        parent[a] += parent[b];
        parent[b] = a;
        return true;
    }

    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -parent[find(x)]; }
};

DSU dsu(5);
dsu.unite(0, 1); // {0,1} {2} {3} {4}
dsu.unite(1, 2); // {0,1,2} {3} {4}
cout << dsu.same(0, 2) << '\n'; // 1: estan conectados
cout << dsu.same(0, 4) << '\n'; // 0: no estan conectados
cout << dsu.size(1) << '\n';    // 3
```

3.2 Fenwick Tree (BIT)

Cuándo usar: Sumas prefijas o de rango con actualizaciones puntuales; preferirlo a Segment Tree cuando la operación tiene inversa y alcanza esta interfaz.

Convención: la estructura es 1-indexada. Actualización puntual y suma de prefijo/rango.

Cuidado: `lower_bound` requiere que los valores actuales sean no negativos, para que las sumas prefijas sean monótonas.

Complejidad: $O(\log n)$ por operación; memoria $O(n)$.

```cpp
struct Fenwick {
    int n;
    vector<ll> bit;

    explicit Fenwick(int size) : n(size), bit(size + 1, 0) {}

    void add(int i, ll delta) {
        assert(1 <= i && i <= n);
        for (; i <= n; i += i & -i) bit[i] += delta;
    }

    ll prefix_sum(int i) const { // [1, i]
        ll ans = 0;
        i = min(i, n);
        for (; i > 0; i -= i & -i) ans += bit[i];
        return ans;
    }

    ll query(int l, int r) const { // [l, r]
        if (l > r) return 0;
        return prefix_sum(r) - prefix_sum(l - 1);
    }

    // Menor i en [1,n] con prefix_sum(i) >= target.
    // Requiere frecuencias no negativas. Devuelve n+1 si no existe.
    int lower_bound(ll target) const {
        if (target <= 0) return 1;
        int pos = 0;
        ll accumulated = 0;
        int step = 1;
        while ((step << 1) <= n) step <<= 1;
        for (; step; step >>= 1) {
            int next = pos + step;
            if (next <= n && accumulated + bit[next] < target) {
                pos = next;
                accumulated += bit[next];
            }
        }
        return pos + 1;
    }
};
```

Para range-add/point-query, aplicar diferencias: sumar x en l y $-x$ en $r+1$ (solamente si $r+1 \le n$).

Para XOR

```cpp
struct FenwickXor {
```

```
    int n;
    vector<ll> bit;

    explicit FenwickXor(int size) : n(size), bit(size + 1, 0) {}

    void add(int i, ll value) {
        assert(1 <= i && i <= n);
        for (; i <= n; i += i & -i)
            bit[i] ^= value;
    }

    ll prefix_xor(int i) const {
        ll answer = 0;
        i = min(i, n);
        for (; i > 0; i -= i & -i)
            answer ^= bit[i];
        return answer;
    }

    ll query(int l, int r) const {
        if (l > r) return 0;
        return prefix_xor(r) ^ prefix_xor(l - 1);
    }
};
```

3.3 Segment Tree iterativo

Cuándo usar: Arreglo dinámico con updates puntuales y consultas de una operación asociativa sobre rangos; también para buscar la primera posición que cumple.

Versión para suma, actualización puntual y consultas semiabiertas $[l, r)$. Cambiar `combine` y `NEUTRAL` para min, max, gcd, etc.

Operaciones comunes

Consulta	oper	NEUT
Minimo	min	LINF
Maximo	max	-LINF
Suma	+	0
Producto	*	1
XOR	^	0
OR bit a bit	|	0
AND bit a bit	&	(~0)
GCD	gcd	0
LCM	lcm	1

Complejidad: build $O(n)$; update/query $O(\log n)$; memoria $O(n)$.

```
struct SegmentTree {
    static constexpr ll NEUTRAL = 0;
    static ll combine(ll a, ll b) { return a + b; }
    int n = 1;
    vector<ll> tree;

    explicit SegmentTree(int size) {
        while (n < size) n <<= 1;
        tree.assign(2 * n, NEUTRAL);
    }

    explicit SegmentTree(const vector<ll> &a) : SegmentTree(a.size()) {
        copy(a.begin(), a.end(), tree.begin() + n);
        for (int p = n - 1; p; --p)
            tree[p] = combine(tree[p << 1], tree[p << 1 | 1]);
    }

    void set_value(int pos, ll value) {
        int p = pos + n;
        tree[p] = value;
        while (p >>= 1)
            tree[p] = combine(tree[p << 1], tree[p << 1 | 1]);
    }

    ll query(int l, int r) const { // [l, r)
        ll left = NEUTRAL, right = NEUTRAL;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) left = combine(left, tree[l++]);
            if (r & 1) right = combine(tree[--r], right);
        }
        return combine(left, right);
    }
};
```

3.4 Lazy Segment Tree

Cuándo usar: Actualizaciones y consultas sobre intervalos, por ejemplo sumar en un rango y consultar suma/mínimo/máximo sin tocar cada elemento.

Suma en rango y suma sobre rango. Los intervalos son semiabiertos $[l, r)$.

Complejidad: build $O(n)$; update/query $O(\log n)$; memoria $O(n)$.

```
struct LazySegmentTree {
    int n;
    vector<ll> tree, lazy;

    explicit LazySegmentTree(const vector<ll> &a)
        : n(a.size()), tree(4 * max(1, n)), lazy(4 * max(1, n)) {
```

```
        if (n) build(1, 0, n, a);
    }

    void build(int p, int l, int r, const vector<ll> &a) {
        if (r - l == 1) { tree[p] = a[l]; return; }
        int m = (l + r) / 2;
        build(p << 1, l, m, a);
        build(p << 1 | 1, m, r, a);
        tree[p] = tree[p << 1] + tree[p << 1 | 1];
    }

    void apply(int p, int l, int r, ll value) {
        tree[p] += value * (r - l);
        lazy[p] += value;
    }

    void push(int p, int l, int r) {
        if (lazy[p] == 0 || r - l == 1) return;
        int m = (l + r) / 2;
        apply(p << 1, l, m, lazy[p]);
        apply(p << 1 | 1, m, r, lazy[p]);
        lazy[p] = 0;
    }

    void add(int ql, int qr, ll value) {
        if (n) add(1, 0, n, ql, qr, value);
    }
    void add(int p, int l, int r, int ql, int qr, ll value) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) { apply(p, l, r, value); return; }
        push(p, l, r);
        int m = (l + r) / 2;
        add(p << 1, l, m, ql, qr, value);
        add(p << 1 | 1, m, r, ql, qr, value);
        tree[p] = tree[p << 1] + tree[p << 1 | 1];
    }

    ll query(int ql, int qr) {
        return n ? query(1, 0, n, ql, qr) : 0;
    }
    ll query(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return tree[p];
        push(p, l, r);
        int m = (l + r) / 2;
        return query(p << 1, l, m, ql, qr)
             + query(p << 1 | 1, m, r, ql, qr);
    }
};
```

Para asignación de rango se necesita un lazy adicional que distinga “sin asignación” de asignar cero.

3.5 Segment Tree persistente

Cuándo usar: Conservar versiones históricas tras updates puntuales, responder consultas por versión o calcular el k-ésimo en subarreglos por diferencia de raíces.

Cada actualización crea una versión nueva copiando solamente los nodos del camino modificado. Esta versión hace asignación puntual y suma en $[l, r)$.

Complejidad: build $O(n)$; update/query $O(\log n)$; cada versión usa $O(\log n)$ nodos nuevos; memoria total $O(n + q \log n)$ para q updates.

```
struct PersistentSegmentTree {
    struct Node { int left = -1, right = -1; ll sum = 0; };
    int n;
    vector<Node> nodes;

    explicit PersistentSegmentTree(const vector<ll> &a) : n(a.size()) {
        assert(n > 0);
        nodes.reserve(2 * n + 20);
    }

    int make_node(Node node) {
        nodes.push_back(node);
        return (int)nodes.size() - 1;
    }

    int build(const vector<ll> &a) { return build(0, n, a); }
    int build(int l, int r, const vector<ll> &a) {
        if (r - l == 1) return make_node({-1, -1, a[l]});
        int m = (l + r) / 2;
        int left = build(l, m, a), right = build(m, r, a);
        return make_node({left, right, nodes[left].sum + nodes[right].sum});
    }

    int set_value(int root, int position, ll value) {
        return set_value(root, 0, n, position, value);
    }

    int set_value(int root, int l, int r, int position, ll value) {
        if (r - l == 1) return make_node({-1, -1, value});
        int m = (l + r) / 2;
        int left = nodes[root].left, right = nodes[root].right;
        if (position < m) left = set_value(left, l, m, position, value);
        else right = set_value(right, m, r, position, value);
        return make_node({left, right, nodes[left].sum + nodes[right].sum});
```

```
    }

    ll query(int root, int ql, int qr) const {
        return query(root, 0, n, ql, qr);
    }

    ll query(int root, int l, int r, int ql, int qr) const {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return nodes[root].sum;
        int m = (l + r) / 2;
        return query(nodes[root].left, l, m, ql, qr)
             + query(nodes[root].right, m, r, ql, qr);
    }
};
```

Guardar cada raíz en un vector: `roots[v]` representa la versión `v`. No modificar nodos existentes.

3.6 Sparse Table

Cuándo usar: Arreglo estático y consultas idempotentes como mínimo, máximo o GCD. Elegir Disjoint Sparse Table si la operación no es idempotente.

Para operaciones idempotentes como min, max o gcd. No requiere inversa y no admite actualizaciones.

Complejidad: build $O(n \log n)$; query $O(1)$; memoria $O(n \log n)$.

```
struct SparseTableMin {
    vector<int> lg;
    vector<vector<ll>> table;

    explicit SparseTableMin(const vector<ll> &a) {
        int n = a.size();
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
        int levels = n ? lg[n] + 1 : 0;
        table.assign(levels, vector<ll>(n));
        if (!n) return;
        table[0] = a;
        for (int k = 1; k < levels; ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                table[k][i] = min(table[k - 1][i],
                                  table[k - 1][i + (1 << (k - 1))]);
    }

    ll query(int l, int r) const { // [l, r], requiere l <= r
        int k = lg[r - l + 1];
        return min(table[k][l], table[k][r - (1 << k) + 1]);
    }
};
```

3.7 Disjoint Sparse Table

Cuándo usar: Arreglo estático, operación asociativa y muchas consultas de rango en $O(1)$, incluso cuando la operación no es idempotente (suma, producto).

Consultas estáticas $O(1)$ para cualquier operación asociativa, incluso no idempotente y no conmutativa. Los rangos son inclusivos.

Complejidad: build $O(n \log n)$; query $O(1)$; memoria $O(n \log n)$.

```
template<class T, class Op = plus<T>>
struct DisjointSparseTable {
    int n = 0, levels = 0;
    Op op;
    vector<T> values;
    vector<vector<T>> table;

    explicit DisjointSparseTable(const vector<T> &a, Op operation = Op())
        : n(a.size()), op(operation), values(a) {
        while ((1 << levels) < max(1, n)) ++levels;
        table.assign(levels, vector<T>(n));
        for (int k = 0; k < levels; ++k) {
            int block = 1 << (k + 1);
            for (int begin = 0; begin < n; begin += block) {
                int mid = min(begin + (block >> 1), n);
                int end = min(begin + block, n);
                if (begin < mid) {
                    table[k][mid - 1] = values[mid - 1];
                    for (int i = mid - 2; i >= begin; --i)
                        table[k][i] = op(values[i], table[k][i + 1]);
                }
                if (mid < end) {
                    table[k][mid] = values[mid];
                    for (int i = mid + 1; i < end; ++i)
                        table[k][i] = op(table[k][i - 1], values[i]);
                }
            }
        }
    }

    T query(int l, int r) const { // [l, r], requiere indices validos
        if (l == r) return values[l];
        if (l > r) swap(l, r);
        int k = 31 - __builtin_clz((unsigned)(l ^ r));
        return op(table[k][l], table[k][r]);
    }
};
```

3.8 Stack y queue monótonas

Cuándo usar: Próximo mayor/menor, spans, histogramas o mínimo/máximo de cada ventana deslizante en tiempo lineal.

```cpp
// Indice del siguiente elemento estrictamente menor; n si no existe.
vector<int> next_smaller(const vector<ll> &a) {
    int n = a.size();
    vector<int> answer(n, n), stack;
    for (int i = 0; i < n; ++i) {
        // Para maximo se cambia por a[i] > a[stack.back()]
        while (!stack.empty() && a[i] < a[stack.back()]) {
            answer[stack.back()] = i;
            stack.pop_back();
        }
        stack.push_back(i);
    }
    return answer;
}

// Minimo de cada ventana de longitud k.
vector<ll> sliding_minimum(const vector<ll> &a, int k) {
    assert(k > 0);
    deque<int> dq;
    vector<ll> answer;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        // Para maximo se cambia por a[dq.back()] <= a[i]
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i + 1 >= k) answer.push_back(a[dq.front()]);
    }
    return answer;
}
```

Complejidad: $O(n)$ tiempo: cada índice entra y sale una sola vez; memoria $O(n)$.

3.9 Ordered Set de PBDS

Cuándo usar: Conjunto ordenado dinámico con consultas por orden: obtener el k-ésimo elemento o contar elementos estrictamente menores que una clave.

Complejidad: insertar, borrar, `find_by_order` y `order_of_key`: $O(\log n)$; memoria $O(n)$.

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<class Key>
using OrderedSet = tree<
    Key,
    null_type,
    less<Key>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;

void ordered_set_example() {
    OrderedSet<int> values;

    values.insert(20);
    values.insert(5);
    values.insert(10);

    cout << *values.find_by_order(0) << '\n'; // 5
    cout << *values.find_by_order(1) << '\n'; // 10
    cout << values.order_of_key(15) << '\n';  // 2
    cout << values.order_of_key(10) << '\n';  // 1

    values.erase(10);
}

// Para repetidos, distinguir cada copia con un ID.
using OrderedMultiset = OrderedSet<pair<int,int>>;
```

- `find_by_order(k)` devuelve un iterador al elemento de índice k, comenzando en cero.
- `order_of_key(x)` cuenta los elementos estrictamente menores que x.
- Si $k \geq size()$, `find_by_order(k)` devuelve `end()`; no debe desreferenciarse.

Cuidado: PBDS no pertenece al estándar de C++; usar solamente si el juez compila con GNU G++ y verificar su disponibilidad durante el contest de práctica.

3.10 Fenwick 2D offline

Cuándo usar: Altas/bajas de puntos y conteos o sumas dentro de rectángulos cuando todas las coordenadas actualizables se conocen antes de procesar.

Cada punto (x, y) tiene un peso: el número que aporta a las sumas. Para contar puntos, agregarlo usa peso $+1$ y eliminarlo peso -1; también puede representar dinero, población u otra cantidad asociada a la coordenada.

Se llama *offline* porque primero se leen todas las operaciones para conocer qué coordenadas podrán actualizarse. Luego se reservan esos puntos, se construye la estructura y se procesan las operaciones en su orden original. No usa archivos. Cada nodo del Fenwick exterior sobre x guarda otro Fenwick con solamente las coordenadas y que puede recibir.

Complejidad: reserva y construcción $O(n \log^2 n)$ en el peor caso; update y consulta $O(\log^2 n)$; memoria $O(n \log n)$.

```cpp
struct OfflineFenwick2D {
    int size_x;
    vector<vector<int>> coordinates_y;
    vector<vector<ll>> bit;

    explicit OfflineFenwick2D(int x_count)
        : size_x(x_count), coordinates_y(x_count + 1), bit(x_count + 1) {}

    void reserve_point(int x, int y) {
        for (int index = x + 1; index <= size_x; index += index & -index)
            coordinates_y[index].push_back(y);
    }

    void build() {
        for (int index = 1; index <= size_x; ++index) {
            auto &values = coordinates_y[index];
            sort(values.begin(), values.end());
            values.erase(unique(values.begin(), values.end()), values.end());
            bit[index].assign(values.size() + 1, 0);
        }
    }

    void add(int x, int y, ll delta) {
        for (int index = x + 1; index <= size_x; index += index & -index) {
            const auto &values = coordinates_y[index];
            auto iterator = lower_bound(values.begin(), values.end(), y);
            assert(iterator != values.end() && *iterator == y);
            int position = iterator - values.begin() + 1;
            for (int inner = position; inner < (int)bit[index].size();
                 inner += inner & -inner)
                bit[index][inner] += delta;
        }
    }

    ll prefix_sum(int x_exclusive, int y_exclusive) const {
        ll answer = 0;
        for (int index = x_exclusive; index > 0; index -= index & -index) {
            int position = lower_bound(coordinates_y[index].begin(),
                                       coordinates_y[index].end(), y_exclusive)
                           - coordinates_y[index].begin();
            for (int inner = position; inner > 0; inner -= inner & -inner)
                answer += bit[index][inner];
        }
        return answer;
    }

    ll rectangle_sum(int x1, int y1, int x2, int y2) const {
        return prefix_sum(x2, y2) - prefix_sum(x1, y2)
               - prefix_sum(x2, y1) + prefix_sum(x1, y1);
    }
};
```

3.10.1 Ejemplo de uso

En un mapa aparecen y desaparecen objetos. Se pide contar cuántos hay dentro de un rectángulo. Todas las coordenadas de futuras altas y bajas aparecen en la entrada, por lo que pueden reservarse antes de procesarla.

```cpp
OfflineFenwick2D points(5); // x validas: [0,5)

// Primera pasada: reservar todo punto que pueda actualizarse.
points.reserve_point(2, 5);
points.reserve_point(4, 3);
points.reserve_point(3, 6);
points.build();

// Segunda pasada: procesar las operaciones en orden.
points.add(2, 5, 1); // aparece un objeto
points.add(4, 3, 1);
points.add(3, 6, 1);

// Rectangulo [2,5) x [3,7): contiene los tres objetos.
cout << points.rectangle_sum(2, 3, 5, 7) << '\n'; // 3

points.add(4, 3, -1); // desaparece un objeto
cout << points.rectangle_sum(2, 3, 5, 7) << '\n'; // 2
```

`prefix_sum(x,y)` suma los pesos de los puntos con coordenadas $X < x$ e $Y < y$. Por inclusión–exclusión, `rectangle_sum(x1,y1,x2,y2)` consulta el rectángulo semiabierto $[x_1, x_2) \times [y_1, y_2)$.

Cuidado: Las coordenadas x deben estar comprimidas en $[0, size_x)$. Llamar `reserve_point` para todo punto actualizable y después `build` exactamente una vez. No intentar actualizar una coordenada que no fue reservada.

3.11 Wavelet Tree

Cuándo usar: Arreglo sin updates y muchas consultas sobre subarreglos: k-ésimo menor, frecuencia de un valor o cantidad de elementos $\leq x$.

Cada nodo divide el rango de valores por la mitad. `go_left[i]` guarda cuántos de los primeros i elementos fueron al hijo izquierdo; así un subarreglo $[l, r)$ se traduce a intervalos de los hijos sin perder su orden. Es una alternativa al Segment Tree persistente cuando no se necesitan versiones ni actualizaciones.

Complejidad: build $O(n \log \sigma)$; consulta $O(\log \sigma)$; memoria $O(n \log \sigma)$, donde $\sigma = max - min + 1$.

```cpp
struct WaveletTree {
    int low, high;
    vector<int> go_left;
    unique_ptr<WaveletTree> left_child, right_child;

    template<class Iterator>
    WaveletTree(Iterator from, Iterator to,
                int minimum_value, int maximum_value)
        : low(minimum_value), high(maximum_value) {
        if (from == to || low == high) return;
        int middle = low + (int)(((ll)high - low) / 2);
        auto goes_left = [middle](int value) { return value <= middle; };
        go_left.reserve(to - from + 1);
        go_left.push_back(0);
        for (Iterator iterator = from; iterator != to; ++iterator)
            go_left.push_back(go_left.back() + goes_left(*iterator));
        Iterator pivot = stable_partition(from, to, goes_left);
        left_child = make_unique<WaveletTree>(from, pivot, low, middle);
        right_child = make_unique<WaveletTree>(pivot, to, middle + 1, high);
    }

    // k empieza en 1; intervalo [left,right).
    int kth(int left, int right, int k) const {
        assert(1 <= k && k <= right - left);
        if (low == high) return low;
        int before_left = go_left[left];
        int inside_left = go_left[right] - before_left;
        if (k <= inside_left)
            return left_child->kth(before_left, go_left[right], k);
        return right_child->kth(left - before_left,
                                right - go_left[right],
                                k - inside_left);
    }

    int count_leq(int left, int right, int value) const {
        if (left >= right || value < low) return 0;
        if (high <= value) return right - left;
        int before_left = go_left[left];
        int before_right = go_left[right];
        return left_child->count_leq(before_left, before_right, value)
             + right_child->count_leq(left - before_left,
                                      right - before_right, value);
    }
};
```

3.11.1 Ejemplo de uso

Para $a = [7, 1, 4, 2, 6, 4]$, el subarreglo $[1, 6)$ es $[1, 4, 2, 6, 4]$.

```cpp
void wavelet_example() {
    vector<int> data = {7, 1, 4, 2, 6, 4};
    int minimum = *min_element(data.begin(), data.end());
    int maximum = *max_element(data.begin(), data.end());
    WaveletTree wavelet(data.begin(), data.end(), minimum, maximum);

    cout << wavelet.kth(1, 6, 3) << '\n';       // 4 el k-esimo menor de [left, right). k comienza en 1.
    cout << wavelet.count_leq(1, 6, 3) << '\n'; // 2: {1,2} elementos de [left, right) son menores o iguales a value

    int frequency_4 = wavelet.count_leq(1, 6, 4)
                    - wavelet.count_leq(1, 6, 3);
    cout << frequency_4 << '\n';                // 2
}
```

Cuidado: El constructor reordena el vector recibido; pasar una copia si hace falta conservar el orden original. Si el rango de valores es enorme, comprimir coordenadas reduce σ.

3.12 Treap implícito

Cuándo usar: Secuencia cuyo largo u orden cambia: insertar/borrar en el medio, invertir, cortar o consultar segmentos. Para un arreglo de largo fijo suele ser más simple un Segment Tree.

El recorrido inorder del árbol es la secuencia. No guarda claves: el tamaño de cada subárbol determina la posición. `split(t,k)` separa los primeros k elementos del resto y `merge(a,b)` concatena dos secuencias. Para operar sobre $[l, r)$ se lo aísla con dos splits y luego se vuelven a unir las tres partes.

Complejidad: split, merge, inserción, borrado, inversión y consulta $O(\log n)$ esperado; memoria $O(n)$.

```cpp
struct ImplicitTreap {
    struct Node {
        ll value, sum;
        unsigned priority;
        int size = 1;
        bool reversed = false;
        Node *left_child = nullptr, *right_child = nullptr;
        Node(ll initial_value, unsigned initial_priority)
            : value(initial_value), sum(initial_value),
              priority(initial_priority) {}
    };

    Node *root = nullptr;

    static unsigned next_priority() {
```

```
        static mt19937 generator(chrono::steady_clock::now()
                                 .time_since_epoch().count());
        return generator();
    }

    static int node_size(Node *node) { return node ? node->size : 0; }
    static ll node_sum(Node *node) { return node ? node->sum : 0; }
    static void push(Node *node) {
        if (!node || !node->reversed) return;
        swap(node->left_child, node->right_child);
        if (node->left_child) node->left_child->reversed ^= true;
        if (node->right_child) node->right_child->reversed ^= true;
        node->reversed = false;
    }
    static void pull(Node *node) {
        node->size = 1 + node_size(node->left_child)
                       + node_size(node->right_child);
        node->sum = node->value + node_sum(node->left_child)
                                + node_sum(node->right_child);
    }

    static void split(Node *node, int left_size,
                      Node *&left_tree, Node *&right_tree) {
        if (!node) { left_tree = right_tree = nullptr; return; }
        push(node);
        if (node_size(node->left_child) >= left_size) {
            split(node->left_child, left_size,
                  left_tree, node->left_child);
            right_tree = node; pull(right_tree);
        } else {
            int remaining = left_size - node_size(node->left_child) - 1;
            split(node->right_child, remaining,
                  node->right_child, right_tree);
            left_tree = node; pull(left_tree);
        }
    }

    static Node *merge(Node *left_tree, Node *right_tree) {
        if (!left_tree || !right_tree)
            return left_tree ? left_tree : right_tree;
        push(left_tree); push(right_tree);
        if (left_tree->priority > right_tree->priority) {
            left_tree->right_child = merge(left_tree->right_child, right_tree);
            pull(left_tree); return left_tree;
        }
        right_tree->left_child = merge(left_tree, right_tree->left_child);
        pull(right_tree); return right_tree;
    }

    void insert(int position, ll value) {
        assert(0 <= position && position <= node_size(root));
        Node *left, *right;
        split(root, position, left, right);
        Node *node = new Node(value, next_priority());
        root = merge(merge(left, node), right);
    }

    void erase(int position) {
        assert(0 <= position && position < node_size(root));
        Node *left, *middle, *right;
        split(root, position, left, middle);
        split(middle, 1, middle, right);
        delete middle;
        root = merge(left, right);
    }

    void reverse_range(int left, int right) { // [left,right)
        assert(0 <= left && left <= right && right <= node_size(root));
        Node *prefix, *middle, *suffix;
        split(root, right, middle, suffix);
        split(middle, left, prefix, middle);
        if (middle) middle->reversed ^= true;
        root = merge(merge(prefix, middle), suffix);
    }

    ll range_sum(int left, int right) { // [left,right)
        assert(0 <= left && left <= right && right <= node_size(root));
        Node *prefix, *middle, *suffix;
        split(root, right, middle, suffix);
        split(middle, left, prefix, middle);
        ll answer = node_sum(middle);
        root = merge(merge(prefix, middle), suffix);
        return answer;
    }
};
```

3.12.1 Ejemplo de uso

```
void implicit_treap_example() {
    ImplicitTreap sequence;
    sequence.insert(0, 10); // [10]
    sequence.insert(1, 20); // [10,20]
    sequence.insert(2, 30); // [10,20,30]
    sequence.insert(1, 5);  // [10,5,20,30]

    cout << sequence.range_sum(1, 4) << '\n'; // 55
```

```
    sequence.reverse_range(0, 3);              // [20,5,10,30]
    cout << sequence.range_sum(0, 2) << '\n'; // 25
    sequence.erase(1);                         // [20,10,30]
}
```

La prioridad aleatoria mantiene el árbol balanceado en esperanza. La bandera `reversed` es lazy: invertir un segmento sólo marca su raíz y el cambio se propaga cuando se baja por ese subárbol.

Cuidado: Esta versión reserva nodos con `new`. Si se crean y destruyen muchas secuencias, agregar un destructor o usar un pool de memoria. No copiar la estructura directamente: dos copias compartirían los mismos punteros.

3.13 Convex Hull Trick y Li Chao Tree

Cuándo usar: Una DP o consulta debe calcular repetidamente $\min_j(m_j x + b_j)$. Usar CHT con deque si pendientes y consultas son monótonas; Li Chao si llegan en cualquier orden.

Cada estado anterior j aporta una recta $y = m_j x + b_j$ y cada nuevo estado consulta el menor valor en x_i. Esto reemplaza probar todos los j, que suele ser $O(n^2)$. La plantilla incluida es Li Chao para mínimos: acepta rectas y consultas desordenadas sobre un dominio entero conocido $[L, R]$.

Complejidad: CHT monótono: inserción amortizada $O(1)$ y consulta $O(1)$; Li Chao: $O(\log X)$ por operación y $O(q \log X)$ memoria en versión dinámica.

```
struct LiChaoTree {
    struct Line {
        ll slope = 0, intercept = (1LL << 62);
        ll evaluate(ll x) const { return slope * x + intercept; }
    };
    struct Node { Line line; int left_child = -1, right_child = -1; };
    ll domain_left, domain_right;
    vector<Node> nodes = vector<Node>(1);

    LiChaoTree(ll left_bound, ll right_bound)
        : domain_left(left_bound), domain_right(right_bound) {
        assert(domain_left <= domain_right);
    }

    int make_node() {
        nodes.push_back(Node{});
        return (int)nodes.size() - 1;
    }
    void add_line(Line added) {
        add_line(0, domain_left, domain_right, added);
    }
    void add_line(int node, ll left, ll right, Line added) {
        ll middle = left + (ll)(((i128)right - left) / 2);
        bool better_left =
            added.evaluate(left) < nodes[node].line.evaluate(left);
        bool better_middle =
            added.evaluate(middle) < nodes[node].line.evaluate(middle);
        if (better_middle) swap(added, nodes[node].line);
        if (left == right) return;
        if (better_left != better_middle) {
            if (nodes[node].left_child == -1)
                nodes[node].left_child = make_node();
            add_line(nodes[node].left_child, left, middle, added);
        } else {
            if (nodes[node].right_child == -1)
                nodes[node].right_child = make_node();
            add_line(nodes[node].right_child, middle + 1, right, added);
        }
    }
    ll query(ll x) const {
        assert(domain_left <= x && x <= domain_right);
        return query(0, domain_left, domain_right, x);
    }
    ll query(int node, ll left, ll right, ll x) const {
        ll answer = nodes[node].line.evaluate(x);
        if (left == right) return answer;
        ll middle = left + (ll)(((i128)right - left) / 2);
        int next = x <= middle ? nodes[node].left_child
                               : nodes[node].right_child;
        if (next == -1) return answer;
        if (x <= middle) return min(answer, query(next, left, middle, x));
        return min(answer, query(next, middle + 1, right, x));
    }
};
```

3.13.1 Ejemplo de uso

Se agregan $y = 2x + 3$, $y = -x + 10$ e $y = 5$ y se consulta el mínimo.

```
void li_chao_example() {
    LiChaoTree lines(0, 10);
    lines.add_line({2, 3});   // 2x+3
    lines.add_line({-1, 10}); // -x+10
    lines.add_line({0, 5});   // 5

    cout << lines.query(2) << '\n'; // min(7,8,5)  = 5
    cout << lines.query(5) << '\n'; // min(13,5,5) = 5
    cout << lines.query(8) << '\n'; // min(19,2,5) = 2
}
```

Cuidado: Puede desbordar $mx + b$; usar `__int128` si los límites lo requieren y elegir un infinito mayor que toda respuesta válida. Para máximos, negar rectas/respuestas o

invertir las comparaciones.

3.14 Intervalos disjuntos (ODT)

Cuándo usar: El arreglo está formado por pocos intervalos constantes y predominan asignaciones de rango; apropiado con fragmentación controlada o datos aleatorios.

Usar para mantener una partición ordenada en intervalos constantes y aplicar asignaciones sobre rangos. Funciona muy bien con datos aleatorios o cuando cada operación crea pocos cortes; no garantiza buen peor caso.

Complejidad: `split` $O(\log I)$; una operación cuesta $O((I_{borrados}+1)\log I)$; memoria $O(I)$. Peor caso total cuadrático.

```
struct DisjointIntervals {
    struct Interval { int right; mutable ll value; };
    map<int, Interval> intervals; // left -> {right,value}, rangos [left,right)

    auto split(int position) {
        auto iterator = prev(intervals.upper_bound(position));
        if (iterator->first == position) return iterator;
        int right = iterator->second.right;
        ll value = iterator->second.value;
        if (position >= right) return intervals.end();
        iterator->second.right = position;
        return intervals.emplace(position, Interval{right, value}).first;
    }

    void assign(int left, int right, ll value) {
        if (left >= right) return; // Rango vacio: no modificar la particion.
        auto after = split(right);
        auto first = split(left);
        intervals.erase(first, after);
        intervals[left] = {right, value};
    }
};
odt.assign(100, 500, 1);
odt.assign(300, 800, 2);
odt.assign(700, 900, 3);
```

Cuidado: Inicializar la partición cubriendo todo el dominio y operar dentro de sus límites. Asignar un rango vacío no hace nada. No usar ODT si un adversario puede fragmentar los intervalos en cada operación.

3.15 KACTL source conventions

The following additions use the original KACTL interfaces and comments. The common prelude is shown below; copy dependencies from `kactl/content/` when including the headers. Some originals are reference fragments; their notes identify the required customization.

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
```

3.16 DSU rollback

```
/**
 * Author: Lukas Polacek, Simon Lindholm
 * Date: 2019-12-26
 * License: CC0
 * Source: folklore
 * Description: Disjoint-set data structure with undo.
 * If undo is not needed, skip st, time() and rollback().
 * Usage: int t = uf.time(); ...; uf.rollback(t);
 * Time: $O(\log(N))$
 * Status: tested as part of DirectedMST.h
 */
#pragma once

struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

3.17 Range assignment, addition and maximum

```
/**
 * Author: Simon Lindholm
 * Date: 2016-10-08
 * License: CC0
 * Source: me
 * Description: Segment tree with ability to add or set values of large
    intervals, and compute max of intervals.
 * Can be changed to other things.
 * Use with a bump allocator for better performance, and SmallPtr or implicit
    indices to save memory.
 * Time: O(\log N).
 * Usage: Node* tr = new Node(v, 0, sz(v));
 * Status: stress-tested a bit
 */
#pragma once

#include "../various/BumpAllocator.h"

const int inf = 1e9;
struct Node {
  Node *l = 0, *r = 0;
  int lo, hi, mset = inf, madd = 0, val = -inf;
  Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
  Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
    if (lo + 1 < hi) {
      int mid = lo + (hi - lo)/2;
      l = new Node(v, lo, mid); r = new Node(v, mid, hi);
      val = max(l->val, r->val);
    }
    else val = v[lo];
  }
  int query(int L, int R) {
    if (R <= lo || hi <= L) return -inf;
    if (L <= lo && hi <= R) return val;
    push();
    return max(l->query(L, R), r->query(L, R));
  }
  void set(int L, int R, int x) {
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) mset = val = x, madd = 0;
    else {
      push(), l->set(L, R, x), r->set(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void add(int L, int R, int x) {
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) {
      if (mset != inf) mset += x;
      else madd += x;
      val += x;
    }
    else {
      push(), l->add(L, R, x), r->add(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void push() {
    if (!l) {
      int mid = lo + (hi - lo)/2;
      l = new Node(lo, mid); r = new Node(mid, hi);
    }
    if (mset != inf)
      l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
    else if (madd)
      l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
  }
};
```

3.18 2D prefix sums

```
/**
 * Author: Johan Sannemo
 * Date: 2014-11-28
 * License: CC0
 * Source: Folklore
 * Description: Calculate submatrix sums quickly, given upper-left and lower-
    right corners (half-open).
 * Usage:
 * SubMatrix<int> m(matrix);
 * m.sum(0, 0, 2, 2); // top left 4 elements
 * Time: O(N^2 + Q)
 * Status: Tested on Kattis
 */
#pragma once

template<class T>
struct SubMatrix {
  vector<vector<T>> p;
  SubMatrix(vector<vector<T>>& v) {
    int R = sz(v), C = sz(v[0]);
```

```
    p.assign(R+1, vector<T>(C+1));
    rep(r,0,R) rep(c,0,C)
      p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
  }
  T sum(int u, int l, int d, int r) {
    return p[d][r] - p[d][l] - p[u][r] + p[u][l];
  }
};
```

3.19 Mo queries on tree paths

Only the tree-path functionality is new here; interval Mo is already covered. The add/del/calc callbacks are problem-specific hooks in the original source.

```
vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}
```

4. Grafos

4.1 BFS, DFS y bipartición

Cuándo usar: Explorar componentes, construir padres/profundidades, detectar ciclos o colorear un grafo; BFS da distancias mínimas cuando todas las aristas pesan uno.

```cpp
vector<int> bfs_distances(const vector<vector<int>> &graph, int source) {
    int n = graph.size();
    vector<int> distance(n, -1);
    queue<int> pending;
    distance[source] = 0;
    pending.push(source);
    while (!pending.empty()) {
        int v = pending.front(); pending.pop();
        for (int to : graph[v]) if (distance[to] == -1) {
            distance[to] = distance[v] + 1;
            pending.push(to);
        }
    }
    return distance;
}

vector<int> undirected_components(const vector<vector<int>> &graph) {
    int n = graph.size(), component_count = 0;
    vector<int> component(n, -1), stack;
    for (int start = 0; start < n; ++start) if (component[start] == -1) {
        component[start] = component_count++;
        stack.push_back(start);
        while (!stack.empty()) {
            int v = stack.back(); stack.pop_back();
            for (int to : graph[v]) if (component[to] == -1) {
                component[to] = component[v];
                stack.push_back(to);
            }
        }
    }
    return component;
}

bool is_bipartite(const vector<vector<int>> &graph) {
    int n = graph.size();
    vector<int> color(n, -1);
    for (int start = 0; start < n; ++start) if (color[start] == -1) {
        queue<int> q;
        color[start] = 0; q.push(start);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int to : graph[v]) {
                if (color[to] == -1) color[to] = color[v] ^ 1, q.push(to);
                else if (color[to] == color[v]) return false;
            }
        }
    }
    return true;
}
```

Complejidad: $O(V + E)$ tiempo y $O(V)$ memoria.

4.2 BFS multiorigen

Cuándo usar: Distancia al origen más cercano o expansión simultánea desde muchas fuentes en un grafo sin pesos.

Usar cuando se pide la distancia al origen más cercano, expansión simultánea, Voronoi sobre un grafo no pesado o el instante en que llega primero alguna fuente. Es una BFS normal inicializando la cola con todas las fuentes a distancia cero.

Complejidad: tiempo $O(n + m)$; memoria $O(n)$.

```cpp
vector<int> multi_source_bfs(const vector<vector<int>> &graph,
                             const vector<int> &sources) {
    int node_count = (int)graph.size();
    vector<int> distance(node_count, -1);
    queue<int> pending;
    for (int source : sources) if (distance[source] == -1) {
        distance[source] = 0;
        pending.push(source);
    }
    while (!pending.empty()) {
        int vertex = pending.front(); pending.pop();
        for (int to : graph[vertex]) if (distance[to] == -1) {
            distance[to] = distance[vertex] + 1;
            pending.push(to);
        }
    }
    return distance;
}
```

Para recuperar qué fuente ganó, propagar además un arreglo `owner`.

4.3 Orden topológico

Cuándo usar: Procesar dependencias de un DAG, hacer DP en grafo dirigido o detectar ciclos comprobando si se ordenaron todos los vértices.

Kahn devuelve un orden solamente si el grafo dirigido es un DAG.

```cpp
vector<int> topological_sort(const vector<vector<int>> &graph) {
    int n = graph.size();
    vector<int> indegree(n, 0), order;
    for (int v = 0; v < n; ++v)
        for (int to : graph[v]) ++indegree[to];

    queue<int> q;
    for (int v = 0; v < n; ++v) if (indegree[v] == 0) q.push(v);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        order.push_back(v);
        for (int to : graph[v]) if (--indegree[to] == 0) q.push(to);
    }
    if ((int)order.size() != n) return {}; // existe un ciclo
    return order;
}
```

Complejidad: $O(V + E)$ tiempo y memoria.

4.4 Componentes fuertemente conexas (Tarjan)

Cuándo usar: Contraer ciclos de un grafo dirigido, obtener su DAG de componentes o resolver problemas de alcanzabilidad y 2-SAT.

Tarjan encuentra las componentes en una sola DFS, sin construir el grafo transpuesto. Esta adaptación de KACTL (Lukas Polacek) conserva el estado dentro de la función y devuelve `component[v]`, sin globales ni callback obligatorio.

`discovery[v]` guarda inicialmente el orden de visita; al regresar del DFS se reutiliza para el mínimo alcanzable. La pila contiene los vértices todavía sin componente. El DFS devuelve el menor orden alcanzable dentro de esa zona activa; si coincide con el de `v`, se desapila una componente completa. Las aristas hacia componentes ya cerradas se ignoran.

Tarjan las cierra en orden topológico inverso. Al terminar se invierten los IDs para conservar el contrato del handbook: si existe una arista $u \to v$ entre componentes distintas, `component[u] < component[v]`. Los IDs son consecutivos desde cero; el grafo vacío devuelve un vector vacío. Esto mantiene compatible la comparación usada por 2-SAT.

```cpp
vector<int> strongly_connected_components(
        const vector<vector<int>> &graph) {
    int n = graph.size();
    vector<int> discovery(n),stack;
    vector<int> component(n, -1);
    int timer=0,count=0;
    auto dfs = [&](auto &&self,int v) -> int {
        int low=discovery[v]=++timer;
        stack.push_back(v);
        for (int to : graph[v]) if (component[to]==-1)
            low=min(low,discovery[to]?discovery[to]:self(self,to));
        if (low==discovery[v]) {
            int u;
            do {
                u=stack.back(); stack.pop_back();
                component[u]=count;
            } while (u!=v);
            ++count;
        }
        return discovery[v]=low;
    };
    for (int v=0;v<n;++v) if (component[v]==-1) dfs(dfs,v);
    for (int &id : component) id=count-1-id; // Orden topológico del handbook.
    return component;
}
```

Complejidad: $O(V + E)$ tiempo y $O(V)$ memoria adicional, incluida la pila recursiva; la lista de adyacencia de entrada ocupa $O(V + E)$.

Cuidado: La DFS recursiva puede agotar la pila en grafos muy profundos, como una cadena larga; en ese caso usar una implementación iterativa.

4.5 2-SAT

Cuándo usar: Elegir verdadero/falso para cada variable bajo restricciones de dos literales: "al menos uno", incompatibilidades e implicaciones.

Cada variable tiene dos nodos: falso $2v$ y verdadero $2v + 1$. La negación se obtiene con XOR 1. Una cláusula $(a \vee b)$ equivale a las implicaciones $(\neg a \Rightarrow b)$ y $(\neg b \Rightarrow a)$.

Complejidad: $O(V + E)$ tiempo y memoria usando SCC sobre $2V$ literales.

```cpp
struct TwoSAT {
    int variables;
    vector<vector<int>> implication;

    explicit TwoSAT(int variable_count)
        : variables(variable_count),
          implication(2 * variable_count) {}

    // Nodo que representa (variable == value).
    int literal(int variable, bool value) const {
        return 2 * variable + (value ? 1 : 0);
    }

    void add_implication(int from, int to) {
        implication[from].push_back(to);
    }
```

```

    // (x == value_x) OR (y == value_y).
    void add_or(int x, bool value_x, int y, bool value_y) {
        int a = literal(x, value_x);
        int b = literal(y, value_y);
        add_implication(a ^ 1, b);
        add_implication(b ^ 1, a);
    }

    // Obliga a que variable == value.
    void force(int variable, bool value) {
        add_or(variable, value, variable, value);
    }

    // (x == value_x) implica (y == value_y).
    void imply(int x, bool value_x, int y, bool value_y) {
        add_or(x, !value_x, y, value_y);
    }

    // Exactamente una de las dos condiciones debe cumplirse.
    void add_xor(int x, bool value_x, int y, bool value_y) {
        add_or(x, value_x, y, value_y);
        add_or(x, !value_x, y, !value_y);
    }

    // x e y deben tener el mismo valor.
    void add_equal(int x, int y) {
        imply(x, true, y, true);
        imply(y, true, x, true);
    }

    // x e y deben tener valores diferentes.
    void add_different(int x, int y) {
        add_xor(x, true, y, true);
    }

    optional<vector<bool>> solve() const {
        vector<int> component =
            strongly_connected_components(implication);

        vector<bool> assignment(variables);
        for (int variable = 0; variable < variables; ++variable) {
            int is_false = literal(variable, false);
            int is_true = literal(variable, true);

            if (component[is_false] == component[is_true])
                return nullopt;

            // SCC del handbook: IDs en orden topologico.
            assignment[variable] =
                component[is_true] > component[is_false];
        }
        return assignment;
    }
};
```

Operaciones frecuentes:

```
TwoSAT sat(3);
int A = 0, B = 1, C = 2;
sat.add_or(A, true, B, true);  // A o B
sat.force(C, true);            // C debe ser verdadero
sat.imply(A, true, C, true);   // A => C
sat.add_xor(A, true, B, true); // Exactamente uno: A o B
sat.add_equal(A, C);           // A == C
sat.add_different(B, C);       // B != C
auto answer = sat.solve();
if (!answer) { cout << "Imposible\n";
} else {
    for (bool value : *answer)
        cout << value << ' ';
    cout << '\n';
}
```

Cuidado: La SCC del handbook invierte los IDs internos de Tarjan para devolverlos en orden topológico; por eso `solve` conserva la comparación `>`. Si se copia otra SCC que devuelve orden topológico inverso, usar `<`. También se requiere `#include <optional>`.

4.6 Dijkstra

Cuándo usar: Caminos mínimos desde una fuente con pesos no negativos. Para pesos 0/1 usar 0–1 BFS; con negativos, Bellman–Ford.

Distancias desde una fuente con pesos no negativos. Para reconstruir caminos, descomentar las líneas marcadas con `Reconstruccion` y el ejemplo opcional. Guardar padres no cambia el orden del tiempo de ejecución y agrega $O(V)$ de memoria; reconstruir un camino cuesta $O(V)$ tiempo y memoria en el peor caso.

```
vector<ll> dijkstra(const vector<vector<pair<int,ll>>> &graph, int source
    // , vector<int>& parent // Reconstruccion: parametro de salida.
) {
    int n = graph.size();
    vector<ll> distance(n, LINF);
    // parent.assign(n, -1); // Reconstruccion: sin padre inicialmente.
    priority_queue<pair<ll,int>, vector<pair<ll,int>>,
                   greater<pair<ll,int>>> pending;
```

```
    distance[source] = 0;
    pending.push({0, source});

    while (!pending.empty()) {
        auto [dist_v, v] = pending.top(); pending.pop();
        if (dist_v != distance[v]) continue;
        for (auto [to, weight] : graph[v]) {
            if (weight < 0) throw invalid_argument("negative edge");
            if (distance[to] > dist_v + weight) {
                distance[to] = dist_v + weight;
                // parent[to] = v; // Reconstruccion: guardar predecesor.
                pending.push({distance[to], to});
            }
        }
    }
    return distance;
}
// Ejemplo de uso: grafo dirigido con pesos no negativos.
vector<vector<pair<int,ll>>> graph(4);
graph[0] = {{1, 4}, {2, 1}};
graph[2] = {{1, 2}};
graph[1] = {{3, 1}};
auto distance = dijkstra(graph, 0);
cout << distance[3] << '\n'; // 4

// Ejemplo opcional de reconstruccion (reemplaza la llamada anterior):
// vector<int> parent;
// auto distance = dijkstra(graph, 0, parent);
// int source = 0, target = 3;
// vector<int> path;
// if (distance[target] != LINF) { // Si es inalcanzable, queda vacio.
//     for (int v = target; v != -1; v = parent[v])
//         path.push_back(v);
//     reverse(path.begin(), path.end());
// }
// for (int v : path) cout << v << ' '; // 0 2 1 3
// Si target == source, path contiene solo source.
```

Complejidad: $O((V+E)\log V)$; memoria $O(V+E)$.

4.7 0–1 BFS

Cuándo usar: Caminos mínimos cuando cada arista pesa exactamente 0 o 1; evita el factor logarítmico de Dijkstra usando una deque.

Para pesos exclusivamente cero o uno.

```
vector<int> zero_one_bfs(const vector<vector<pair<int,int>>> &graph,
                         int source) {
    int n = graph.size();
    vector<int> distance(n, INF);
    deque<int> q;
    distance[source] = 0; q.push_front(source);
    while (!q.empty()) {
        int v = q.front(); q.pop_front();
        for (auto [to, weight] : graph[v]) {
            assert(weight == 0 || weight == 1);
            if (distance[to] > distance[v] + weight) {
                distance[to] = distance[v] + weight;
                if (weight == 0) q.push_front(to);
                else q.push_back(to);
            }
        }
    }
    return distance;
}
vector<vector<pair<int,int>>> graph(4);
graph[0].push_back({1, 0}); // 0 -> 1, costo 0
graph[0].push_back({2, 1}); // 0 -> 2, costo 1
graph[1].push_back({2, 0}); // 1 -> 2, costo 0
graph[2].push_back({3, 1}); // 2 -> 3, costo 1

vector<int> distance = zero_one_bfs(graph, 0);
cout << distance[3] << '\n'; // 1
```

Complejidad: $O(V+E)$ tiempo y memoria.

4.8 Bellman–Ford

Cuándo usar: Caminos mínimos con pesos negativos y detección de ciclos negativos alcanzables; sólo para grafos moderados por su costo cuadrático.

Admite pesos negativos. Tras $V-1$ rondas, una relajación adicional indica un ciclo negativo alcanzable desde la fuente. Usa `i128` internamente, sin recortar distancias durante la detección. Devuelve `{distancias,false}` o `{{},true}` si hay ciclo negativo alcanzable. Los inalcanzables valen `LINF`; lanza `overflow_error` si una distancia finita no entra en `long long` o coincide con esa marca. Para reconstruir, descomentar las líneas marcadas `Reconstruccion` y el ejemplo opcional: agrega $O(V)$ memoria; recuperar un camino cuesta $O(V)$.

```
struct WeightedEdge { int from, to; ll weight; };

pair<vector<ll>, bool> bellman_ford(
        int n, const vector<WeightedEdge> &edges, int source
        // , vector<int>& parent // Reconstruccion: parametro de salida.
) {
    assert(n > 0 && 0 <= source && source < n);
    // parent.assign(n, -1); // Reconstruccion: sin padre inicialmente.
```

```
    const i128 INF128 = (i128)1 << 126;
    vector<i128> distance(n, INF128);
    distance[source] = 0;
    for (int iteration = 0; iteration < n - 1; ++iteration) {
        bool changed = false;
        for (auto [from, to, weight] : edges) {
            if (distance[from] == INF128) continue;
            i128 candidate = distance[from] + weight;
            if (candidate < distance[to]) {
                distance[to] = candidate;
                // parent[to] = from; // Reconstruccion: guardar predecesor.
                changed = true;
            }
        }
        if (!changed) break;
    }
    for (auto [from, to, weight] : edges)
        if (distance[from] != INF128 && distance[from] + weight < distance[to])
            return {{}, true};
    vector<ll> result(n, LINF);
    for (int v = 0; v < n; ++v) if (distance[v] != INF128) {
        if (distance[v] < numeric_limits<ll>::min() ||
            distance[v] > numeric_limits<ll>::max() || distance[v] == LINF)
            throw overflow_error("distance outside output range");
        result[v] = (ll)distance[v];
    }
    return {std::move(result), false};
}

// Ejemplo opcional: activar antes las tres lineas de Reconstruccion.
// vector<WeightedEdge> edges = {{0,1,4}, {0,2,5}, {1,2,-2}, {2,3,3}};
// vector<int> parent;
// int source = 0, target = 3;
// auto [distance, negative_cycle] = bellman_ford(4, edges, source, parent);
// vector<int> path;
// if (!negative_cycle && distance[target] != LINF) {
//     for (int v = target; v != -1; v = parent[v]) path.push_back(v);
//     reverse(path.begin(), path.end());
// }
// for (int v : path) cout << v << ' '; // 0 1 2 3, costo 5
// Si no hay ciclo y target == source, path contiene solo source.
```

Complejidad: $O(VE)$ tiempo y $O(V)$ memoria.

Cuidado: Requiere vértices 0-indexados y pesos `long long`. No usar las distancias si el indicador de ciclo es verdadero.

Ejemplo de manejo de salida: cada arista entra en `long long`, pero la suma de dos costos máximos no. El `catch` imprime `overflow`.

```
void bellman_ford_output_example() {
    ll weight = numeric_limits<ll>::max();
    vector<WeightedEdge> edges = {{0,1,weight}, {1,2,weight}};
    try {
        auto [distance, negative_cycle] = bellman_ford(3, edges, 0);
        if (negative_cycle) cout << "Ciclo negativo\n";
        else if (distance[2] == LINF) cout << "Inalcanzable\n";
        else cout << distance[2] << '\n';
    } catch (const overflow_error&) {
        cout << "overflow\n";
    }
}
```

4.9 Floyd–Warshall

Cuándo usar: Distancias entre todos los pares cuando V es pequeño; también para clausura transitiva o detectar ciclos negativos mediante la diagonal.

Todos contra todos. Inicializar diagonal en cero, aristas en su peso mínimo y el resto en `LINF`. Usar pesos y distancias finitas estrictamente entre $-$`LINF` y `LINF`; alcanza con $V \max|w| <$ `LINF`. Al detectar una diagonal negativa se activa la saturación por debajo en $-$`LINF`, antes de procesar ese vértice como intermediario. Evita que los ciclos negativos hagan decrecer los valores hasta desbordar y conserva el bucle sin saturación mientras no se detecta un ciclo negativo. Un par afectado por un ciclo negativo no tiene distancia mínima finita: consultar `negative_cycle_between`, no interpretar su valor numérico.

```
void floyd_warshall(vector<vector<ll>> &distance) {
    int n = distance.size();
    bool negative_cycle = false;
    for (int k = 0; k < n; ++k) {
        negative_cycle = negative_cycle || distance[k][k] < 0;
        auto relax = [&](bool saturate) {
            for (int i = 0; i < n; ++i) if (distance[i][k] != LINF)
                for (int j = 0; j < n; ++j) if (distance[k][j] != LINF) {
                    ll candidate = distance[i][k] + distance[k][j];
                    if (saturate) candidate = max(-LINF, candidate);
                    distance[i][j] = min(distance[i][j], candidate);
                }
        };
        if (negative_cycle) relax(true);
        else relax(false);
    }
}

bool negative_cycle_between(const vector<vector<ll>> &d, int a, int b) {
    for (int k = 0; k < (int)d.size(); ++k)
        if (d[a][k] != LINF && d[k][k] < 0 && d[k][b] != LINF)
```

```
            return true;
    return false;
}
```

Complejidad: $O(V^3)$ tiempo y $O(V^2)$ memoria.

4.10 Árbol de expansión mínima (Kruskal)

Cuándo usar: Conectar todos los vértices con costo total mínimo. Kruskal es cómodo con lista de aristas; Prim con adyacencias y grafos densos/implícitos.

Kruskal acepta pesos negativos. Si usa menos de $n-1$ aristas, el grafo es disconexo y el resultado es un bosque de expansión mínima.

```
struct MstEdge { int u, v; ll weight; };

pair<ll, vector<MstEdge>> kruskal(int n, vector<MstEdge> edges) {
    sort(edges.begin(), edges.end(), [](const MstEdge &a, const MstEdge &b) {
        return a.weight < b.weight;
    });
    DSU dsu(n);
    ll cost = 0;
    vector<MstEdge> chosen;
    for (const auto &edge : edges) if (dsu.unite(edge.u, edge.v)) {
        cost += edge.weight;
        chosen.push_back(edge);
    }
    return {cost, chosen};
}
```

Complejidad: Kruskal $O(E \log E)$ tiempo y $O(V+E)$ memoria.

Prim es conveniente cuando el grafo ya está en listas de adyacencia.

```
optional<ll> prim(const vector<vector<pair<int,ll>>> &graph) {
    int n = graph.size();
    if (n == 0) return 0LL;
    vector<char> used(n, false);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>,
                   greater<pair<ll,int>>> q;
    q.push({0, 0});
    ll cost = 0;
    int visited = 0;
    while (!q.empty()) {
        auto [weight, v] = q.top(); q.pop();
        if (used[v]) continue;
        used[v] = true; ++visited; cost += weight;
        for (auto [to, edge_weight] : graph[v])
            if (!used[to]) q.push({edge_weight, to});
    }
    if (visited != n) return nullopt;
    return cost;
}
```

Complejidad: Prim $O(E \log V)$ tiempo y $O(V+E)$ memoria.

4.11 Puentes y puntos de articulación

Cuándo usar: Detectar una arista o vértice cuya eliminación aumenta el número de componentes de un grafo no dirigido.

Versión para multigrafos no dirigidos: identifica la arista padre por ID, no solamente por vértice. La DFS usa una pila explícita en memoria dinámica y admite cadenas largas.

```
struct LowLink {
    int timer = 0;
    vector<vector<pair<int,int>>> graph; // (to, edge_id)
    vector<int> tin, low;
    vector<char> articulation;
    vector<pair<int,int>> bridges;

    explicit LowLink(int n) : graph(n), tin(n, -1), low(n),
                              articulation(n, false) {}

    void add_edge(int u, int v, int id) {
        graph[u].push_back({v, id});
        graph[v].push_back({u, id});
    }

    void run() {
        struct Frame { int v, parent_edge, next = 0, children = 0; };
        vector<Frame> pending;
        for (int root = 0; root < (int)graph.size(); ++root) {
            if (tin[root] != -1) continue;
            tin[root] = low[root] = timer++;
            pending.push_back({root, -1});
            while (!pending.empty()) {
                auto &frame = pending.back();
                int v = frame.v;
                if (frame.next < (int)graph[v].size()) {
                    auto [to, id] = graph[v][frame.next++];
                    if (id == frame.parent_edge) continue;
                    if (tin[to] != -1) low[v] = min(low[v], tin[to]);
                    else {
                        ++frame.children;
                        tin[to] = low[to] = timer++;
                        pending.push_back({to, id});
```

```
                    }
                } else {
                    if (frame.parent_edge == -1 && frame.children > 1)
                        articulation[v] = true;
                    pending.pop_back();
                    if (pending.empty()) continue;
                    int p = pending.back().v;
                    low[p] = min(low[p], low[v]);
                    if (low[v] > tin[p]) bridges.push_back({p, v});
                    if (pending.back().parent_edge != -1 && low[v] >= tin[p])
                        articulation[p] = true;
                }
            }
        }
    }
};

LowLink lowlink(5);

int id = 0;
lowlink.add_edge(0, 1, id++); // 5 veces con los q correspondan conexiones
lowlink.run();
for (auto [u, v] : lowlink.bridges)
    cout << "Puente: " << u << ' ' << v << '\n';
for (int v = 0; v < 5; ++v)
    if (lowlink.articulation[v])
        cout << "Articulacion: " << v << '\n';
```

Complejidad: $O(V+E)$ tiempo y $O(V+E)$ memoria.

4.12 Componentes biconexas por vértices

Cuándo usar: Separar un grafo no dirigido en bloques que no se desconectan al borrar un solo vértice; construir el block-cut tree o localizar articulaciones.

Una componente biconexa por vértices es un bloque maximal que no puede separarse eliminando uno de sus vértices. Los puntos de articulación pueden pertenecer a varios bloques; una arista puente forma por sí sola un bloque de dos vértices.

Por ejemplo:

```
0---1
 \ /
  2---3
      / \
     4---5
```

produce los bloques $\{0,1,2\}$, $\{2,3\}$ y $\{3,4,5\}$. Los vértices 2 y 3 son articulaciones porque aparecen en más de un bloque.

Durante el DFS se apilan las aristas recorridas. Si para un hijo `to` se cumple

$$\texttt{low[to]} \geq \texttt{discovery[vertex]},$$

su subárbol no puede alcanzar un antecesor de `vertex` sin pasar por él; las aristas de la pila hasta `vertex-to` forman un bloque completo.

Complejidad: Esta implementación: tiempo $O(n+m\log(m+1))$ y memoria $O(n+m)$. El DFS es lineal, pero ordenar los extremos de las aristas de cada bloque para eliminar repetidos agrega el factor logarítmico.

```
struct VertexBiconnectedComponents {
    struct StackedEdge { int from, to, id; };

    int timer = 0;
    vector<int> discovery, low;
    vector<StackedEdge> edge_stack;
    vector<vector<int>> components;
    const vector<vector<pair<int,int>>> &graph; // (to, edge_id)

    explicit VertexBiconnectedComponents(
            const vector<vector<pair<int,int>>> &input_graph)
        : discovery(input_graph.size(), -1),
          low(input_graph.size()), graph(input_graph) {
        for (int vertex = 0; vertex < (int)graph.size(); ++vertex) {
            if (discovery[vertex] != -1) continue;
            if (graph[vertex].empty()) {
                discovery[vertex] = low[vertex] = timer++;
                components.push_back({vertex});
            } else {
                dfs(vertex, -1);
            }
        }
    }

    void extract_component(int stop_edge) {
        vector<int> vertices;
        while (true) {
            auto edge = edge_stack.back();
            edge_stack.pop_back();
            vertices.push_back(edge.from);
            vertices.push_back(edge.to);
            if (edge.id == stop_edge) break;
        }
        sort(vertices.begin(), vertices.end());
        vertices.erase(unique(vertices.begin(), vertices.end()),
                       vertices.end());
        components.push_back(move(vertices));
```

```
    }

    void dfs(int vertex, int parent_edge) {
        discovery[vertex] = low[vertex] = timer++;

        for (auto [to, edge_id] : graph[vertex]) {
            if (edge_id == parent_edge) continue;

            if (discovery[to] == -1) {
                edge_stack.push_back({vertex, to, edge_id});
                dfs(to, edge_id);
                low[vertex] = min(low[vertex], low[to]);

                if (low[to] >= discovery[vertex])
                    extract_component(edge_id);
            } else if (discovery[to] < discovery[vertex]) {
                edge_stack.push_back({vertex, to, edge_id});
                low[vertex] = min(low[vertex], discovery[to]);
            }
        }
    }
};
```

Ejemplo de uso:

```
vector<vector<pair<int,int>>> graph(n);
for (int id = 0; id < m; ++id) {
    int u, v;
    cin >> u >> v;
    graph[u].push_back({v, id});
    graph[v].push_back({u, id});
}
VertexBiconnectedComponents bcc(graph);
for (const auto &component : bcc.components) {
    for (int vertex : component) cout << vertex << ' ';
    cout << '\n';
}
```

Un vértice es de articulación si aparece en más de un bloque:

```
vector<int> appearances(n);
for (const auto &component : bcc.components)
    for (int vertex : component)
        ++appearances[vertex];

for (int vertex = 0; vertex < n; ++vertex)
    if (appearances[vertex] > 1)
        cout << vertex << " es articulacion\n";
```

Cuidado: Cada arista no dirigida debe usar el mismo `edge_id` en ambas direcciones. Los vértices de articulación aparecen en varias componentes; esto es intencional.

4.13 Ancestro común más bajo (LCA)

Cuándo usar: Consultar ancestros comunes y distancias en un árbol enraizado; versión con arreglos de tamaño fijo.

Complejidad: Preproceso $O(NK)$ y consulta $O(K)$, con $K = O(\log N)$; memoria $O(K2^K)$ por los arreglos y $O(N)$ por la recursión.

`F[k][v]` guarda el ancestro 2^k de v y `D[v]` su profundidad. Un DFS calcula padres y profundidades; luego se construyen los saltos. Para consultar, igualar profundidades y subir ambos nodos, desde el salto mayor, mientras sus ancestros sean distintos. La distancia es $D[x] + D[y] - 2D[\text{lca}(x, y)]$. Elegir K con $2^K \geq N$; un árbol muy profundo puede agotar la pila del DFS.

```
const int K = 10;
vector<int> g[1<<K]; int N; // K de tal manera que 2^K>=N
int F[K][1<<K], D[1<<K]; // F[i][j] es el padre 2^i del nodo j
// F[i][j] donde 'i' representa la potencia de dos en la que
//se encuentra el padre de 'j'

void lca_dfs(int x){
    for(int hijo : g[x]){
        // Si es el padre, se continua
        if(hijo==F[0][x]) continue;
         // Se marca x como el padre del hijo
        F[0][hijo]=x;
        // La distancia del hijo es la del padre mas 1
        D[hijo]=D[x]+1;
        // Se recorren los hijos del nodo hijo
        lca_dfs(hijo);
    }
}

void lca_init(){
    D[0]=0;F[0][0]=-1;
    lca_dfs(0); // Se recorre el arbol
    // Para cada potencia de 2 y para cada nodo
    fore(k,1,K) fore(x,0,N){
        // Si el padre que esta en la potencia 'k-1' no existe,
        // entonces el padre en la potencia k, tampoco existe
        if(F[k-1][x]<0) F[k][x]=-1;
        // Si no, el padre es el padre 'k-1' del padre 'k-1' de x
        else F[k][x] = F[k-1][F[k-1][x]];
    }
}
```

```cpp

int lca(int x, int y){
    // Nos aseguramos de que el x sea el de mayor profundidad
    if (D[x]<D[y]) swap(x,y);
    // Buscamos el padre del nodo x que esta en la misma
    // profundidad que y.
    for(int k=K-1;k>=0;--k) if(D[x]-(1<<k)>=D[y]) x=F[k][x];
    if (x==y) return x; // Si son el mismo nodo, devolvemos x
    // Buscamos los primeros padres distintos de ambos nodos
    for(int k=K-1;k>=0;--k) if(F[k][x]!=F[k][y]) x=F[k][x], y=F[k][y];
    return F[0][x]; // Devolvemos su padre.
}
```

4.14 LCA con binary lifting

Cuándo usar: Encontrar el ancestro común más cercano, calcular distancias o subir k niveles en un árbol enraizado.

El LCA de dos vértices es el punto donde se unen sus caminos hacia la raíz. Se precomputa `up[k][v]`, el ancestro ubicado 2^k niveles arriba de v, para realizar saltos de $1, 2, 4, 8, \ldots$ niveles.

Para un árbol sin pesos:

$$dist(a, b) = depth[a] + depth[b] - 2\,depth[LCA(a, b)].$$

Complejidad: Preproceso $O(n \log n)$; consulta y salto $O(\log n)$; memoria $O(n \log n)$.

```cpp
struct LCA {
    int n, levels;
    vector<int> depth;
    vector<vector<int>> up; // up[k][v]: ancestro 2^k de v
    LCA(const vector<vector<int>> &tree, int root = 0)
        : n(tree.size()) {
        assert(n > 0 && 0 <= root && root < n);
        levels = 1;
        while ((1 << levels) <= n) ++levels;
        depth.assign(n, 0);
        vector<int> parent(n, -1);
        // Enraizar el arbol y calcular padres y profundidades.
        queue<int> q;
        parent[root] = root;
        q.push(root);
        while (!q.empty()) {
            int v = q.front();
            q.pop();

            for (int to : tree[v]) {
                if (parent[to] != -1) continue;
                parent[to] = v;
                depth[to] = depth[v] + 1;
                q.push(to);
            }
        }
        // Construir la tabla de saltos.
        up.assign(levels, vector<int>(n));
        up[0] = parent;
        for (int k = 1; k < levels; ++k)
            for (int v = 0; v < n; ++v)
                up[k][v] = up[k - 1][up[k - 1][v]];
    }
    // Sube distance niveles desde v.
    int jump(int v, int distance) const {
        for (int k = 0; k < levels; ++k)
            if (distance & (1 << k))
                v = up[k][v];
        return v;
    }
    // Ancestro comun mas cercano de a y b.
    int query(int a, int b) const {
        // Igualar profundidades.
        if (depth[a] < depth[b]) swap(a, b);
        a = jump(a, depth[a] - depth[b]);
        if (a == b) return a;
        // Subir ambos sin alcanzar todavia el LCA.
        for (int k = levels - 1; k >= 0; --k) {
            if (up[k][a] != up[k][b]) {
                a = up[k][a];
                b = up[k][b];
            }
        }
        return up[0][a];
    }
    // Cantidad de aristas en el camino entre a y b.
    int distance(int a, int b) const {
        int ancestor = query(a, b);
        return depth[a] + depth[b] - 2 * depth[ancestor];
    }
};
vector<vector<int>> tree(n);
auto add_edge = [&](int u, int v) {
    tree[u].push_back(v);
    tree[v].push_back(u);
};
add_edge(0, 1);
add_edge(0, 2);
add_edge(1, 3);
```

```
add_edge(1, 4);
LCA lca(tree, 0);
cout << lca.query(3, 4) << '\n';    // 1
cout << lca.jump(3, 2) << '\n';     // 0
cout << lca.distance(3, 2) << '\n'; // 3
```

Cuidado: El grafo debe ser un árbol conectado. `jump(v,k)` supone $0 \le k \le depth[v]$; la distancia devuelta cuenta aristas.

4.15 LCA $O(1)$ con Euler tour y RMQ

Cuándo usar: Árbol estático con muchísimas consultas de LCA. Usar binary lifting si también se necesita subir k niveles o ahorrar memoria.

El Euler tour guarda un vértice al visitarlo y cada vez que el DFS regresa a él. El LCA de a y b es el vértice de menor profundidad entre sus primeras apariciones en el recorrido. Una Sparse Table responde este mínimo en $O(1)$. La DFS usa marcos explícitos para registrar las entradas y los regresos, sin consumir pila de llamadas proporcional a la altura.

Complejidad: Preproceso $O(n \log n)$; consulta $O(1)$; memoria $O(n \log n)$.

```
struct EulerRMQLCA {
    vector<int> first, depth, euler, logarithm;
    vector<vector<int>> table;
    EulerRMQLCA(const vector<vector<int>> &tree, int root = 0)
        : first(tree.size(), -1), depth(tree.size()) {
        assert(!tree.empty());
        dfs(root, -1, tree);
        int length = euler.size();
        logarithm.assign(length + 1, 0);
        for (int i = 2; i <= length; ++i)
            logarithm[i] = logarithm[i / 2] + 1;
        // table[k][i]: vertice de menor profundidad
        // en euler[i .. i + 2^k).
        table.push_back(euler);

        for (int k = 1; (1 << k) <= length; ++k) {
            int half = 1 << (k - 1);
            int count = length - (1 << k) + 1;
            table.push_back(vector<int>(count));
            for (int i = 0; i < count; ++i) {
                int a = table[k - 1][i];
                int b = table[k - 1][i + half];
                table[k][i] = depth[a] < depth[b] ? a : b;
            }
        }
    }
    void dfs(int v, int parent,
             const vector<vector<int>> &tree) {
        struct Frame { int v, parent, next = 0; };
        vector<Frame> pending{{v, parent}};
        first[v] = euler.size();
        euler.push_back(v);
        while (!pending.empty()) {
            auto &frame = pending.back();
            int from = frame.v;
            if (frame.next == (int)tree[from].size()) {
                pending.pop_back();
                if (!pending.empty()) euler.push_back(pending.back().v);
            } else {
                int to = tree[from][frame.next++];
                if (to == frame.parent) continue;
                depth[to] = depth[from] + 1;
                first[to] = euler.size(); euler.push_back(to);
                pending.push_back({to, from});
            }
        }
    }
    int lca(int a, int b) const {
        int left = first[a], right = first[b];
        if (left > right) swap(left, right);
        int k = logarithm[right - left + 1];
        int x = table[k][left];
        int y = table[k][right - (1 << k) + 1];

        return depth[x] < depth[y] ? x : y;
    }
    int distance(int a, int b) const {
        int ancestor = lca(a, b);
        return depth[a] + depth[b] - 2 * depth[ancestor];
    }
};

EulerRMQLCA lca(tree, 0);
cout << lca.lca(3, 4) << '\n';      // Ancestro comun
cout << lca.distance(3, 2) << '\n'; // Distancia en aristas
```

Cuidado: El grafo debe ser un árbol conectado no vacío. Los marcos de DFS se guardan en memoria dinámica; no se necesita aumentar el límite de pila.

4.16 Virtual Tree

Cuándo usar: Cada consulta marca pocos vértices de un árbol grande y sólo importa la unión de los caminos que los conectan.

El árbol virtual comprime el árbol original conservando únicamente los vértices marcados y los LCA necesarios para conectarlos. Una arista virtual (u, v) representa el camino completo desde el ancestro u hasta v en el árbol original.

Por ejemplo, si los marcados son a, b, c, puede ser necesario agregar $LCA(a, b)$ para conectarlos. No se agregan los vértices intermedios que no cambian la estructura de los caminos.

Requiere precomputar `tin`, `tout` y LCA. La función devuelve los vértices virtuales y sus aristas `(padre,hijo)`.

Complejidad: Por consulta $O(k \log k + kT_{LCA})$ tiempo y $O(k)$ memoria adicional, donde T_{LCA} es el costo de consultar un LCA. Con Euler tour y RMQ, $T_{LCA} = O(1)$; con binary lifting, $T_{LCA} = O(\log n)$ y el tiempo es $O(k \log k + k \log n)$. Estas cotas excluyen el preproceso y almacenamiento del LCA, `tin` y `tout`. Para $k \geq 1$ marcados se crean como máximo $2k - 1$ vértices; sin marcados se devuelve un árbol vacío.

```cpp
template<class LCAFunction>
pair<vector<int>,vector<pair<int,int>>> build_virtual_tree(
        vector<int> marked,const vector<int>& tin,const vector<int>& tout,
        LCAFunction get_lca) {
    auto ancestor=[&](int a,int b) {
        return tin[a]<=tin[b]&&tout[b]<=tout[a];
    };
    auto by_tin=[&](int a,int b) { return tin[a]<tin[b]; };
    sort(marked.begin(),marked.end(),by_tin);
    marked.erase(unique(marked.begin(),marked.end()),marked.end());
    int original_size=marked.size();
    for(int i=1;i<original_size;++i)
        marked.push_back(get_lca(marked[i-1],marked[i]));
    sort(marked.begin(),marked.end(),by_tin);
    marked.erase(unique(marked.begin(),marked.end()),marked.end());
    vector<pair<int,int>> edges;
    vector<int> st;
    for(int vertex:marked) {
        while(!st.empty()&&!ancestor(st.back(),vertex)) st.pop_back();
        if(!st.empty()) edges.push_back({st.back(),vertex});
        st.push_back(vertex);
    }
    return {marked,edges};
}

// Uso
auto [vertices,edges]=build_virtual_tree(
    marked,tin,tout,[&](int a,int b){ return lca.query(a,b); });
for(auto [parent,child]:edges)
    cout<<parent<<" "<<child<<" "
        <<depth[child]-depth[parent]<<'\n';
```

Si el árbol original tiene pesos, reemplazar la diferencia de profundidades por:

$$distRoot[child] - distRoot[parent].$$

Cuidado: Los LCA agregados no necesariamente estaban marcados originalmente. Guardar aparte los marcados si la DP necesita distinguirlos. Incluir la raíz original explícitamente sólo si la consulta necesita que forme parte del árbol virtual.

4.17 Heavy-Light Decomposition

Cuándo usar: Updates o consultas sobre caminos y subárboles, combinando el árbol con Fenwick o Segment Tree.

Para cada vértice se elige como hijo pesado al hijo con mayor subárbol. Las aristas pesadas forman cadenas cuyos vértices ocupan posiciones consecutivas. Así, un camino se divide en $O(\log n)$ intervalos del arreglo base. La construcción usa recorridos iterativos, sin depender de la altura para consumir pila de llamadas.

Complejidad: Build $O(n)$; camino con Segment Tree $O(\log^2 n)$; subárbol $O(\log n)$; memoria $O(n)$.

```cpp
struct HLD {
    int n,timer=0;
    vector<vector<int>> tree;
    vector<int> parent,depth,size,heavy,head,pos;
    HLD(vector<vector<int>> tree,int root=0):n(tree.size()),
    tree(move(tree)),parent(n,-1),depth(n),size(n),heavy(n,-1),
        head(n),pos(n) {
        assert(n>0);
        vector<int> order{root};
        for(int at=0;at<(int)order.size();++at) {
            int v=order[at];
            for(int to:this->tree[v]) if(to!=parent[v]) {
                parent[to]=v; depth[to]=depth[v]+1; order.push_back(to);
            }
        }
        for(int at=n-1;at>=0;--at) {
            int v=order[at]; size[v]=1;
            for(int to:this->tree[v]) if(to!=parent[v]) {
                size[v]+=size[to];
                if(heavy[v]==-1||size[to]>size[heavy[v]]) heavy[v]=to;
            }
        }
        head[root]=root; order.clear(); order.push_back(root);
        while(!order.empty()) {
            int v=order.back(); order.pop_back(); pos[v]=timer++;
            for(auto it=this->tree[v].rbegin();it!=this->tree[v].rend();++it)
                if(*it!=parent[v]&&*it!=heavy[v]) {
                    head[*it]=*it; order.push_back(*it);
```

```
                }
            if(heavy[v]!=-1) {
                head[heavy[v]]=head[v]; order.push_back(heavy[v]);
            }
        }
    }
    // Intervalos [l,r) que cubren el camino a--b.
    template<class Callback>
    void for_each_path(int a,int b,Callback callback) const {
        while(head[a]!=head[b]) {
            if(depth[head[a]]<depth[head[b]]) swap(a,b);
            callback(pos[head[a]],pos[a]+1);
            a=parent[head[a]];
        }
        if(depth[a]>depth[b]) swap(a,b);
        callback(pos[a],pos[b]+1);
    }
    // template<class Callback>
    // void for_each_edge_path(int a, int b, Callback callback) const {
    //     while (head[a] != head[b]) {
    //         if (depth[head[a]] < depth[head[b]]) swap(a,b);
    //         callback(pos[head[a]], pos[a]+1);
    //         a = parent[head[a]];
    //     }
    //     if (depth[a] > depth[b]) swap(a,b);
    //     if (a != b) callback(pos[a]+1, pos[b]+1); // Excluye solo el LCA.
    // }
    // Los vertices del subarbol de v son consecutivos.
    pair<int,int> subtree_interval(int v) const {
        return {pos[v],pos[v]+size[v]};
    }
};
```

Ejemplo con un Segment Tree de suma:

```
HLD hld(tree,0);
vector<ll> base(n);
for(int v=0;v<n;++v) base[hld.pos[v]]=value[v];
SegmentTree segment(base);

// Cambiar el valor del vertice v.
segment.set_value(hld.pos[v],new_value);

// Suma de los vertices del camino a--b.
ll path_sum=0;
hld.for_each_path(a,b,[&](int l,int r) {
    path_sum+=segment.query(l,r);
});

// Suma del subarbol de v.
auto [l,r]=hld.subtree_interval(v);
ll subtree_sum=segment.query(l,r);
```

Para guardar valores en aristas, almacenar cada arista en la posición de su vértice hijo y dejar cero en la raíz. Activar `for_each_edge_path`: excluye el LCA sólo en el último segmento; los anteriores se incluyen completos. Si los extremos coinciden, el camino no tiene aristas. Mantiene $O(\log^2 n)$ por consulta con Segment Tree y $O(1)$ memoria adicional.

```
// void hld_edge_example() {
//     vector<vector<int>> tree = {{1,2}, {0,3,4}, {0}, {1}, {1}};
//     HLD hld(tree,0);
//     vector<ll> edge_to_parent = {0,4,7,2,3}, base(5);
//     for (int v = 0; v < 5; ++v) base[hld.pos[v]] = edge_to_parent[v];
//     SegmentTree segment(base);
//     auto path_sum = [&](int a, int b) {
//         ll sum = 0;
//         hld.for_each_edge_path(a,b,[&](int l, int r) { sum += segment.query(l
    ,r); });
//         return sum;
//     };
//     cout << path_sum(3,2) << '\n'; // 13: 2+4+7
//     segment.set_value(hld.pos[3],5); // Cambia la arista 1--3.
//     cout << path_sum(3,2) << '\n'; // 16
//     cout << path_sum(1,1) << '\n'; // 0
// }
```

Cuidado: El callback no entrega necesariamente los segmentos en el orden del camino. Esto no importa para suma, mínimo, máximo o XOR; para operaciones no conmutativas se deben separar y orientar los segmentos de ambos lados.

4.18 Descomposición por centroides

Cuándo usar: Dividir un árbol en componentes balanceadas para contar caminos o mantener consultas de distancia, por ejemplo al vértice marcado más cercano.

Un centroide de una componente de tamaño s es un vértice cuya eliminación deja componentes de tamaño como máximo $s/2$. Se elige un centroide y se descompone recursivamente cada componente restante. El árbol de centroides tiene altura $O(\log n)$; sus aristas no son necesariamente aristas originales.

Complejidad: Construcción $O(n \log n)$ tiempo y $O(n)$ memoria, incluyendo la pila de los DFS. La profundidad de la descomposición es $O(\log n)$, pero un DFS auxiliar puede alcanzar profundidad $O(n)$.

```
struct CentroidDecomposition {
    const vector<vector<int>> &tree;
```

```
    vector<int> size, parent;
    vector<char> removed;
    int root = -1;

    explicit CentroidDecomposition(const vector<vector<int>> &input_tree)
        : tree(input_tree), size(tree.size()), parent(tree.size(), -1),
          removed(tree.size(), false) {
        if (!tree.empty()) root = decompose(0, -1);
    }

    int calculate_size(int v, int previous) {
        size[v] = 1;
        for (int to : tree[v])
            if (to != previous && !removed[to])
                size[v] += calculate_size(to, v);
        return size[v];
    }

    int find_centroid(int v, int previous, int total) {
        for (int to : tree[v])
            if (to != previous && !removed[to] && size[to] > total / 2)
                return find_centroid(to, v, total);
        return v;
    }

    int decompose(int entry, int previous_centroid) {
        int total = calculate_size(entry, -1);
        int centroid = find_centroid(entry, -1, total);
        parent[centroid] = previous_centroid;
        removed[centroid] = true;
        for (int to : tree[centroid])
            if (!removed[to]) decompose(to, centroid);
        return centroid;
    }
};
```

`parent[v]` es el padre de v en el árbol de centroides; la raíz tiene padre -1. La lista de adyacencia original no se modifica. Al terminar, todos los vértices quedan marcados en `removed`.

```
void centroid_example() {
    vector<vector<int>> tree = {{1}, {0, 2, 3}, {1}, {1, 4}, {3}};
    CentroidDecomposition decomposition(tree);
    cout << decomposition.root << '\n'; // 1

    // Ancestros centroides del vertice 4, incluyendo al propio 4.
    for (int v = 4; v != -1; v = decomposition.parent[v])
        cout << v << ' '; // 4 3 1
}
```

Para consultas de distancias, agregar la información del problema en cada centroide y recorrer estos ancestros. Las distancias deben medirse en el árbol original, mediante LCA o precomputándolas durante la construcción.

Cuidado: Requiere un árbol no dirigido conectado; los vértices son 0-indexados. El constructor conserva una referencia: el árbol debe vivir más que la estructura. Los DFS recursivos pueden desbordar la pila en cadenas largas; usar recorridos iterativos si los límites lo requieren.

4.19 Rerooting DP

Cuándo usar: Se necesita una respuesta para cada vértice considerando que ese vértice es la raíz o punto de inicio: suma de distancias a todos, cantidad de nodos alcanzables, mejor costo al elegir una capital o valor de una DP para cada raíz. La respuesta de raíces vecinas debe poder actualizarse reutilizando la información de sus subárboles. Evita ejecutar una DFS $O(n)$ desde cada vértice, lo que costaría $O(n^2)$.

Complejidad: tiempo $O(n)$; memoria $O(n)$.

Se calcula un orden padre–hijos y se recorre en ambos sentidos, sin DFS recursiva. El grafo debe ser un árbol conectado; se admite el árbol vacío.

```
struct DistanceRerooting {
    int node_count;
    const vector<vector<int>> &tree;
    vector<int> subtree_size;
    vector<ll> distance_sum;

    explicit DistanceRerooting(const vector<vector<int>> &input_tree)
        : node_count(input_tree.size()), tree(input_tree),
          subtree_size(node_count, 1), distance_sum(node_count) {
        if (!node_count) return;
        vector<int> parent(node_count, -1), order{0};
        for (int at = 0; at < node_count; ++at) {
            int v = order[at];
            for (int to : tree[v]) if (to != parent[v]) {
                parent[to] = v; order.push_back(to);
            }
        }
        for (int at = node_count - 1; at > 0; --at) {
            int v = order[at], p = parent[v];
            subtree_size[p] += subtree_size[v];
            distance_sum[p] += distance_sum[v] + subtree_size[v];
        }
        for (int at = 1; at < node_count; ++at) {
            int v = order[at];
            distance_sum[v] = distance_sum[parent[v]]
```

```
                            + node_count - 2 * subtree_size[v];
        }
    }
};
```

```
tree[u].push_back(v);
tree[v].push_back(u);

DistanceRerooting rerooting(tree);
for(int v=0;v<n;++v)
    cout<<rerooting.distance_sum[v]<<'\n';
```

Para una combinación genérica no invertible, usar prefijos y sufijos de las contribuciones de los hijos para excluir uno en $O(1)$.

4.20 Diámetro de un àrbol y camino

Cuándo usar: Hallar el camino más largo de un àrbol con pesos no negativos. Con pesos uno, su centro y radio también ayudan a conectar varios àrboles.

Complejidad: Dos recorridos: tiempo y memoria $O(n)$. Son iterativos para admitir cadenas largas sin agotar la pila.

```
pair<ll, vector<int>> tree_diameter(
        const vector<vector<pair<int,ll>>> &tree) {
    int n = tree.size();
    if (n == 0) return {0, {}};
    vector<ll> distance(n);
    vector<int> parent(n);
    auto farthest = [&](int source) {
        fill(distance.begin(), distance.end(), -1);
        fill(parent.begin(), parent.end(), -1);
        vector<int> order{source};
        distance[source] = 0;
        for (int i = 0; i < (int)order.size(); ++i) {
            int v = order[i];
            for (auto [to, weight] : tree[v]) if (distance[to] == -1) {
                assert(weight >= 0);
                distance[to] = distance[v] + weight;
                parent[to] = v; order.push_back(to);
            }
        }
        assert((int)order.size() == n);
        return (int)(max_element(distance.begin(), distance.end()) - distance.
    begin());
    };
    int start = farthest(0), finish = farthest(start);
    vector<int> path;
    for (int v = finish; v != -1; v = parent[v]) path.push_back(v);
    reverse(path.begin(), path.end());
    return {distance[finish], path};
}
void tree_diameter_example() {
    vector<vector<pair<int,ll>>> tree(4);
    auto add = [&](int a, int b, ll w) {
        tree[a].push_back({b,w}); tree[b].push_back({a,w});
    };
    add(0,1,2); add(1,2,5); add(1,3,4);
    auto [length, path] = tree_diameter(tree);
    cout << length << '\n'; // 9; camino 2--1--3 (o su inverso).
}
```

Cuidado: Requiere un àrbol conectado, cada arista en ambos sentidos y distancias que entren en `long long`. Con pesos negativos, usar DP de àrbol en lugar de estos dos recorridos. Para un bosque, procesar componentes.

Si todas las aristas pesan uno, el radio es $\lceil diametro/2 \rceil$ y un centro es `path[path.size()/2]`. Es la herramienta base de Adjoin the Networks (NCPC 2015); no resolver cada componente recorriendo de nuevo todos los vértices del bosque.

4.21 Camino euleriano

Cuándo usar: Recorrer cada arista exactamente una vez; también cuando objetos son aristas entre dos tipos de entidad, como filas y columnas.

Hierholzer usa cada arista exactamente una vez. El chequeo final de longitud tambíen valida que todas las aristas pertenezcan al componente recorrido.

Complejidad: $O(V + E)$ tiempo y memoria.

```
vector<int> directed_eulerian_path(
        const vector<vector<pair<int,int>>> &graph, int edge_count) {
    int n = graph.size();
    if (n == 0) return edge_count == 0 ? vector<int>{} : vector<int>{};
    vector<int> indegree(n, 0), outdegree(n, 0);
    for (int v = 0; v < n; ++v)
        for (auto [to, id] : graph[v]) {
            (void)id;
            ++outdegree[v]; ++indegree[to];
        }

    int start = -1, finish = -1;
    for (int v = 0; v < n; ++v) {
        int difference = outdegree[v] - indegree[v];
        if (difference == 1) {
            if (start != -1) return {};
            start = v;
        } else if (difference == -1) {
```

```
            if (finish != -1) return {};
            finish = v;
        } else if (difference != 0) return {};
    }
    if ((start == -1) != (finish == -1)) return {};
    if (start == -1) {
        start = 0;
        while (start < n && outdegree[start] == 0) ++start;
        if (start == n) return {0};
    }

    vector<int> next_edge(n, 0), stack{start}, path;
    vector<char> used(edge_count, false);
    while (!stack.empty()) {
        int v = stack.back();
        while (next_edge[v] < (int)graph[v].size() &&
               used[graph[v][next_edge[v]].second]) ++next_edge[v];
        if (next_edge[v] == (int)graph[v].size()) {
            path.push_back(v); stack.pop_back();
        } else {
            auto [to, id] = graph[v][next_edge[v]++];
            if (id < 0 || id >= edge_count) return {};
            used[id] = true;
            stack.push_back(to);
        }
    }
    if ((int)path.size() != edge_count + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}

vector<int> undirected_eulerian_path(
        const vector<vector<pair<int,int>>> &graph, int edge_count) {
    int n = graph.size();
    if (n == 0) return {};
    vector<int> odd;
    for (int v = 0; v < n; ++v)
        if (graph[v].size() % 2 == 1) odd.push_back(v);
    if (!odd.empty() && odd.size() != 2) return {};

    int start = odd.empty() ? 0 : odd[0];
    while (start < n && graph[start].empty()) ++start;
    if (start == n) return {0};

    vector<int> next_edge(n, 0), stack{start}, path;
    vector<char> used(edge_count, false);
    while (!stack.empty()) {
        int v = stack.back();
        while (next_edge[v] < (int)graph[v].size() &&
               used[graph[v][next_edge[v]].second]) ++next_edge[v];
        if (next_edge[v] == (int)graph[v].size()) {
            path.push_back(v); stack.pop_back();
        } else {
            auto [to, id] = graph[v][next_edge[v]++];
            if (id < 0 || id >= edge_count) return {};
            used[id] = true;
            stack.push_back(to);
        }
    }
    if ((int)path.size() != edge_count + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}
```

En el grafo no dirigido, cada arista debe aparecer dos veces en la lista de adyacencia con el mismo ID. En el dirigido aparece una sola vez.

4.22 Flujo máximo (Ford–Fulkerson)

Cuándo usar: Red pequeña con capacidades enteras y flujo total bajo, o como implementación conceptual rápida. Para casos grandes preferir Dinic.

Busca caminos aumentantes con DFS y actualiza el grafo residual. Esta versión usa capacidades enteras no negativas.

Cada arista representa cuánto flujo puede enviarse desde `source` hasta `sink`. Al enviar x, se resta x de la capacidad directa y se suma a la inversa; esta arista residual permite deshacer o redirigir flujo anterior. Ford–Fulkerson repite: buscar un camino con capacidad, enviar el mínimo de sus capacidades y terminar cuando ya no existe otro camino.

```
struct FordFulkerson {
    struct Edge { int to, reverse; ll capacity; };
    int n;
    vector<vector<Edge>> graph;
    vector<char> seen;

    explicit FordFulkerson(int size)
        : n(size), graph(size), seen(size) {}

    void add_edge(int from, int to, ll capacity) {
        assert(capacity >= 0);
        Edge forward{to, (int)graph[to].size(), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
```

```
    }

    ll dfs(int vertex, int sink, ll pushed) {
        if (vertex == sink) return pushed;
        seen[vertex] = true;
        for (Edge &edge : graph[vertex]) {
            if (seen[edge.to] || edge.capacity == 0) continue;
            ll flow = dfs(edge.to, sink, min(pushed, edge.capacity));
            if (flow == 0) continue;
            edge.capacity -= flow;
            graph[edge.to][edge.reverse].capacity += flow;
            return flow;
        }
        return 0;
    }

    ll max_flow(int source, int sink) {
        assert(source != sink);
        ll answer = 0;
        while (true) {
            fill(seen.begin(), seen.end(), false);
            ll pushed = dfs(source, sink, LINF);
            if (pushed == 0) break;
            answer += pushed;
        }
        return answer;
    }
};
```

Complejidad: $O(EF)$ para capacidades enteras, donde F es el flujo máximo; es pseudopolinomial; memoria $O(V+E)$. Con capacidades grandes suele convenir Dinic.

4.23 Flujo máximo (Dinic)

Cuándo usar: Capacidades, cortes mínimos, asignaciones con cupos y selección con restricciones que se modelan como una red fuente–sumidero.

Calcula el mismo flujo máximo que Ford–Fulkerson, pero agrupa los aumentos. Un BFS construye niveles usando aristas residuales y los DFS envían todo el flujo posible avanzando sólo al nivel siguiente. `next_edge` evita revisar aristas que ya se demostraron inútiles en la fase actual. Es la opción usual de flujo máximo en competencia.

```
struct Dinic {
    struct Edge { int to, reverse; ll capacity; };
    int n;
    vector<vector<Edge>> graph;
    vector<int> level, next_edge;

    explicit Dinic(int size)
        : n(size), graph(size), level(size), next_edge(size) {}

    void add_edge(int from, int to, ll capacity) {
        Edge forward{to, (int)graph[to].size(), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    bool bfs(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[source] = 0; q.push(source);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (const Edge &edge : graph[v])
                if (edge.capacity > 0 && level[edge.to] == -1)
                    level[edge.to] = level[v] + 1, q.push(edge.to);
        }
        return level[sink] != -1;
    }

    ll dfs(int v, int sink, ll pushed) {
        if (v == sink || pushed == 0) return pushed;
        for (int &i = next_edge[v]; i < (int)graph[v].size(); ++i) {
            Edge &edge = graph[v][i];
            if (edge.capacity == 0 || level[edge.to] != level[v] + 1) continue;
            ll flow = dfs(edge.to, sink, min(pushed, edge.capacity));
            if (flow == 0) continue;
            edge.capacity -= flow;
            graph[edge.to][edge.reverse].capacity += flow;
            return flow;
        }
        return 0;
    }

    ll max_flow(int source, int sink) {
        assert(source != sink);
        ll answer = 0;
        while (bfs(source, sink)) {
            fill(next_edge.begin(), next_edge.end(), 0);
            while (ll pushed = dfs(source, sink, LINF)) answer += pushed;
        }
        return answer;
    }

    // pair<int,int> add_tracked_edge(int from, int to, ll capacity) {
    //     assert(from != to && capacity >= 0);
```

```
    //     int index = graph[from].size();
    //     add_edge(from, to, capacity);
    //     return {from, index}; // ID estable de la arista original.
    // }
    // ll edge_flow(pair<int,int> id) const {
    //     const Edge& edge = graph[id.first][id.second];
    //     return graph[edge.to][edge.reverse].capacity;
    // }
    // vector<char> min_cut_side(int source) const {
    //     vector<char> side(n, false);
    //     queue<int> q; q.push(source); side[source] = true;
    //     while (!q.empty()) {
    //         int v = q.front(); q.pop();
    //         for (const Edge& edge : graph[v])
    //             if (edge.capacity > 0 && !side[edge.to]) {
    //                 side[edge.to] = true; q.push(edge.to);
    //             }
    //     }
    //     return side;
    // }
};
```

Complejidad: general $O(V^2E)$ tiempo; memoria $O(V+E)$; suele ser mucho más rápido en práctica.

Opcional: activar los helpers comentados y guardar el ID al crear cada arista con extremos distintos. `edge_flow` cuesta $O(1)$. Después de `max_flow`, `min_cut_side` devuelve el lado de la fuente en $O(V+E)$ tiempo y $O(V)$ memoria; el corte cruza de ese lado al otro. Capacidades no negativas y flujo total representable en `long long`.

```
// void dinic_recovery_example() {
//     Dinic flow(4);
//     vector<pair<int,int>> ids;
//     ids.push_back(flow.add_tracked_edge(0,1,3));
//     ids.push_back(flow.add_tracked_edge(0,2,2));
//     ids.push_back(flow.add_tracked_edge(1,2,1));
//     ids.push_back(flow.add_tracked_edge(1,3,2));
//     ids.push_back(flow.add_tracked_edge(2,3,3));
//     cout << flow.max_flow(0,3) << '\n'; // 5
//     auto side = flow.min_cut_side(0);
//     for (auto id : ids) {
//         const auto& edge = flow.graph[id.first][id.second];
//         cout << id.first << "->" << edge.to << ": " << flow.edge_flow(id) <<
    '\n';
//         if (side[id.first] && !side[edge.to])
//             cout << "Corte: " << id.first << "->" << edge.to << '\n';
//     }
// }
```

4.24 Flujo de costo mínimo

Cuándo usar: Enviar cierta cantidad de flujo optimizando un costo: transporte, matching ponderado con capacidades o asignación de recursos.

SPFA por aumento; admite costos negativos si no hay ciclos negativos de costo alcanzables con capacidad residual positiva.

Cada arista tiene capacidad y costo por unidad. En cada iteración se busca el camino residual de menor costo, se envía todo lo permitido y se acumula `flujo` × `costo`. `run(s,t,k)` intenta enviar hasta k unidades y devuelve `{flujo_enviado,costo_total}`; si no existe capacidad suficiente, el flujo devuelto es menor que k.

```
struct MinCostMaxFlow {
    struct Edge { int to, reverse; ll capacity, cost; };
    int n;
    vector<vector<Edge>> graph;

    explicit MinCostMaxFlow(int size) : n(size), graph(size) {}

    void add_edge(int from, int to, ll capacity, ll cost) {
        Edge forward{to, (int)graph[to].size(), capacity, cost};
        Edge backward{from, (int)graph[from].size(), 0, -cost};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    pair<ll,ll> run(int source, int sink, ll flow_limit = LINF) {
        ll total_flow = 0, total_cost = 0;
        vector<ll> distance(n);
        vector<int> parent_vertex(n), parent_edge(n);
        while (total_flow < flow_limit) {
            fill(distance.begin(), distance.end(), LINF);
            vector<char> in_queue(n, false);
            queue<int> q;
            distance[source] = 0; q.push(source); in_queue[source] = true;
            while (!q.empty()) {
                int v = q.front(); q.pop(); in_queue[v] = false;
                for (int i = 0; i < (int)graph[v].size(); ++i) {
                    Edge const &edge = graph[v][i];
                    if (edge.capacity == 0 ||
                        distance[edge.to] <= distance[v] + edge.cost) continue;
                    distance[edge.to] = distance[v] + edge.cost;
                    parent_vertex[edge.to] = v; parent_edge[edge.to] = i;
                    if (!in_queue[edge.to]) q.push(edge.to), in_queue[edge.to] =
     true;
                }
            }
            if (distance[sink] == LINF) break;
```

```
            ll pushed = flow_limit - total_flow;
            for (int v = sink; v != source; v = parent_vertex[v])
                pushed = min(pushed,
                    graph[parent_vertex[v]][parent_edge[v]].capacity);
            for (int v = sink; v != source; v = parent_vertex[v]) {
                Edge &edge = graph[parent_vertex[v]][parent_edge[v]];
                edge.capacity -= pushed;
                graph[v][edge.reverse].capacity += pushed;
            }
            total_flow += pushed;
            total_cost += pushed * distance[sink];
        }
        return {total_flow, total_cost};
    }
};
```

Complejidad: $O(FVE)$ en el peor caso para flujo entero F; memoria $O(V+E)$.

4.24.1 Elección rápida

- Sólo maximizar cantidad: preferir **Dinic**.
- Maximizar cantidad y minimizar costo: **Min-Cost Max-Flow**.
- Ford–Fulkerson: implementación conceptual o flujo total pequeño.

En matching bipartito se modela `source` → `izquierda` → `derecha` → `sink`, normalmente con capacidades 1. En la versión ponderada, el costo de la arista central es el costo de realizar esa asignación.

4.25 Matching bipartito: Hopcroft–Karp

Cuándo usar: Emparejar el máximo número de elementos entre dos conjuntos con compatibilidades binarias y sin costos.

Los vértices izquierdos son $[0, n)$ y los derechos $[0, m)$.

Una arista (l, r) indica que el elemento izquierdo l puede emparejarse con el derecho r; cada vértice puede pertenecer a lo sumo a una pareja. Hopcroft–Karp busca caminos aumentantes: alterna aristas fuera/dentro del matching para reemplazar parejas y aumentar su tamaño en uno. El BFS agrupa los caminos aumentantes más cortos por niveles y los DFS procesan varios en la misma fase. `maximum_matching()` devuelve la cantidad de parejas; `match_left[l]` y `match_right[r]` contienen su pareja o -1.

Complejidad: $O(E\sqrt{V})$ tiempo y $O(V+E)$ memoria.

```
struct HopcroftKarp {
    int left_size, right_size;
    vector<vector<int>> graph;
    vector<int> match_left, match_right, distance;

    HopcroftKarp(int left_count, int right_count)
        : left_size(left_count), right_size(right_count), graph(left_count),
          match_left(left_count, -1), match_right(right_count, -1),
          distance(left_count) {}

    void add_edge(int left, int right) { graph[left].push_back(right); }

    bool bfs() {
        queue<int> q;
        fill(distance.begin(), distance.end(), -1);
        for (int left = 0; left < left_size; ++left)
            if (match_left[left] == -1) distance[left] = 0, q.push(left);
        bool augmenting_path = false;
        while (!q.empty()) {
            int left = q.front(); q.pop();
            for (int right : graph[left]) {
                int next_left = match_right[right];
                if (next_left == -1) augmenting_path = true;
                else if (distance[next_left] == -1) {
                    distance[next_left] = distance[left] + 1;
                    q.push(next_left);
                }
            }
        }
        return augmenting_path;
    }

    bool dfs(int left) {
        for (int right : graph[left]) {
            int next_left = match_right[right];
            if (next_left == -1 ||
                (distance[next_left] == distance[left] + 1 && dfs(next_left))) {
                match_left[left] = right;
                match_right[right] = left;
                return true;
            }
        }
        distance[left] = -1;
        return false;
    }

    int maximum_matching() {
        int matching = 0;
        while (bfs())
            for (int left = 0; left < left_size; ++left)
                if (match_left[left] == -1 && dfs(left)) ++matching;
        return matching;
    }
};
```

Ejemplo: asignar el máximo número de personas a trabajos compatibles.

```
HopcroftKarp matching(3,3);
matching.add_edge(0,0); matching.add_edge(0,1);
matching.add_edge(1,1); matching.add_edge(2,1);
int pairs=matching.maximum_matching();
for(int person=0;person<3;++person)
    cout<<person<<" -> "<<matching.match_left[person]<<'\n';
```

4.26 Asignación de costo mínimo: Húngaro

Cuándo usar: Asignar cada elemento de un lado a uno distinto del otro minimizando un costo total en una matriz completa o casi completa.

Matriz rectangular con $n \le m$. Devuelve costo y columna asignada a cada fila. Admite costos negativos.

`cost[i][j]` es el costo de asignar la fila i a la columna j; todas las filas reciben columnas distintas. El algoritmo incorpora una fila por vez y usa potenciales para conservar costos reducidos no negativos mientras encuentra la reasignación más barata. Devuelve `{costo,assignment}`, donde `assignment[i]` es la columna elegida para la fila i.

Complejidad: $O(n^2 m)$ tiempo y $O(n+m)$ memoria.

```
pair<ll, vector<int>> hungarian(const vector<vector<ll>> &cost) {
    int n = cost.size();
    if (n == 0) return {0, {}};
    int m = cost[0].size();
    assert(n <= m && m > 0);

    vector<ll> row_potential(n + 1), column_potential(m + 1);
    vector<int> row_for_column(m + 1), previous_column(m + 1);
    for (int row = 1; row <= n; ++row) {
        row_for_column[0] = row;
        int column0 = 0;
        vector<ll> minimum(m + 1, LINF);
        vector<char> used(m + 1, false);
        do {
            used[column0] = true;
            int row0 = row_for_column[column0], column1 = 0;
            ll delta = LINF;
            for (int column = 1; column <= m; ++column) if (!used[column]) {
                ll current = cost[row0 - 1][column - 1]
                           - row_potential[row0] - column_potential[column];
                if (current < minimum[column])
                    minimum[column] = current, previous_column[column] = column0
    ;
                if (minimum[column] < delta)
                    delta = minimum[column], column1 = column;
            }
            for (int column = 0; column <= m; ++column) {
                if (used[column]) {
                    row_potential[row_for_column[column]] += delta;
                    column_potential[column] -= delta;
                } else minimum[column] -= delta;
            }
            column0 = column1;
        } while (row_for_column[column0] != 0);

        do {
            int column1 = previous_column[column0];
            row_for_column[column0] = row_for_column[column1];
            column0 = column1;
        } while (column0 != 0);
    }

    vector<int> assignment(n, -1);
    for (int column = 1; column <= m; ++column)
        if (row_for_column[column])
            assignment[row_for_column[column] - 1] = column - 1;
    return {-column_potential[0], assignment};
}
```

Ejemplo: asignar tres personas a trabajos distintos minimizando el costo.

```
vector<vector<ll>> cost={{4,1,3},{2,0,5},{3,2,2}};
auto [minimum_cost,job]=hungarian(cost);
cout<<minimum_cost<<'\n'; // 5
for(int person=0;person<3;++person)
    cout<<person<<" -> "<<job[person]<<'\n';
```

4.26.1 Elección rápida

- Compatibilidad sí/no y maximizar parejas: **Hopcroft–Karp**.
- Matriz de costos y asignar todas las filas: **Húngaro**.
- Capacidades, cupos o cantidad de asignaciones variable con costos: **Min-Cost Max-Flow**.

4.27 Cobertura mínima bipartita: König

Cuándo usar: Elegir el menor conjunto de vértices que toca todas las aristas de un grafo bipartito. El complemento es un conjunto independiente máximo.

Complejidad: Con matching máximo ya calculado: tiempo $O(V+E)$ y memoria $O(V)$. Incluyendo Hopcroft–Karp, tiempo $O(E\sqrt{V}+V+E)$.

Usar el `HopcroftKarp` de la sección de matching. Desde los vértices izquierdos libres se recorren caminos alternantes; si Z son los alcanzados, la cobertura es $(L \setminus Z) \cup (R \cap Z)$.

```cpp
pair<vector<int>, vector<int>> minimum_vertex_cover(const HopcroftKarp &matching) {
    vector<char> left(matching.left_size), right(matching.right_size);
    queue<int> q;
    for (int u = 0; u < matching.left_size; ++u)
        if (matching.match_left[u] == -1) { left[u] = true; q.push(u); }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : matching.graph[u]) {
            if (matching.match_left[u] == v || right[v]) continue;
            right[v] = true;
            int next = matching.match_right[v];
            assert(next != -1); // Si llega a uno libre, el matching no era maximo.
            if (!left[next]) { left[next] = true; q.push(next); }
        }
    }
    vector<int> cover_left, cover_right;
    for (int u = 0; u < matching.left_size; ++u) if (!left[u]) cover_left.push_back(u);
    for (int v = 0; v < matching.right_size; ++v) if (right[v]) cover_right.push_back(v);
    return {cover_left, cover_right};
}
void konig_example() {
    HopcroftKarp matching(3, 2);
    matching.add_edge(0,0); matching.add_edge(1,0); matching.add_edge(2,1);
    int size = matching.maximum_matching();
    auto [left, right] = minimum_vertex_cover(matching);
    cout << size << ' ' << left.size() + right.size() << '\n'; // 2 2
}
```

Cuidado: Calcular el matching máximo antes y recalcularlo si se agregan aristas. Los IDs izquierdos y derechos pertenecen a conjuntos distintos. El teorema no permite sustituir cobertura por matching en grafos generales.

4.28 Bellman–Ford: affected destinations

This variant labels each destination affected by a negative cycle, instead of only returning a cycle flag.

```cpp
/**
 * Author: Simon Lindholm
 * Date: 2015-02-23
 * License: CC0
 * Source: http://en.wikipedia.org/wiki/Bellman-Ford_algorithm
 * Description: Calculates shortest paths from $s$ in a graph that might have negative edge weights.
 * Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf.
 * Assumes $V^2 \max |w_i| < \tilde{} 2^{63}$.
 * Time: O(VE)
 * Status: Tested on kattis:shortestpath3
 */
#pragma once

const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });

  int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
  rep(i,0,lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  rep(i,0,lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

4.29 Biconnected edge blocks

This variant returns edge IDs via a callback. Bridges are not emitted by the original implementation.

```cpp
/**
 * Author: Simon Lindholm
 * Date: 2017-04-17
 * License: CC0
 * Source: folklore
 * Description: Finds all biconnected components in an undirected graph, and
 *  runs a callback for the edges in each. In a biconnected component there
 *  are at least two internally disjoint paths between any two nodes (a cycle
 *  exists through them). Note that a node can be in several components. An
 *  edge which is not in a component is a bridge, i.e., not part of any cycle.
 * Usage:
```

```
 *  int eid = 0; ed.resize(N);
 *  for each edge (a,b) {
 *    ed[a].emplace_back(b, eid);
 *    ed[b].emplace_back(a, eid++); }
 *  bicomps([\&](const vi\& edgelist) {...});
 * Time: O(E + V)
 * Status: tested during MIPT ICPC Workshop 2017
 */
#pragma once

vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time, top = me;
  for (auto [y, e] : ed[at]) if (e != par) {
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = sz(st);
      int up = dfs(y, e, f);
      top = min(top, up);
      if (up == me) {
        st.push_back(e);
        f(vi(st.begin() + si, st.end()));
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top;
}

template<class F>
void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

4.30 Recovering a minimum s-t cut

The original source explains extraction from residual reachability after maximum flow; it does not define another flow algorithm.

```
/**
 * Author: Simon Lindholm
 * Date: 2015-05-13
 * Source: Wikipedia
 * Description: After running max-flow, the left side of a min-cut from $s$ to
    $t$ is given
 * by all vertices reachable from $s$, only traversing edges with positive
    residual capacity.
 * Status: works
 */
```

4.31 Global minimum cut

```
/**
 * Author: Simon Lindholm
 * Date: 2021-01-09
 * License: CC0
 * Source: https://en.wikipedia.org/wiki/Stoer%E2%80%93Wagner_algorithm
 * Description: Find a global minimum cut in an undirected graph, as represented
     by an adjacency matrix.
 * Time: O(V^3)
 * Status: Stress-tested together with GomoryHu
 */
#pragma once

pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

4.32 Gomory–Hu tree

```cpp
/**
 * Author: chilli, Takanori MAEHARA
 * Date: 2020-04-03
 * License: CC0
 * Source: https://github.com/spaghetti-source/algorithm/blob/master/graph/gomory_hu_tree.cc#L102
 * Description: Given a list of edges representing an undirected flow graph,
 * returns edges of the Gomory-Hu tree. The max flow between any pair of
 * vertices is given by minimum edge weight along the Gomory-Hu tree path.
 * Time: $O(V)$ Flow Computations
 * Status: Tested on CERC 2015 J, stress-tested
 *
 * Details: The implementation used here is not actually the original
 * Gomory-Hu, but Gusfield's simplified version: "Very simple methods for all
 * pairs network flow analysis". PushRelabel is used here, but any flow
 * implementation that supports `leftOfMinCut` also works.
 */
#pragma once

#include "PushRelabel.h"

typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    PushRelabel D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])});
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
}
```

4.33 Matching in general graphs

```cpp
/**
 * Author: Simon Lindholm
 * Date: 2016-12-09
 * License: CC0
 * Source: http://www.mimuw.edu.pl/~mucha/pub/mucha_sankowski_focs04.pdf
 * Description: Matching for general graphs.
 * Fails with probability $N / mod$.
 * Time: O(N^3)
 * Status: not very well tested
 */
#pragma once

#include "../numerical/MatrixInverse-mod.h"

vector<pii> generalMatching(int N, vector<pii>& ed) {
  vector<vector<ll>> mat(N, vector<ll>(N)), A;
  for (pii pa : ed) {
    int a = pa.first, b = pa.second, r = rand() % mod;
    mat[a][b] = r, mat[b][a] = (mod - r) % mod;
  }

  int r = matInv(A = mat), M = 2*N - r, fi, fj;
  assert(r % 2 == 0);

  if (M != N) do {
    mat.resize(M, vector<ll>(M));
    rep(i,0,N) {
      mat[i].resize(M);
      rep(j,N,M) {
        int r = rand() % mod;
        mat[i][j] = r, mat[j][i] = (mod - r) % mod;
      }
    }
  } while (matInv(A = mat) != M);

  vi has(M, 1); vector<pii> ret;
  rep(it,0,M/2) {
    rep(i,0,M) if (has[i])
      rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
        fi = i; fj = j; goto done;
    } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0;
    rep(sw,0,2) {
      ll a = modpow(A[fi][fj], mod-2);
      rep(i,0,M) if (has[i] && A[i][fj]) {
        ll b = A[i][fj] * a % mod;
        rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
      }
      swap(fi,fj);
    }
  }
  return ret;
}
```

4.34 Edge coloring

```
/**
 * Author: Simon Lindholm
 * Date: 2020-10-12
 * License: CC0
 * Source: https://en.wikipedia.org/wiki/Misra_%26_Gries_edge_coloring_algorithm
 * https://codeforces.com/blog/entry/75431 for the note about bipartite graphs.
 * Description: Given a simple, undirected graph with max degree $D$, computes a
 * $(D + 1)$-coloring of the edges such that no neighboring edges share a color.
 * ($D$-coloring is NP-hard, but can be done for bipartite graphs by repeated
    matchings of
 * max-degree nodes.)
 * Time: O(NM)
 * Status: stress-tested, tested on kattis:gamescheduling
 */
#pragma once

vi edgeColoring(int N, vector<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
  return ret;
}
```

4.35 Enumerating maximal cliques

```
/**
 * Author: Simon Lindholm
 * Date: 2018-07-18
 * License: CC0
 * Source: https://en.wikipedia.org/wiki/Bron%E2%80%93Kerbosch_algorithm
 * Description: Runs a callback for all maximal cliques in a graph (given as a
 * symmetric bitset matrix; self-edges not allowed). Callback is given a bitset
 * representing the maximal clique.
 * Time: O(3^{n/3}), much faster for sparse graphs
 * Status: stress-tested
 */
#pragma once
/// Possible optimization: on the top-most
/// recursion level, ignore 'cands', and go through nodes in order of increasing
/// degree, where degrees go down as nodes are removed.
/// (mostly irrelevant given MaximumClique)

typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

4.36 Maximum clique

```
/**
 * Author: chilli, SJTU, Janez Konc
 * Date: 2019-05-10
 * License: GPL3+
 * Source: https://en.wikipedia.org/wiki/MaxCliqueDyn_maximum_clique_algorithm,
    https://gitlab.com/janezkonc/mcqd/blob/master/mcqd.h
 * Description: Quickly finds a maximum clique of a graph (given as symmetric
    bitset
 * matrix; self-edges not allowed). Can be used to find a maximum independent
 * set by finding a clique of the complement graph.
```

```
 * Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs
 * faster for sparse graphs.
 * Status: stress-tested
 */
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
};
```

4.37 Maximum independent set

The original source explains the reduction to MaximumClique; it does not define a separate function.

```
/**
 * Author: chilli
 * Date: 2019-05-17
 * Source: Wikipedia
 * Description: To obtain a maximum independent set of a graph, find a max
 * clique of the complement. If the graph is bipartite, see MinimumVertexCover.
 */
```

4.38 Dynamic forests: link-cut tree

```
/**
 * Author: Simon Lindholm
 * Date: 2016-07-25
 * Source: https://github.com/ngthanhtrung23/ACM_Notebook_new/blob/master/
    DataStructure/LinkCutTree.h
 * Description: Represents a forest of unrooted trees. You can add and remove
 * edges (as long as the result is still a forest), and check whether
 * two nodes are in the same tree.
 * Time: All operations take amortized O(\log N).
 * Status: Stress-tested a bit for N <= 20
 */
#pragma once

struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
```

```
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() { /// Splay this up to the root. Always finishes without flip set
    .
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() { /// Return the min element of the subtree rooted at this,
    splayed to the top.
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) { /// Move u to root of represented tree.
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }
  Node* access(Node* u) { /// Move u to root aux tree. Return the root of the
    root aux tree.
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

4.39 Directed minimum spanning arborescence

```
/**
 * Author: chilli, Takanori MAEHARA, Benq, Simon Lindholm
 * Date: 2019-05-10
 * License: CC0
 * Source: https://github.com/spaghetti-source/algorithm/blob/master/graph/
    arborescence.cc
 * and https://github.com/bqi343/USACO/blob/42
    d177dfb9d6ce350389583cfa71484eb8ae614c/Implementations/content/graphs%20(12)
    /Advanced/DirectedMST.h for the reconstruction
 * Description: Finds a minimum spanning
 * tree/arborescence of a directed graph, given a root node. If no MST exists,
    returns -1.
```

```
 * Time: O(E \log V)
 * Status: Stress-tested, also tested on NWERC 2018 fastestspeedrun
 */
#pragma once

#include "../data-structures/UnionFindRollback.h"

struct Edge { int a, b; ll w; };
struct Node { /// lazy skew heap node
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) { /// found cycle, contract
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

5. Strings

5.1 Rolling hash

Cuándo usar: Comparar substrings, detectar repeticiones o hacer binary search sobre longitudes aceptando una probabilidad pequeña de colisión.

Hash doble para comparar substrings en $O(1)$ tras preprocesar. Los rangos son semiabiertos $[l, r)$. Una igualdad de hashes sigue siendo probabilística.

```
struct RollingHash {
    static const int MOD1 = 1000000007;
    static const int MOD2 = 1000000009;
    static const int BASE = 911382323;
    vector<int> power1, power2, hash1, hash2;

    explicit RollingHash(const string &s) {
        int n = s.size();
        power1.assign(n + 1, 1); power2.assign(n + 1, 1);
        hash1.assign(n + 1, 0);  hash2.assign(n + 1, 0);
        for (int i = 0; i < n; ++i) {
            int value = (unsigned char)s[i] + 1;
            power1[i + 1] = (ll)power1[i] * BASE % MOD1;
            power2[i + 1] = (ll)power2[i] * BASE % MOD2;
            hash1[i + 1] = ((ll)hash1[i] * BASE + value) % MOD1;
            hash2[i + 1] = ((ll)hash2[i] * BASE + value) % MOD2;
        }
    }

    pair<int,int> get(int l, int r) const {
        ll first = hash1[r] - (ll)hash1[l] * power1[r - l] % MOD1;
        ll second = hash2[r] - (ll)hash2[l] * power2[r - l] % MOD2;
        if (first < 0) first += MOD1;
        if (second < 0) second += MOD2;
        return {(int)first, (int)second};
    }
};
```

Complejidad: construcción $O(n)$; consulta $O(1)$; memoria $O(n)$.

Comparar dos substrings de la misma longitud:

```
string s="banana";
RollingHash hash(s);
bool equal=hash.get(1,4)==hash.get(3,6);
cout<<equal<<'\n'; // 1: "ana" == "ana"
```

Buscar apariciones de un patrón:

```
string text="bananana",pattern="ana";
RollingHash text_hash(text),pattern_hash(pattern);
auto target=pattern_hash.get(0,pattern.size());
for(int i=0;i+(int)pattern.size()<=(int)text.size();++i)
    if(text_hash.get(i,i+pattern.size())==target)
        cout<<i<<' '; // 1 3 5
```

`get(l,r)` devuelve el hash del substring semiabierto $[l, r)$. Sólo deben compararse substrings de igual longitud. Aunque se use hash doble, existe una probabilidad muy pequeña de colisión.

5.2 Prefix function, KMP

Cuándo usar: Buscar exactamente un patrón en un texto o analizar bordes y períodos de una cadena sin hashing.

```
vector<int> prefix_function(const string &s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_occurrences(const string &text, const string &pattern) {
    if (pattern.empty()) {
        vector<int> all(text.size() + 1);
        iota(all.begin(), all.end(), 0);
        return all;
    }
    vector<int> pi = prefix_function(pattern), answer;
    int matched = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (matched > 0 && text[i] != pattern[matched])
            matched = pi[matched - 1];
        if (text[i] == pattern[matched]) ++matched;
        if (matched == (int)pattern.size()) {
            answer.push_back(i - matched + 1);
            matched = pi[matched - 1];
        }
    }
    return answer;
}
```

Complejidad: $O(n + m)$ tiempo; $O(m)$ memoria para buscar.

5.2.1 Ejemplo de KMP

`pi[i]` es la longitud del mayor prefijo propio de `s[0..i]` que también es sufijo. KMP usa esta información para no volver al inicio del patrón después de una diferencia.

```
string text="bananana",pattern="ana";
vector<int> positions=kmp_occurrences(text,pattern);
for(int position:positions) cout<<position<<' '; // 1 3 5
```

También permite encontrar el mayor borde de una cadena:

```
string s="abcab";
vector<int> pi=prefix_function(s);
int longest_border=pi.back(); // 2: "ab"
```

Un borde es un prefijo propio que también aparece como sufijo.

5.3 Z-function

Cuándo usar: Conocer para cada posición cuánto coincide el sufijo que empieza allí con el prefijo; ùtil para patrones, bordes y periodicidad.

`z[i]` es la longitud del prefijo de s que coincide desde i.

```
vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
```

Complejidad: $O(n)$ tiempo y memoria.

5.3.1 Ejemplo de Z-function

`z[i]` indica cuántos caracteres desde `s[i]` coinciden con el prefijo completo de `s`. Por ejemplo, para ‘‘aaaaa’’:

$$z = [0, 4, 3, 2, 1].$$

Para buscar un patrón, concatenar patrón, un separador ausente y texto:

```
string text="bananana",pattern="ana";
string combined=pattern+"#"+text;
vector<int> z=z_function(combined);
int offset=pattern.size()+1;
for(int i=offset;i<(int)combined.size();++i)
    if(z[i]>=(int)pattern.size())
        cout<<i-offset<<' '; // 1 3 5
```

Cuidado: El separador, aquí `#`, no debe aparecer ni en el patrón ni en el texto.

- **KMP/prefix function:** buscar exactamente un patrón y estudiar bordes o períodos.
- **Z-function:** conocer cuánto coincide cada sufijo con el prefijo; suele resultar más directa para matching mediante concatenación.
- Ambos son deterministas y cuestan $O(n+m)$; no tienen colisiones como rolling hash.

5.4 Trie

Cuándo usar: Mantener un diccionario de cadenas y consultar palabras exactas o cuántas comienzan con cierto prefijo en $O(|s|)$. Útil para autocompletado, prefijos compartidos y XOR máximo mediante un Trie binario. Conviene con alfabetos pequeños; puede consumir mucha memoria si hay pocas coincidencias entre prefijos.

Versión para letras minúsculas inglesas.

```
struct Trie {
    struct Node {
        array<int, 26> next;
        int terminal = 0, passing = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> nodes{Node()};

    void insert(const string &word) {
        int v = 0;
        ++nodes[v].passing;
        for (char c : word) {
            int letter = c - 'a';
            assert(0 <= letter && letter < 26);
            if (nodes[v].next[letter] == -1) {
                nodes[v].next[letter] = nodes.size();
                nodes.emplace_back();
            }
            v = nodes[v].next[letter];
            ++nodes[v].passing;
        }
        ++nodes[v].terminal;
    }

    int count_word(const string &word) const {
        int v = 0;
        for (char c : word) {
            int letter = c - 'a';
            if (letter < 0 || letter >= 26 || nodes[v].next[letter] == -1)
                return 0;
            v = nodes[v].next[letter];
```

```
        }
        return nodes[v].terminal;
    }

    int count_prefix(const string &prefix) const {
        int v = 0;
        for (char c : prefix) {
            int letter = c - 'a';
            if (letter < 0 || letter >= 26 || nodes[v].next[letter] == -1)
                return 0;
            v = nodes[v].next[letter];
        }
        return nodes[v].passing;
    }
};
```

Complejidad: $O(|s|)$ por inserción o búsqueda; memoria proporcional a la suma de longitudes, hasta $O(\Sigma|s|)$.

Cada nodo representa un prefijo. `terminal` cuenta cuántas veces se insertó exactamente la palabra que termina allí; `passing` cuenta cuántas palabras, incluyendo repetidas, pasan por ese prefijo.

Ejemplo de uso:

```
Trie trie;
trie.insert("car");
trie.insert("cat");
trie.insert("car");
trie.insert("dog");
cout<<trie.count_word("car")<<'\n';    // 2
cout<<trie.count_word("ca")<<'\n';     // 0
cout<<trie.count_prefix("ca")<<'\n';   // 3
cout<<trie.count_prefix("do")<<'\n';   // 1
cout<<trie.count_prefix("x")<<'\n';    // 0
```

`count_word(s)` exige que la palabra completa esté insertada; `count_prefix(s)` cuenta todas las palabras que comienzan con `s`.

5.5 Manacher

Cuándo usar: Calcular en tiempo lineal el radio del palíndromo máximo centrado en cada posición, para responder existencia/longitud de substrings palindrómicos.

`odd[i]` es el radio del palíndromo impar centrado en i, incluyendo el centro. `even[i]` es el radio del palíndromo par centrado entre $i-1$ e i.

Complejidad: $O(n)$ tiempo y memoria.

```
pair<vector<int>, vector<int>> manacher(const string &s) {
    int n = s.size();
    vector<int> odd(n), even(n);

    for (int i = 0, left = 0, right = -1; i < n; ++i) {
        int radius = i > right ? 1 : min(odd[left + right - i], right - i + 1);
        while (i - radius >= 0 && i + radius < n &&
               s[i - radius] == s[i + radius]) ++radius;
        odd[i] = radius--;
        if (i + radius > right) left = i - radius, right = i + radius;
    }

    for (int i = 0, left = 0, right = -1; i < n; ++i) {
        int radius = i > right ? 0
                   : min(even[left + right - i + 1], right - i + 1);
        while (i - radius - 1 >= 0 && i + radius < n &&
               s[i - radius - 1] == s[i + radius]) ++radius;
        even[i] = radius--;
        if (i + radius > right) left = i - radius - 1, right = i + radius;
    }
    return {odd, even};
}
```

La longitud máxima es $\max(2 \cdot \texttt{odd[i]} - 1, 2 \cdot \texttt{even[i]})$.

5.6 Aho–Corasick

Cuándo usar: Buscar simultáneamente muchas palabras en uno o varios textos, reportando coincidencias mientras se recorre cada texto una sola vez.

Busca simultáneamente muchos patrones no vacíos. Esta versión usa letras minúsculas inglesas y devuelve el ID de cada patrón junto con su posición final. Cada nodo guarda sólo sus propios patrones; un enlace de salida apunta al sufijo terminal más cercano, evitando copiar las coincidencias heredadas.

Complejidad: Con $L = \sum |p|$, alfabeto de tamaño $\Sigma = 26$ y z coincidencias: construcción $O(L\Sigma)$ con transiciones completas; búsqueda $O(|texto| + z)$. Autómata: memoria $O(L\Sigma)$; devolver las coincidencias agrega $O(z)$ memoria de salida.

```
struct AhoCorasick {
    static const int ALPHABET = 26;
    struct Node {
        array<int, ALPHABET> next;
        int suffix_link = 0;
        int output_link = -1; // Sufijo terminal mas cercano.
        vector<int> output;
        Node() { next.fill(-1); }
    };
    vector<Node> nodes{Node()};

    void insert(const string &pattern, int pattern_id) {
        assert(!pattern.empty());
```

```
        int v = 0;
        for (char character : pattern) {
            int letter = character - 'a';
            assert(0 <= letter && letter < ALPHABET);
            if (nodes[v].next[letter] == -1) {
                nodes[v].next[letter] = nodes.size();
                nodes.emplace_back();
            }
            v = nodes[v].next[letter];
        }
        nodes[v].output.push_back(pattern_id);
    }

    void build() {
        queue<int> q;
        for (int letter = 0; letter < ALPHABET; ++letter) {
            int child = nodes[0].next[letter];
            if (child == -1) nodes[0].next[letter] = 0;
            else nodes[child].suffix_link = 0, q.push(child);
        }

        while (!q.empty()) {
            int v = q.front(); q.pop();
            int link = nodes[v].suffix_link;
            nodes[v].output_link = !nodes[link].output.empty()
                                   ? link : nodes[link].output_link;
            for (int letter = 0; letter < ALPHABET; ++letter) {
                int child = nodes[v].next[letter];
                if (child == -1) {
                    nodes[v].next[letter] = nodes[link].next[letter];
                } else {
                    nodes[child].suffix_link = nodes[link].next[letter];
                    q.push(child);
                }
            }
        }
    }

    vector<pair<int,int>> find_all(const string &text) const {
        vector<pair<int,int>> matches; // (end_position, pattern_id)
        int state = 0;
        for (int position = 0; position < (int)text.size(); ++position) {
            int letter = text[position] - 'a';
            assert(0 <= letter && letter < ALPHABET);
            state = nodes[state].next[letter];
            for (int v = state; v != -1; v = nodes[v].output_link)
                for (int pattern_id : nodes[v].output)
                    matches.push_back({position, pattern_id});
        }
        return matches;
    }
};
```

Llamar `build()` exactamente una vez después de insertar todos los patrones. Para contar sin enumerar coincidencias, acumular visitas por el árbol de suffix links.

```
vector<string> patterns={"he","she","his","hers"};
AhoCorasick aho;
for(int id=0;id<(int)patterns.size();++id)
    aho.insert(patterns[id],id);
aho.build(); // Una vez, despues de insertar todos.

string text="ahishers";
for(auto [end,id]:aho.find_all(text)) {
    int start=end-(int)patterns[id].size()+1;
    cout<<patterns[id]<<" ["<<start<<","<<end+1<<")\n";
}
// his [1,4), she [3,6), he [4,6), hers [4,8)
// find_all devuelve (posicion final,id); admite coincidencias superpuestas.
```

5.7 Suffix Array y LCP

Cuándo usar: Ordenar todos los sufijos para comparar substrings, buscar patrones, contar substrings distintos o conocer prefijos comunes sin usar hashing.

`sa[i]` es la posición donde comienza el i-ésimo sufijo en orden lexicográfico. `lcp[i]` es la longitud del prefijo común entre los sufijos `sa[i]` y `sa[i+1]`. Para una cadena de longitud n, SA tiene n posiciones y LCP tiene $\max(0, n-1)$ valores. Ambas funciones aceptan la cadena vacía.

Construcción basada en el doubling con counting sort de KACTL (Luo Suiqian y chilli): en cada ronda se ordenan pares de clases que representan prefijos de longitud k, obteniendo clases para longitud $2k$. Ordenar por clases enteras en tiempo lineal por ronda evita el `sort` comparativo.

Internamente se agrega un centinela menor que cualquier carácter y se elimina su posición antes de devolver SA. Se codifica cada byte como `(unsigned char)c+1`, reservando cero para el centinela: no se modifica la entrada y también se admiten bytes NUL y valores mayores que 127. `lcp_array` conserva la convención del handbook y usa Kasai; no devuelve el LCP adicional asociado al centinela de la interfaz original de KACTL.

Complejidad: Suffix Array $O(n \log n)$; LCP $O(n)$; memoria $O(n)$.

```
vector<int> suffix_array(const string &s) {
    int n=(int)s.size()+1, classes=257; // Incluye el centinela.
    vector<int> order(n),rank(n),work(n),count(max(n,classes));
    iota(order.begin(),order.end(),0);
    for(int i=0;i<n-1;++i) rank[i]=(unsigned char)s[i]+1;
```

```
    for(int length=0,groups=0;groups<n;
        length=length?2*length:1,classes=groups) {
        int at=length;
        iota(work.begin(),work.begin()+length,n-length);
        for(int position:order)
            if(position>=length) work[at++]=position-length;
        fill(count.begin(),count.begin()+classes,0);
        for(int value:rank) ++count[value];
        for(int i=1;i<classes;++i) count[i]+=count[i-1];
        for(int i=n;i--;)
            order[--count[rank[work[i]]]]=work[i];
        rank.swap(work); // work conserva las clases anteriores.
        groups=1; rank[order[0]]=0;
        for(int i=1;i<n;++i) {
            int a=order[i-1],b=order[i];
            bool same=work[a]==work[b] &&
                (a+length<n?work[a+length]:-1)==
                (b+length<n?work[b+length]:-1);
            rank[b]=same?groups-1:groups++;
        }
    }
    order.erase(order.begin()); // Quitar el sufijo vacío, posición s.size().
    return order;
}

vector<int> lcp_array(const string &s,const vector<int> &sa) {
    int n=s.size();
    assert((int)sa.size()==n);
    if(n<=1) return {};
    vector<int> rank(n),lcp(n-1);
    for(int i=0;i<n;++i) rank[sa[i]]=i;
    for(int position=0,length=0;position<n;++position) {
        int index=rank[position];
        if(index==n-1) {
            length=0;
            continue;
        }
        int next=sa[index+1];
        while(position+length<n&&next+length<n&&
              s[position+length]==s[next+length]) ++length;
        lcp[index]=length;
        if(length) --length;
    }
    return lcp;
}
```

Ejemplo de uso:

```
string s="banana";
vector<int> sa=suffix_array(s);  // 5 3 1 0 4 2
vector<int> lcp=lcp_array(s,sa); // 1 3 0 0 2
// Cantidad de substrings distintos:
ll n=s.size();
ll total=n*(n+1)/2;
ll repeated=accumulate(lcp.begin(),lcp.end(),0LL);
ll distinct=total-repeated;
cout<<distinct<<'\n'; // 15
// Cada lcp[i] cuenta prefijos compartidos por dos sufijos consecutivos.
```

La cantidad de substrings distintos es

$$\frac{n(n+1)}{2} - \sum_i lcp[i].$$

Para consultar repetidamente el LCP entre dos sufijos cualesquiera, guardar la posición de cada sufijo en `sa` y construir un RMQ sobre el arreglo `lcp`.

5.8 Minimum string rotation

```
/**
 * Author: Stjepan Glavina
 * License: Unlicense
 * Source: https://github.com/stjepang/snippets/blob/master/min_rotation.cpp
 * Description: Finds the lexicographically smallest rotation of a string.
 * Time: O(N)
 * Usage:
 *  rotate(v.begin(), v.begin()+minRotation(v), v.end());
 * Status: Stress-tested
 */
#pragma once

int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

5.9 Online suffix tree and longest common substring

As in the original comments, LCS requires ALPHA=28 for the two separator symbols; the default tree uses ALPHA=26.

```
/**
```

```
 * Author: Unknown
 * Date: 2017-05-15
 * Source: https://e-maxx.ru/algo/ukkonen
 * Description: Ukkonen's algorithm for online suffix tree construction.
 *  Each node contains indices [l, r) into the string, and a list of child nodes
    .
 *  Suffixes are given by traversals of this tree, joining [l, r) substrings.
 *  The root is 0 (has l = -1, r = 0), non-existent children are -1.
 *  To get a complete tree, append a dummy symbol -- otherwise it may contain
 *  an incomplete path (still useful for substring matching, though).
 * Time: $O(26N)$
 * Status: stress-tested a bit
 */
#pragma once

struct SuffixTree {
  enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
  int toi(char c) { return c - 'a'; }
  string a; // v = cur node, q = cur position
  int t[N][ALPHA],l[N],r[N],p[N],s[N],v=0,q=0,m=2;

  void ukkadd(int i, int c) { suff:
    if (r[v]<=q) {
      if (t[v][c]==-1) { t[v][c]=m;  l[m]=i;
        p[m++]=v; v=s[v]; q=r[v];  goto suff; }
      v=t[v][c]; q=l[v];
    }
    if (q==-1 || c==toi(a[q])) q++; else {
      l[m+1]=i;  p[m+1]=m;  l[m]=l[v];  r[m]=q;
      p[m]=p[v];  t[m][c]=m+1;  t[m][toi(a[q])]=v;
      l[v]=q;  p[v]=m;  t[p[m]][toi(a[l[m]])]=m;
      v=s[p[m]];  q=l[m];
      while (q<r[m]) { v=t[v][toi(a[q])];  q+=r[v]-l[v]; }
      if (q==r[m])  s[m]=v;  else s[m]=m+2;
      q=r[v]-(q-r[m]);  m+=2;  goto suff;
    }
  }

  SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a));
    memset(s, 0, sizeof s);
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
  }

  // example: find longest common substring (uses ALPHA = 28)
  pii best;
  int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
      mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
      best = max(best, {len, r[node] - len});
    return mask;
  }
  static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
  }
};
```

6. Matemática

6.1 Teoremas e identidades para reconocer soluciones

Cuándo usar: Antes de programar, cuando el problema pide contar, demostrar una cota, trabajar módulo primo o transformar una suma aparentemente cuadrática.

Complejidad: Las fórmulas son $O(1)$ una vez conocidos sus datos; inclusión–exclusión cuesta normalmente $O(2^k)$ y Burnside recorre las simetrías. Precomputaciones y factorizaciones conservan sus propios costos.

6.1.1 Conteo y cotas

- **Palomar:** al repartir m objetos entre n cajas, alguna recibe al menos $\lceil m/n \rceil$. Se usa para demostrar que una repetición o colisión es inevitable sin identificar cuál. Por ejemplo, entre 13 personas al menos dos cumplen años en el mismo mes.
- **Handshake:**

$$\sum_v deg(v) = 2|E|.$$

  Cada arista aporta uno al grado de cada extremo, por eso se cuenta dos veces. Como la suma es par, la cantidad de vértices de grado impar también es par. Es útil para validar secuencias de grados y recorridos eulerianos.
- **Suma triangular:**

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}.$$

  Aparece cuando la iteración i cuesta i, se eligen pares o cada paso consume una unidad más. Si la suma no supera n, entonces $k \approx \sqrt{2n}$ y por tanto $k = O(\sqrt{n})$.
- **Serie armónica entera:**

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = O(n \log n).$$

  Cuenta, por ejemplo, todos los múltiplos procesados por bucles de la forma `for(j=i;j<=n;j+=i)`. Además, $\lfloor n/i \rfloor$ toma sólo $O(\sqrt{n})$ valores distintos; para un cociente q, el último índice con el mismo valor es $\lfloor n/q \rfloor$, permitiendo agrupar rangos.
- **Inclusión–exclusión:** suma los casos que cumplen cada propiedad y corrige los contados varias veces:

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

$$|A \cup B \cup C| = \sum |A_i| - \sum |A_i \cap A_j| + |A \cap B \cap C|.$$

  Para k propiedades se recorren normalmente las 2^k intersecciones. Se usa para contar objetos que cumplen al menos una restricción o, tomando el complemento, los que no cumplen ninguna.

6.1.2 Combinatoria

- **Reglas de suma y producto:** si los casos son excluyentes, sus cantidades se suman. Si una elección de a formas va seguida de otra independiente de b formas, se multiplican: ab. Por ejemplo, 3 camisas y 2 pantalones producen $3 \cdot 2 = 6$ conjuntos.
- **Permutaciones:** ordenar n objetos distintos produce $n!$ órdenes. Elegir y ordenar k de ellos produce

$$P(n,k) = n(n-1)\cdots(n-k+1) = \frac{n!}{(n-k)!}.$$

  Se usa cuando el orden importa; elegir presidente y vicepresidente entre n personas da $P(n,2)$, no $\binom{n}{2}$.
- **Combinaciones:** elegir k objetos entre n sin importar el orden:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \binom{n}{n-k}.$$

  Si un multiconjunto tiene cantidades $c_1, \ldots, c_r$ con $\sum c_i = n$, sus órdenes distintos son

$$\frac{n!}{c_1!c_2!\cdots c_r!}.$$

  Por ejemplo, `AAB` tiene $3!/2! = 3$ permutaciones distintas.
- **Subconjuntos:** cada uno de los n elementos puede estar o no, por lo que existen 2^n subconjuntos. Los de tamaño k son $\binom{n}{k}$ y

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

- **Identidad de Pascal:**

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

  Al fijar un elemento, una selección o lo contiene o no lo contiene. Permite calcular combinaciones mediante DP, incluso cuando no existe división modular.
- **Hockey-stick:**

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}.$$

  Convierte una suma de combinaciones consecutivas en una sola. Combinatoriamente, clasifica cada selección de $r+1$ elementos según cuál es su elemento mayor.
- **Binomio de Newton:**

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

  El coeficiente $\binom{n}{k}$ cuenta en cuáles k de los n factores se elige x. Sustituir valores produce identidades; con $x = y = 1$ se obtiene $\sum_k \binom{n}{k} = 2^n$.
- **Stars and bars:** la cantidad de soluciones enteras no negativas de

$$x_1 + \cdots + x_k = n$$

  es $\binom{n+k-1}{k-1}$: se ordenan n estrellas y $k-1$ separadores. Si todas las variables deben ser positivas, entregar una unidad a cada una deja $n-k$ por repartir y resulta $\binom{n-1}{k-1}$. Sirve para distribuir objetos idénticos entre cajas distinguibles.

- **Vandermonde:**
$$\sum_i \binom{r}{i}\binom{s}{n-i} = \binom{r+s}{n}.$$
Elegir n elementos de dos grupos de tamaños r y s puede hacerse directamente o fijando cuántos i se toman del primero.
- **Catalan:**
$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}.$$
Cuenta estructuras que nunca cruzan una frontera: paréntesis balanceados con n pares, caminos que no bajan de cero, triangulaciones de un polígono de $n+2$ lados y varios árboles ordenados. Los primeros valores son $1, 1, 2, 5, 14, 42$.
- **Desarreglos:** permutaciones donde ningún elemento queda en su posición original:
$$D_n = (n-1)(D_{n-1} + D_{n-2}), \qquad D_0 = 1,\ D_1 = 0.$$
Al decidir dónde va el primer elemento, el caso restante se reduce a un desarreglo de $n-1$ o de $n-2$. Aparece en asignaciones donde nadie puede recibir su objeto original.
- **Burnside:** para contar configuraciones iguales bajo un grupo de rotaciones o reflexiones,
$$\#\text{clases} = \frac{1}{|G|}\sum_{g\in G} |\operatorname{Fix}(g)|.$$
Para cada simetría se cuentan las configuraciones que no cambian y luego se promedia. Es útil para collares, pulseras, coloraciones y tableros considerados iguales bajo simetrías.

6.1.3 Teoría de números

Si $n = \prod p_i^{a_i}$, cada divisor elige para p_i un exponente entre 0 y a_i. De esta observación se obtienen:

$$\tau(n) = \prod_i (a_i+1), \qquad \sigma(n) = \prod_i \frac{p_i^{a_i+1}-1}{p_i-1}, \qquad \varphi(n) = n\prod_{p\mid n}\left(1-\frac{1}{p}\right).$$

$\tau(n)$ cuenta divisores, $\sigma(n)$ suma sus valores y $\varphi(n)$ cuenta enteros de 1 a n coprimos con n. Por ejemplo, $12 = 2^2\cdot 3$ da $\tau(12) = 6$, $\sigma(12) = 28$ y $\varphi(12) = 4$.

- **Bézout:** existen enteros x, y tales que
$$ax + by = \gcd(a,b).$$
Por tanto, $ax + by = c$ tiene soluciones enteras si y sólo si $\gcd(a,b) \mid c$. Euclides extendido encuentra x, y y permite construir inversos modulares cuando el gcd es 1.
- **Pequeño teorema de Fermat:** si p es primo y $p \nmid a$,
$$a^{p-1} \equiv 1 \pmod p.$$
Entonces $a^{-1} \equiv a^{p-2} \pmod p$ y los exponentes pueden reducirse módulo $p-1$ cuando la base no es divisible por p.
- **Teorema de Euler:** si $\gcd(a,m) = 1$, entonces
$$a^{\varphi(m)} \equiv 1 \pmod m.$$
Generaliza Fermat a módulos no primos y permite reducir exponentes módulo $\varphi(m)$ bajo la condición de coprimalidad.
- **Wilson:** $p > 1$ es primo si y sólo si
$$(p-1)! \equiv -1 \pmod p.$$
Es útil en demostraciones y algunos cálculos cercanos a p, pero calcular el factorial cuesta $O(p)$ y no es una prueba de primalidad eficiente para valores grandes.

6.1.4 Grafos y geometría

- **Caracterización de árboles:** un árbol conectado con n vértices tiene exactamente $n-1$ aristas. En un árbol existe un único camino entre cada par; agregar una arista crea exactamente un ciclo y quitar cualquiera lo desconecta. Un grafo conectado con $n-1$ aristas es un árbol.
- **Caminos eulerianos:** ignorando vértices aislados, las aristas deben pertenecer a una sola componente conexa. Existe circuito euleriano si todos los grados son pares; existe camino abierto que usa cada arista una vez si exactamente dos grados son impares, que serán sus extremos. Con cero impares también existe camino, pero comienza y termina en el mismo vértice.
- **Fórmula de Euler planar:** para un grafo planar conexo,
$$V - E + F = 2,$$
donde F incluye la cara exterior. Para un grafo planar simple con $V \geq 3$ implica $E \leq 3V - 6$, una cota útil para demostrar que un grafo no es planar o que un algoritmo sobre todas sus aristas es lineal en V.
- **Teorema de Pick:** para un polígono simple cuyos vértices son puntos enteros,
$$A = I + \frac{B}{2} - 1,$$
donde I es la cantidad de puntos enteros interiores y B los de la frontera. Los puntos de frontera de un lado (x_1, y_1)–(x_2, y_2) se cuentan mediante $\gcd(|x_2 - x_1|, |y_2 - y_1|)$ al sumar todos los lados.

6.2 GCD, LCM y Euclides extendido

Cuándo usar: Divisibilidad, simplificar fracciones, resolver $ax + by = gcd(a,b)$ o construir inversos cuando los números son coprimos.

Complejidad: $O(\log \min(|a|, |b|))$ tiempo; memoria logarítmica por el stack en la versión recursiva.

```
optional<ll> checked_lcm(ll a, ll b) {
```

```
    if (a == 0 || b == 0) return 0;
    if (a == numeric_limits<ll>::min() || b == numeric_limits<ll>::min())
        return nullopt;
    ll divisor = gcd(a, b);
    i128 value = (i128)(a / divisor) * b;
    if (value < 0) value = -value;
    if (value > numeric_limits<ll>::max()) return nullopt;
    return (ll)value;
}

ll extended_gcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extended_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

6.3 Aritmética modular

Cuándo usar: Potencias, divisiones e inversos bajo módulo; el inverso por Fermat requiere módulo primo y denominador no divisible por él.

```
ll mod_normalize(ll value, ll mod) {
    assert(mod > 0);
    value %= mod;
    if (value < 0) value += mod;
    return value;
}

ll mod_add(ll a, ll b, ll mod) {
    return ((i128)mod_normalize(a, mod) + mod_normalize(b, mod)) % mod;
}

ll mod_subtract(ll a, ll b, ll mod) {
    i128 result = (i128)mod_normalize(a, mod) - mod_normalize(b, mod);
    if (result < 0) result += mod;
    return (ll)result;
}

ll mod_multiply(ll a, ll b, ll mod) {
    return (i128)mod_normalize(a, mod) * mod_normalize(b, mod) % mod;
}

ll mod_power(ll base, unsigned long long exponent, ll mod) {
    ll result = 1 % mod;
    base = mod_normalize(base, mod);
    while (exponent) {
        if (exponent & 1ULL) result = mod_multiply(result, base, mod);
        base = mod_multiply(base, base, mod);
        exponent >>= 1;
    }
    return result;
}

optional<ll> mod_inverse(ll a, ll mod) {
    assert(mod > 0);
    ll x, y;
    ll g = extended_gcd(mod_normalize(a, mod), mod, x, y);
    if (g != 1) return nullopt;
    return mod_normalize(x, mod);
}

optional<ll> mod_divide(ll a, ll b, ll mod) {
    auto inverse = mod_inverse(b, mod);
    if (!inverse) return nullopt;
    return mod_multiply(a, *inverse, mod);
}
```

Complejidad: $O(\log e)$ para potencia; $O(\log m)$ para inverso; memoria $O(1)$ salvo el stack de Euclides.

6.4 Teorema chino del resto para dos congruencias

Te permite saber **qué números cumplen las dos condiciones al mismo tiempo** y **cada cuánto se repiten**.

Cuándo usar: Combinar condiciones $x \equiv a \pmod{m}$; esta versión también detecta si dos congruencias con módulos no coprimos son incompatibles.

Complejidad: $O(\log \min(m_1, m_2))$ tiempo por combinación; memoria logarítmica por Euclides extendido.

Admite módulos no coprimos. Devuelve la solución normalizada y el módulo mínimo, o `nullopt` si no hay solución o el módulo no entra en `long long`.

```
optional<pair<ll,ll>> crt_pair(ll a1, ll m1, ll a2, ll m2) {
    assert(m1 > 0 && m2 > 0);
    a1 = mod_normalize(a1, m1);
    a2 = mod_normalize(a2, m2);
    ll x, y;
    ll g = extended_gcd(m1, m2, x, y);
    ll difference = a2 - a1;
    if (difference % g != 0) return nullopt;

    i128 modulus = (i128)(m1 / g) * m2;
    if (modulus > numeric_limits<ll>::max()) return nullopt;
```

```
    ll reduced = m2 / g;
    i128 t = (i128)(difference / g) * x % reduced;
    if (t < 0) t += reduced;
    i128 answer = ((i128)a1 + (i128)m1 * t) % modulus;
    if (answer < 0) answer += modulus;
    return pair<ll,ll>{(ll)answer, (ll)modulus};
}
```

6.5 Combinatoria módulo primo

Cuándo usar: Muchas consultas de combinaciones/permutaciones módulo primo con $n < mod$, precomputando factoriales e inversos. Para n enorme y primo pequeño, Lucas. Precalcula factoriales para responder $\binom{n}{k}$ cuando el módulo es primo y $n < mod$.

```
struct Combinations {
    ll mod;
    vector<ll> factorial, inverse_factorial;

    Combinations(int maximum_n, ll prime_mod) : mod(prime_mod),
        factorial(max(0, maximum_n) + 1, 1),
        inverse_factorial(max(0, maximum_n) + 1, 1) {
        assert(maximum_n >= 0 && maximum_n < prime_mod);
        for (int i = 1; i <= maximum_n; ++i)
            factorial[i] = factorial[i - 1] * i % mod;
        inverse_factorial[maximum_n] = mod_power(
            factorial[maximum_n], mod - 2, mod);
        for (int i = maximum_n; i > 0; --i)
            inverse_factorial[i - 1] = inverse_factorial[i] * i % mod;
    }

    ll choose(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return factorial[n] * inverse_factorial[k] % mod
             * inverse_factorial[n - k] % mod;
    }
};
```

Complejidad: preproceso $O(N + \log mod)$; consulta $O(1)$; memoria $O(N)$.

6.6 Teorema de Lucas

Cuándo usar: Calcular $\binom{n}{k}$ mod p con p primo pequeño y n, k demasiado grandes para precomputar factoriales hasta n.

Usar para calcular $\binom{n}{k}$ mod p cuando p es primo pequeño pero n y k pueden ser enormes. Se descomponen ambos números en base p.

Complejidad: preproceso $O(p)$; consulta $O(\log_p n)$; memoria $O(p)$.

```
struct LucasTheorem {
    ll prime;
    vector<ll> factorial, inverse_factorial;

    explicit LucasTheorem(ll modulus)
        : prime(modulus), factorial(prime), inverse_factorial(prime) {
        factorial[0] = 1;
        for (ll value = 1; value < prime; ++value)
            factorial[value] = factorial[value - 1] * value % prime;
        inverse_factorial[prime - 1] = mod_pow(factorial[prime - 1], prime - 2);
        for (ll value = prime - 1; value > 0; --value)
            inverse_factorial[value - 1] = inverse_factorial[value] * value %
    prime;
    }

    ll mod_pow(ll base, ll exponent) const {
        ll result = 1;
        while (exponent) {
            if (exponent & 1) result = result * base % prime;
            base = base * base % prime;
            exponent >>= 1;
        }
        return result;
    }

    ll small_combination(ll n, ll k) const {
        if (k < 0 || k > n) return 0;
        return factorial[n] * inverse_factorial[k] % prime
             * inverse_factorial[n - k] % prime;
    }

    ll combination(ll n, ll k) const {
        ll answer = 1;
        while (n || k) {
            answer = answer * small_combination(n % prime, k % prime) % prime;
            n /= prime; k /= prime;
        }
        return answer;
    }
};
```

Cuidado: Requiere p primo y guardar $O(p)$ valores; no usar si el módulo es grande.

6.7 Criba lineal y menor factor primo

Cuándo usar: Muchas consultas de primalidad/factorización para valores acotados por N; precomputar una vez y factorizar luego por el menor primo.

```cpp
struct LinearSieve {
    vector<int> primes, smallest_prime;

    explicit LinearSieve(int n) : smallest_prime(max(0, n) + 1, 0) {
        for (int value = 2; value <= n; ++value) {
            if (smallest_prime[value] == 0) {
                smallest_prime[value] = value;
                primes.push_back(value);
            }
            for (int prime : primes) {
                if (prime > smallest_prime[value] || 1LL * value * prime > n)
                    break;
                smallest_prime[value * prime] = prime;
            }
        }
    }

    vector<int> factorize(int value) const {
        assert(1 <= value && value < (int)smallest_prime.size());
        vector<int> factors;
        while (value > 1) {
            factors.push_back(smallest_prime[value]);
            value /= smallest_prime[value];
        }
        return factors;
    }
};
```

Complejidad: preproceso $O(N)$; factorización $O(\log n)$; memoria $O(N)$.

6.8 Criba segmentada: primos en un intervalo

Cuándo usar: Listar los primos de $[L, R]$ cuando el intervalo es corto y R es grande. Debe ser viable cribar hasta $\lfloor\sqrt{R}\rfloor$ y almacenar $R-L+1$ marcas; por ejemplo, $R \le 10^{12}$ y ancho hasta 10^6.

Complejidad: Con $B = \lfloor\sqrt{R}\rfloor$ y $W = R - L + 1$, tiempo $O(B \log\log B + \pi(B) + W \log\log R)$ y memoria $O(B+W)$ bits para las marcas, más $O(k)$ palabras para devolver los k primos. Las cotas se entienden asintóticamente; intervalos con $R < 2$ se resuelven directamente.

```cpp
// Requiere 0 <= L <= R y memoria/tiempo suficientes para sqrt(R) y el ancho.
vector<ll> segmented_primes(ll L, ll R) {
    assert(0 <= L && L <= R);
    if (R < 2) return {};

    ll root = (ll)sqrtl((long double)R);
    while (root > R / root) --root;
    while (root + 1 <= R / (root + 1)) ++root;

    vector<bool> base((size_t)root + 1, true);
    base[0] = base[1] = false;
    vector<bool> segment((size_t)(R - L) + 1, true);

    for (ll p = 2; p <= root; ++p) {
        if (!base[p]) continue;
        if (p <= root / p)
            for (ll x = p * p; x <= root; x += p)
                base[x] = false;

        // Primer multiplo >= L, sin usar (L+p-1) ni una suma fuera de R.
        ll delta = (p - L % p) % p;
        if (delta > R - L) continue;
        ll start = max(p * p, L + delta);
        if (start > R) continue;
        for (ll x = start; ; ) {
            segment[x - L] = false;
            if (R - x < p) break;
            x += p;
        }
    }

    if (L == 0) segment[0] = false;
    if (L <= 1) segment[1 - L] = false;
    vector<ll> answer;
    for (size_t i = 0; i < segment.size(); ++i)
        if (segment[i]) answer.push_back(L + (ll)i);
    return answer;
}
```

Ejemplo: `segmented_primes(100,120)` devuelve $101, 103, 107, 109, 113$. Se marca desde $\max(p^2, \lceil L/p \rceil p)$ para no borrar al propio primo p. No construye la criba completa hasta R. Tampoco evita el costo de cribar hasta $\sqrt{R}$: para un solo entero grande, usar Miller–Rabin. Si hay muchas consultas con el mismo límite, conviene reutilizar los primos base; esta versión los recalcula para mantener una interfaz autocontenida.

6.9 Factorización por división por tentativa

Cuándo usar: Una o pocas consultas con n moderado, por ejemplo hasta 10^9. No necesita criba ni generador aleatorio. Para muchas consultas, usar SPF; para factores grandes de 64 bits, Miller–Rabin y Brent.

Complejidad: $O(\sqrt{n})$ tiempo en el peor caso; $O(\log n)$ memoria de salida.

```cpp
vector<ll> trial_factorize(ll n) {
    assert(n >= 1);
    vector<ll> out;
    while (n % 2 == 0) { out.push_back(2); n /= 2; }
    for (ll d = 3; d <= n / d; d += 2)
        while (n % d == 0) { out.push_back(d); n /= d; }
    if (n > 1) out.push_back(n);
    return out;
}
```

Devuelve factores ordenados con repeticiones; para 1, un vector vacío. La condición `d <= n/d` evita overflow de `d*d`.

6.10 Listar divisores

Cuándo usar: Listar todos los divisores de un número aislado moderado sin necesitar preprocesamiento.

```cpp
vector<ll> divisors(ll n) {
    assert(n > 0);
    vector<ll> answer;
    for (ll d = 1; d <= n / d; ++d) if (n % d == 0) {
        answer.push_back(d);
        if (d != n / d) answer.push_back(n / d);
    }
    sort(answer.begin(), answer.end());
    return answer;
}
```

Complejidad: $O(\sqrt{n})$ tiempo y $O(\tau(n))$ memoria de salida.

6.11 Pollard–Rho con Brent

Cuándo usar: Factorizar enteros de 64 bits con poco código. Incluye Miller–Rabin determinista de siete bases y Brent con bloque 128 y reinicios.

Complejidad: Primalidad: $O(\log n)$ productos modulares. Para encontrar un factor p, tiempo heurístico $O(\sqrt{p})$, aproximadamente $O(n^{1/4})$ en el caso difícil; memoria $O(\log n)$ por recursión. Sin cota de tiempo total por los reinicios.

Esta versión es completa: copiar sus dos bloques. Usa `u128%n`; no requiere el código optimizado de abajo. Requiere $1 \le n < 2^{64}$. Miller–Rabin verifica primalidad; Brent encuentra un divisor compuesto o primo.

```cpp
using u64 = uint64_t;
using u128 = __uint128_t;

u64 multiply_mod_u64(u64 a, u64 b, u64 mod) {
    return (u128)a * b % mod;
}

u64 power_mod_u64(u64 base, u64 exponent, u64 mod) {
    u64 result = 1;
    while (exponent) {
        if (exponent & 1) result = multiply_mod_u64(result, base, mod);
        base = multiply_mod_u64(base, base, mod);
        exponent >>= 1;
    }
    return result;
}

bool is_prime_u64(u64 n) {
    if (n < 2) return false;
    for (u64 prime : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                      19ULL, 23ULL, 29ULL, 31ULL, 37ULL})
        if (n % prime == 0) return n == prime;

    int power_of_two = 0;
    u64 odd = n - 1;
    while ((odd & 1) == 0) odd >>= 1, ++power_of_two;

    for (u64 base : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL,
                     9780504ULL, 1795265022ULL}) {
        if (base % n == 0) continue;
        u64 value = power_mod_u64(base % n, odd, n);
        if (value == 1 || value == n - 1) continue;
        bool witness = true;
        for (int round = 1; round < power_of_two; ++round) {
            value = multiply_mod_u64(value, value, n);
            if (value == n - 1) { witness = false; break; }
        }
        if (witness) return false;
    }
    return true;
}
```

Brent y factorización. Un MCD por bloque; si da n, recuperar el bloque individualmente. Reiniciar si no se obtiene un divisor propio.

```cpp
u64 pollard_rho(u64 n) {
    if (n % 2 == 0) return 2;
```

```
    if (n % 3 == 0) return 3;
    static mt19937_64 rng(chrono::steady_clock::now()
                          .time_since_epoch().count());

    while (true) {
        u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
        u64 y = uniform_int_distribution<u64>(0, n - 1)(rng);
        u64 x = 0, saved = y, divisor = 1;
        constexpr u64 BLOCK = 128, LIMIT = 1ULL << 18;
        u64 length = 1, batch = 0;
        auto advance = [&](u64 value) {
            return (u64)(((u128)multiply_mod_u64(value, value, n) + c) % n);
        };
        while (divisor == 1 && length <= LIMIT) {
            x = y;
            for (u64 i = 0; i < length; ++i) y = advance(y);
            for (u64 offset = 0; offset < length && divisor == 1;
                 offset += batch) {
                saved = y;
                batch = min(BLOCK, length - offset);
                u64 product = 1;
                for (u64 i = 0; i < batch; ++i) {
                    y = advance(y);
                    u64 difference = x > y ? x - y : y - x;
                    product = multiply_mod_u64(product, difference, n);
                }
                divisor = gcd(product, n);
            }
            length <<= 1;
        }
        if (divisor == n) {
            divisor = 1;
            for (u64 i = 0; i < batch && divisor == 1; ++i) {
                saved = advance(saved);
                u64 difference = x > saved ? x - saved : saved - x;
                divisor = gcd(difference, n);
            }
        }
        if (divisor > 1 && divisor < n) return divisor;
    }
}

void factor_u64(u64 n, map<u64,int> &factors) {
    assert(n >= 1);
    if (n == 1) return;
    if (is_prime_u64(n)) { ++factors[n]; return; }
    u64 divisor = pollard_rho(n);
    factor_u64(divisor, factors);
    factor_u64(n / divisor, factors);
}
```

Uso: `factor_u64(n, factores)`, donde `factores` es un `map<u64,int>`. Agrega los exponentes al mapa; para $n = 1$ no agrega nada. Llamar directamente a `pollard_rho` solo con un compuesto. Las siete bases garantizan exactitud de primalidad para 64 bits; la aleatoriedad de Brent afecta el tiempo, no los factores.

6.12 Pollard–Rho con Brent optimizado

Cuándo usar: Muchas factorizaciones o límites ajustados. Esta alternativa completa incluye Montgomery, Miller–Rabin de siete bases, Brent con bloque 256, reinicios, extracción de primos pequeños, cuadrados y acumulación en vector.

Complejidad: Conserva el tiempo heurístico de Brent: $O(\sqrt{p})$ operaciones modulares para hallar un factor p. Ordenar k factores: $O(k \log k)$; memoria de salida y recursión $O(\log n)$. No garantiza un tiempo total máximo.

Copiar **solo este bloque**: no depende de la versión anterior. Se usa un namespace para que ambas opciones puedan convivir en el notebook. La entrada es `rho_optimized::factorize(n)` y requiere $n \geq 1$; devuelve factores primos ordenados con repeticiones, o un vector vacío para 1. La primalidad está disponible como `rho_optimized::is_prime(n)`.

Montgomery representa a como $a2^{64} \bmod n$ y evita dividir en cada producto. La reducción por diferencia de partes altas evita necesitar un acarreo de 129 bits, incluso cuando $n \geq 2^{63}$. Los pares se quitan antes de construir el contexto. Cada contexto guarda su propio módulo.

```
namespace rho_optimized {
using u64 = uint64_t;
using u128 = __uint128_t;

constexpr array<u64, 12> SMALL_PRIMES64 = {
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37
};

struct Montgomery64 {
    u64 mod, inverse = 1, r2;
    explicit Montgomery64(u64 n) : mod(n) {
        assert(n > 1 && (n & 1));
        // Inversa positiva de n modulo 2^64; overflow unsigned intencional.
        for (int i = 0; i < 6; ++i)
            inverse *= 2 - mod * inverse;
        u64 r = (u128(1) << 64) % mod;
        r2 = (u128)r * r % mod;
    }
    // Requiere t < mod * 2^64. Todo producto de residuos < mod cumple.
    u64 reduce(u128 t) const {
        u64 q = (u64)t * inverse;
```

```
        u64 hi = t >> 64;
        u64 subtract = ((u128)q * mod) >> 64;
        return hi >= subtract ? hi - subtract : mod - (subtract - hi);
    }
    u64 encode(u64 a) const { return mul(a % mod, r2); }
    u64 decode(u64 a) const { return reduce(a); }
    u64 one() const { return reduce(r2); }
    u64 mul(u64 a, u64 b) const { return reduce((u128)a * b); }
    u64 add(u64 a, u64 b) const {
        return a >= mod - b ? a - (mod - b) : a + b;
    }
};

u64 power_mod_encoded(const Montgomery64& ctx, u64 base, u64 exponent) {
    u64 result = ctx.one();
    while (exponent) {
        if (exponent & 1) result = ctx.mul(result, base);
        base = ctx.mul(base, base);
        exponent >>= 1;
    }
    return result;
}

bool is_prime(u64 n) {
    if (n < 2) return false;
    for (u64 p : SMALL_PRIMES64)
        if (n % p == 0) return n == p;

    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) d >>= 1, ++s;
    Montgomery64 ctx(n);
    u64 one = ctx.one(), minus_one = ctx.encode(n - 1);
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL,
                  9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = power_mod_encoded(ctx, ctx.encode(a), d);
        if (x == one || x == minus_one) continue;
        bool composite = true;
        for (int r = 1; r < s; ++r) {
            x = ctx.mul(x, x);
            if (x == minus_one) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}

// Requiere n compuesto. Bloque 256 y reinicios por intento.
u64 pollard(u64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    static mt19937_64 rng(chrono::steady_clock::now()
                          .time_since_epoch().count());
    constexpr u64 BLOCK = 256, LIMIT = 1ULL << 18;
    Montgomery64 ctx(n);
    while (true) {
        u64 c = ctx.encode(uniform_int_distribution<u64>(1, n - 1)(rng));
        u64 y = ctx.encode(uniform_int_distribution<u64>(0, n - 1)(rng));
        u64 x = 0, saved = y, divisor = 1;
        u64 length = 1, batch = 0;
        auto advance = [&](u64 value) {
            return ctx.add(ctx.mul(value, value), c);
        };
        while (divisor == 1 && length <= LIMIT) {
            x = y;
            for (u64 i = 0; i < length; ++i) y = advance(y);
            for (u64 offset = 0; offset < length && divisor == 1;
                 offset += batch) {
                saved = y;
                batch = min(BLOCK, length - offset);
                u64 product = ctx.one();
                for (u64 i = 0; i < batch; ++i) {
                    y = advance(y);
                    u64 difference = x > y ? x - y : y - x;
                    product = ctx.mul(product, difference);
                }
                divisor = gcd(product, n);
            }
            length <<= 1;
        }
        if (divisor == n) {
            divisor = 1;
            for (u64 i = 0; i < batch && divisor == 1; ++i) {
                saved = advance(saved);
                u64 difference = x > saved ? x - saved : saved - x;
                divisor = gcd(difference, n);
            }
        }
        if (divisor > 1 && divisor < n) return divisor;
    }
}

// sqrtl solo aproxima; las correcciones enteras hacen exacta la raiz.
u64 isqrt(u64 n) {
    u64 r = (u64)sqrtl((long double)n);
```

```
    while ((u128)r * r > n) --r;
    while ((u128)(r + 1) * (r + 1) <= n) ++r;
    return r;
}

void collect(u64 n, vector<u64>& out) {
    if (n == 1) return;
    u64 r = isqrt(n);
    if ((u128)r * r == n) {
        size_t begin = out.size();
        collect(r, out);
        size_t end = out.size();
        for (size_t i = begin; i < end; ++i) {
            u64 p = out[i];
            out.push_back(p);
        }
        return;
    }
    if (is_prime(n)) { out.push_back(n); return; }
    u64 d = pollard(n);
    collect(d, out);
    collect(n / d, out);
}

vector<u64> factorize(u64 n) {
    assert(n >= 1); // 0 no tiene factorizacion prima finita.
    vector<u64> out;
    out.reserve(64);
    // Quitar factores pequenos una sola vez, antes de la recursion.
    for (u64 p : SMALL_PRIMES64) {
        while (n % p == 0) { out.push_back(p); n /= p; }
        if (n == 1) break;
    }
    collect(n, out);
    sort(out.begin(), out.end());
    return out;
}
} // namespace rho_optimized
```

Detalles importantes. La inversa usa overflow unsigned intencional; los productos usan `u128`. No mezclar residuos normales y codificados; la identidad codificada es `ctx.one()`. Los MCD también sirven sobre residuos codificados porque $\gcd(2^{64}a, n) = \gcd(a, n)$ para n impar. La raíz usa `sqrtl` solo como aproximación y se corrige exactamente. Los cuadrados se factorizan por la raíz una vez y se duplican sus factores.

El bloque 256 mejoró los tiempos locales para factores grandes, pero no es un óptimo universal. La versión corta puede pasar el mismo problema y ser preferible por simplicidad. En *Fuzzy Factorization*, conservar diez dígitos deja un número menor que 10^{10}: incluso división por tentativa alcanza. `assert` detecta entradas inválidas durante pruebas y se elimina con `-DNDEBUG`; el llamador debe respetar $n \geq 1$.

6.13 Exponenciación de matrices

Cuándo usar: Acelerar una recurrencia lineal o transiciones repetidas un número enorme de veces, expresando un paso como multiplicación por una matriz.

```
using Matrix = vector<vector<ll>>;

Matrix matrix_multiply(const Matrix &a, const Matrix &b, ll mod) {
    assert(!a.empty() && !b.empty() && !b[0].empty());
    assert(!a[0].empty() && a[0].size() == b.size() && mod > 0);
    int rows = a.size(), inner = b.size(), columns = b[0].size();
    Matrix result(rows, vector<ll>(columns, 0));
    for (int i = 0; i < rows; ++i)
        for (int k = 0; k < inner; ++k)
            for (int j = 0; j < columns; ++j)
                result[i][j] = (result[i][j] +
                    (i128)a[i][k] * b[k][j]) % mod;
    return result;
}

Matrix matrix_power(Matrix base, unsigned long long exponent, ll mod) {
    int n = base.size();
    assert(n > 0 && (int)base[0].size() == n && mod > 0);
    Matrix result(n, vector<ll>(n, 0));
    for (int i = 0; i < n; ++i) result[i][i] = 1 % mod;
    while (exponent) {
        if (exponent & 1ULL) result = matrix_multiply(result, base, mod);
        base = matrix_multiply(base, base, mod);
        exponent >>= 1;
    }
    return result;
}
```

Complejidad: $O(n^3 \log e)$ tiempo; $O(n^2)$ memoria.

6.14 Eliminación gaussiana y determinante

Cuándo usar: Resolver sistemas lineales, calcular rango o determinante; elegir la versión real con pivoteo o modular según el dominio del problema.

Usar para resolver sistemas lineales, calcular rango o determinante. Esta versión trabaja con reales y pivoteo parcial; para módulo primo reemplazar la división por inverso modular.

Complejidad: tiempo $O(nm \min(n, m))$ para n ecuaciones y m variables; memoria $O(nm)$. Determinante cuadrado: $O(n^3)$.

```cpp
int gaussian_elimination(vector<vector<long double>> matrix,
                         vector<long double> &solution) {
    const long double epsilon = 1e-12L;
    int rows = (int)matrix.size();
    int columns = (int)matrix[0].size() - 1;
    vector<int> pivot_row(columns, -1);
    int row = 0;
    for (int column = 0; column < columns && row < rows; ++column) {
        int selected = row;
        for (int candidate = row; candidate < rows; ++candidate)
            if (fabsl(matrix[candidate][column]) > fabsl(matrix[selected][column
    ]))
                selected = candidate;
        if (fabsl(matrix[selected][column]) < epsilon) continue;
        swap(matrix[selected], matrix[row]);
        pivot_row[column] = row;
        for (int candidate = 0; candidate < rows; ++candidate) if (candidate !=
    row) {
            long double factor = matrix[candidate][column] / matrix[row][column
    ];
            for (int index = column; index <= columns; ++index)
                matrix[candidate][index] -= factor * matrix[row][index];
        }
        ++row;
    }
    for (int candidate = 0; candidate < rows; ++candidate) {
        bool all_zero = true;
        for (int column = 0; column < columns; ++column)
            all_zero &= fabsl(matrix[candidate][column]) < epsilon;
        if (all_zero && fabsl(matrix[candidate][columns]) >= epsilon) return 0;
    }
    solution.assign(columns, 0);
    for (int column = 0; column < columns; ++column)
        if (pivot_row[column] != -1)
            solution[column] = matrix[pivot_row[column]][columns]
                             / matrix[pivot_row[column]][column];
    return row == columns ? 1 : 2; // unica / infinitas
}
// matriz cuadrada es invertible:
long double determinant(vector<vector<long double>> matrix) {
    const long double epsilon = 1e-12L;
    int size = (int)matrix.size();
    long double answer = 1;
    for (int column = 0; column < size; ++column) {
        int pivot = column;
        for (int row = column; row < size; ++row)
            if (fabsl(matrix[row][column]) > fabsl(matrix[pivot][column])) pivot
     = row;
        if (fabsl(matrix[pivot][column]) < epsilon) return 0;
        if (pivot != column) swap(matrix[pivot], matrix[column]), answer = -
    answer;
        answer *= matrix[column][column];
        for (int row = column + 1; row < size; ++row) {
            long double factor = matrix[row][column] / matrix[column][column];
            for (int index = column + 1; index < size; ++index)
                matrix[row][index] -= factor * matrix[column][index];
        }
    }
    return answer;
}
```

```cpp
vector<vector<long double>> augmented={
    {1,1,3},
    {2,-1,0}
};
vector<long double> solution;
int type=gaussian_elimination(augmented,solution);

if(type==0) cout<<"Sin solucion\n";
else if(type==1)
    cout<<solution[0]<<" "<<solution[1]<<'\n'; // 1 2
else
    cout<<"Infinitas soluciones\n";
vector<vector<long double>> coefficients={
    {1,1},
    {2,-1}
};
cout<<(double)determinant(coefficients)<<'\n'; // -3
```

6.15 Base XOR de enteros de 64 bits

Cuándo usar: Consultar si un valor es XOR de un subconjunto o maximizar ese XOR. Es eliminación gaussiana sobre $GF(2)$, sin divisiones.

Complejidad: Insertar, consultar pertenencia y maximizar: $O(B)$ con $B = 64$; memoria $O(B)$. El rango indica cuántos vectores son independientes.

```cpp
struct XorBasis {
    array<uint64_t, 64> basis{};
    int rank = 0;
    bool insert(uint64_t value) {
        for (int bit = 63; bit >= 0; --bit) if (value >> bit & 1) {
            if (basis[bit]) value ^= basis[bit];
            else { basis[bit] = value; ++rank; return true; }
        }
        return false;
```

```
    }
    bool contains(uint64_t value) const {
        for (int bit = 63; bit >= 0; --bit)
            if (value >> bit & 1) value ^= basis[bit];
        return value == 0;
    }
    uint64_t maximum(uint64_t initial = 0) const {
        for (int bit = 63; bit >= 0; --bit)
            initial = max(initial, initial ^ basis[bit]);
        return initial;
    }
};
void xor_basis_example() {
    XorBasis basis;
    basis.insert(3); basis.insert(5); basis.insert(6);
    cout << basis.rank << ' ' << basis.maximum() << ' ' << basis.contains(7)
         << '\n'; // 2 6 0
}
```

6.16 Dependencia XOR de bitsets con reconstrucción

Cuándo usar: Los vectores tienen más de 64 bits y se necesita recuperar un subconjunto no vacío con XOR cero, como Brazilian FootXOR.

Complejidad: Con n vectores de B bits y capacidad $N \geq n$, tiempo $O(nB\lceil(B+N)/w\rceil)$ y memoria $O(B(B+N))$ bits, más la entrada. Las operaciones de bitset procesan palabras de w bits.

```
template<size_t B, size_t N>
optional<vector<int>> xor_dependency(const vector<bitset<B>> &input) {
    assert(input.size() <= N);
    vector<bitset<B>> basis(B);
    vector<bitset<N>> combination(B);
    vector<char> used(B);
    for (int id = 0; id < (int)input.size(); ++id) {
        auto value = input[id];
        bitset<N> chosen; chosen[id] = true;
        for (int bit = (int)B - 1; bit >= 0; --bit) if (value[bit]) {
            if (used[bit]) {
                value ^= basis[bit]; chosen ^= combination[bit];
            } else {
                used[bit] = true; basis[bit] = value; combination[bit] = chosen;
                break;
            }
        }
        if (value.none()) {
            vector<int> indices;
            for (int i = 0; i <= id; ++i) if (chosen[i]) indices.push_back(i);
            return indices;
        }
    }
    return nullopt;
}
void xor_dependency_example() {
    vector<bitset<4>> input = {bitset<4>(3), bitset<4>(5), bitset<4>(6)};
    auto chosen = xor_dependency<4, 3>(input);
    if (chosen) for (int i : *chosen) cout << i << ' '; // 0 1 2
}
```

Para exigir cantidad par de vectores, agregar a cada entrada un bit extra en 1. Toda dependencia resultante tendrá cardinalidad par; dividir sus `indices` en dos mitades da equipos del mismo tamaño y XOR igual. En Brazilian FootXOR (Latin America Championship 2025), $n, k \leq 1500$: usar `xor_dependency<1501,1500>` con el bit 1500 de paridad. El siguiente adaptador lee un caso y escribe los dos equipos:

```
void brazilian_footxor() {
    int n, k; cin >> n >> k;
    assert(1 <= n && n <= 1500 && 1 <= k && k <= 1500);
    vector<bitset<1501>> players(n);
    for (auto &player : players) {
        string s; cin >> s;
        for (int j = 0; j < k; ++j) player[j] = s[j] == '1';
        player[1500] = true; // Paridad de la cantidad de integrantes.
    }
    auto chosen = xor_dependency<1501,1500>(players);
    if (!chosen) { cout << "*\n"; return; }
    string teams(n, '0');
    for (int i = 0; i < (int)chosen->size(); ++i)
        teams[(*chosen)[i]] = i < (int)chosen->size() / 2 ? '1' : '2';
    cout << teams << '\n';
}
```

Cuidado: Una base escalar de 64 bits no sustituye esta variante para cadenas de 1500 bits. `nullopt` significa independencia, no un subconjunto vacío.

6.17 NTT y convolución

Cuándo usar: Multiplicar polinomios o contar combinaciones mediante convolución exacta bajo un primo compatible, evitando errores de punto flotante.

Convolución exacta módulo $998244353 = 119 \cdot 2^{23} + 1$, cuya raíz primitiva es 3. La longitud transformada no puede superar 2^{23}.

Las raíces de cada etapa se precalculan una sola vez y se reutilizan en llamadas posteriores. Para invertir se revierten los índices $1, \ldots, n-1$, se aplica la misma transformada y se divide por n módulo `NTT_MOD`. Se conservan `vector<int>`, la raíz primitiva 3 y la interfaz `number_theoretic_transform(values,inverse)`.

Complejidad: Para una longitud transformada n: $O(n \log n)$ tiempo; $O(n)$ memoria de trabajo y $O(N_{\max})$ memoria persistente para raíces, donde $N_{\max}$ es la mayor longitud transformada hasta el momento.

Cuidado: La caché conserva la memoria de la mayor transformada, incluso si las siguientes son pequeñas. En jueces con `int` de 4 bytes, las raíces ocupan aproximadamente 2 MiB para 2^{19} elementos y 32 MiB para 2^{23}, además de los arreglos de entrada y trabajo.

```
const int NTT_MOD = 998244353;
const int NTT_ROOT = 3;

void number_theoretic_transform(vector<int> &values, bool inverse) {
    int n = values.size();
    assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
    static vector<int> roots(2, 1);
    for (int half = roots.size() / 2; half < n / 2; half *= 2) {
        int root = mod_power(NTT_ROOT, (NTT_MOD - 1) / (4 * half), NTT_MOD);
        roots.resize(4 * half);
        for (int i = 2 * half; i < 4 * half; ++i)
            roots[i] = (ll)roots[i / 2] * (i & 1 ? root : 1) % NTT_MOD;
    }
    if (inverse) reverse(values.begin() + 1, values.end());
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(values[i], values[j]);
    }
    for (int half = 1; half < n; half *= 2)
        for (int start = 0; start < n; start += 2 * half)
            for (int offset = 0; offset < half; ++offset) {
                int even = values[start + offset];
                int odd = (ll)roots[half + offset] * values[start + half +
    offset] % NTT_MOD;
                values[start + offset] = even + odd;
                if (values[start + offset] >= NTT_MOD) values[start + offset] -=
     NTT_MOD;
                values[start + half + offset] = even - odd;
                if (values[start + half + offset] < 0) values[start + half +
    offset] += NTT_MOD;
            }
    if (inverse) {
        int inverse_n = mod_power(n, NTT_MOD - 2, NTT_MOD);
        for (int &value : values) value = (ll)value * inverse_n % NTT_MOD;
    }
}

vector<int> convolution(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int result_size = a.size() + b.size() - 1;
    int size = 1;
    while (size < result_size) size <<= 1;
    a.resize(size); b.resize(size);
    number_theoretic_transform(a, false);
    number_theoretic_transform(b, false);
    for (int i = 0; i < size; ++i) a[i] = (ll)a[i] * b[i] % NTT_MOD;
    number_theoretic_transform(a, true);
    a.resize(result_size);
    return a;
}
```

Normalizar coeficientes al intervalo $[0, NTT_MOD)$ antes de llamar a `convolution`. Para enteros grandes sin módulo, usar varias NTT y CRT.

```
vector<int> a={1,2}; // 1+2x
vector<int> b={3,4}; // 3+4x
vector<int> c=convolution(a,b);
for(int value:c) cout<<value<<' '; // 3 10 8
```

6.17.1 Ejemplo de aplicación: sumas de pares

Se tienen dos listas A y B. Para cada valor s, contar cuántos pares (a, b) cumplen $a + b = s$.

Construir arreglos de frecuencias:

$$freqA[x] = \#\{a \in A : a = x\}, \qquad freqB[y] = \#\{b \in B : b = y\}.$$

Entonces

$$ways[s] = \sum_{x+y=s} freqA[x] freqB[y],$$

que es exactamente una convolución.

```
vector<int> A={1,2,2},B={0,2};
int maximum=2;
vector<int> frequency_a(maximum+1),frequency_b(maximum+1);
for(int value:A) ++frequency_a[value];
for(int value:B) ++frequency_b[value];

vector<int> ways=convolution(frequency_a,frequency_b);
for(int sum=0;sum<(int)ways.size();++sum)
    cout<<sum<<": "<<ways[sum]<<'\n';
// 0:0, 1:1, 2:2, 3:1, 4:2
```

6.18 Continued fractions

```
/**
 * Author: Simon Lindholm
 * Date: 2018-07-15
 * License: CC0
 * Source: Wikipedia
 * Description: Given $N$ and a real number $x \ge 0$, finds the closest
    rational approximation $p/q$ with $p, q \le N$.
 * It will obey $|p/q - x| \le 1/qN$.
 *
 * For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$.
 * ($p_k/q_k$ alternates between $>x$ and $<x$.)
 * If $x$ is rational, $y$ eventually becomes $\infty$;
 * if $x$ is the root of a degree $2$ polynomial the $a$'s eventually become
    cyclic.
 * Time: O(\log N)
 * Status: stress-tested for n <= 300
 */

typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
       a = (ll)floor(y), b = min(a, lim),
       NP = b*P + LP, NQ = b*Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // Return {P, Q} here for a more canonical approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1/(y - (d)a)) > 3*N) {
      return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}
```

6.19 Binary search on fractions

```
/**
 * Author: Lucian Bicsi, Simon Lindholm
 * Date: 2017-10-31
 * License: CC0
 * Description: Given $f$ and $N$, finds the smallest fraction $p/q \in [0, 1]$
 * such that $f(p/q)$ is true, and $p, q \le N$.
 * You may want to throw an exception from $f$ if it finds an exact solution,
 * in which case $N$ can be removed.
 * Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
 * Time: O(\log(N))
 * Status: stress-tested for n <= 300
 */

struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

6.20 Table of modular inverses

The original source is a precomputation fragment; configure LIM and mod and place the loop inside initialization.

```
/**
 * Author: Simon Lindholm
 * Date: 2016-07-24
 * License: CC0
 * Source: Russian page
 * Description: Pre-computation of modular inverses. Assumes LIM $\le$ mod and
    that mod is a prime.
 * Status: Works
 */
```

```
#pragma once

// const ll mod = 1000000007, LIM = 200000; ///include-line
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

6.21 Discrete logarithm

```
/**
 * Author: Bjorn Martinsson
 * Date: 2020-06-03
 * License: CC0
 * Source: own work
 * Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or
 * $-1$ if no such $x$ exists. modLog(a,1,m) can be used to
 * calculate the order of $a$.
 * Time: $O(\sqrt m)$
 * Status: tested for all 0 <= a,x < 500 and 0 < m < 500.
 *
 * Details: This algorithm uses the baby-step giant-step method to
 * find (i,j) such that a^(n * i) = b * a^j (mod m), where n > sqrt(m)
 * and 0 < i, j <= n. If a and m are coprime then a^j has a modular
 * inverse, which means that a^(i * n - j) = b (mod m$).
 *
 * However this particular implementation of baby-step giant-step works even
 * without assuming a and m are coprime, using the following idea:
 *
 * Assume p^x is a prime divisor of m. Then we have 3 cases
 *   1. b is divisible by p^x
 *   2. b is divisible only by some p^y, 0<y<x
 *   3. b is not divisible by p
 * The important thing to note is that in case 2, modLog(a,b,m) (if
 * it exists) cannot be > sqrt(m), (technically it cannot be >= log2(m)).
 * So once all exponenents of a that are <= sqrt(m) has been checked, you
 * cannot have case 2. Case 2 is the only tricky case.
 *
 * So the modification allowing for non-coprime input involves checking all
 * exponents of a that are <= n, and then handling the non-tricky cases by
 * a simple gcd(a^n,m) == gcd(b,m) check.
 */
#pragma once

ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

6.22 Modular square root

```
/**
 * Author: Simon Lindholm
 * Date: 2016-08-31
 * License: CC0
 * Source: http://eli.thegreenplace.net/2009/03/07/computing-modular-square-
    roots-in-python/
 * Description: Tonelli-Shanks algorithm for modular square roots. Finds $x$ s.t
    . $x^2 = a \pmod p$ ($-x$ gives the other solution).
 * Time: O(\log^2 p) worst case, O(\log p) for most $p$
 * Status: Tested for all a,p <= 10000
 */
#pragma once

#include "ModPow.h"

ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  /// find a non-square mod p
  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
```

```
  }
}
```

6.23 Sums of floors and modular progressions

```
/**
 * Author: Simon Lindholm
 * Date: 2015-06-23
 * License: CC0
 * Source: own work
 * Description: Sums of mod'ed arithmetic progressions.
 *
 * \texttt{modsum(to, c, k, m)} = $\sum_{i=0}^{\mathrm{to}-1}{(ki+c) \% m}$.
 * \texttt{divsum} is similar but for floored division.
 * Time: $\log(m)$, with a large constant.
 * Status: Tested for all |k|,|c|,to,m <= 50, and on kattis:aladin
 */
#pragma once

typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
/// ^ written in a weird way to deal with overflows correctly

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

6.24 Totient sieve

```
/**
 * Author: Håkan Terelius
 * Date: 2009-09-25
 * License: CC0
 * Source: http://en.wikipedia.org/wiki/Euler's_totient_function
 * Description: \emph{Euler's $\phi$} function is defined as $\phi(n):=\#$ of
    positive integers $\leq n$ that are coprime with $n$.
 * $\phi(1)=1$, $p$ prime $\Rightarrow \phi(p^k)=(p-1)p^{k-1}$, $m,n$ coprime $\
    Rightarrow \phi(mn)=\phi(m)\phi(n)$.
 * If $n=p_1^{k_1}p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1-1)p_1^{k_1
    -1}...(p_r-1)p_r^{k_r-1}$.
 * $\phi(n)=n \cdot \prod_{p|n}(1-1/p)$.
 *
 * $\sum_{d|n} \phi(d) = n$, $\sum_{1\leq k \leq n, \gcd(k,n)=1} k = n \phi(n)
    /2, n>1$
 *
 * \textbf{Euler's thm}: $a,n$ coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n
    }$.
 *
 * \textbf{Fermat's little thm}: $p$ prime $\Rightarrow a^{p-1} \equiv 1 \pmod{p
    }$ $\forall a$.
 * Status: Tested
 */
#pragma once

const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

6.25 Recovering a linear recurrence

```
/**
 * Author: Lucian Bicsi
 * Date: 2017-10-31
 * License: CC0
 * Source: Wikipedia
 * Description: Recovers any $n$-order linear recurrence relation from the first
 * $2n$ terms of the recurrence.
 * Useful for guessing linear recurrences after brute-forcing the first terms.
 * Should work on any field, but numerical stability for floats is not
    guaranteed.
 * Output will have size $\le n$.
 * Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
 * Time: O(N^2)
 * Status: bruteforce-tested mod 5 for n <= 5 and all s
 */
#pragma once

#include "../number-theory/ModPow.h"
```

```

vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;

  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }

  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

6.26 Distant term of a linear recurrence

```
/**
 * Author: Lucian Bicsi
 * Date: 2018-02-14
 * License: CC0
 * Source: Chinese material
 * Description: Generates the $k$'th term of an $n$-order
 * linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$,
 * given $S[0 \ldots \ge n-1]$ and $tr[0 \ldots n-1]$.
 * Faster than matrix multiplication.
 * Useful together with Berlekamp--Massey.
 * Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number
 * Time: O(n^2 \log k)
 * Status: bruteforce-tested mod 5 for n <= 5
 */
#pragma once

const ll mod = 5; /** exclude-line */

typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }

  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```

6.27 Polynomial operations

```
/**
 * Author: David Rydh, Per Austrin
 * Date: 2003-03-16
 * Description:
 */
#pragma once

struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
  }
```

```
};
```

6.28 Polynomial interpolation

```
/**
 * Author: Simon Lindholm
 * Date: 2017-05-10
 * License: CC0
 * Source: Wikipedia
 * Description: Given $n$ points (x[i], y[i]), computes an n-1-degree polynomial $p$ that
 *  passes through them: $p(x) = a[0]*x^0 + ... + a[n-1]*x^{n-1}$.
 *  For numerical precision, pick $x[k] = c*\cos(k/(n-1)*\pi), k=0 \dots n-1$.
 * Time: O(n^2)
 */
#pragma once

typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

6.29 Real polynomial roots

```
/**
 * Author: Per Austrin
 * Date: 2004-02-08
 * License: CC0
 * Description: Finds the real roots to a polynomial.
 * Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0
 * Time: O(n^2 \log(1/\epsilon))
 */
#pragma once

#include "Polynomial.h"

vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```

6.30 Real convolution with FFT

```
/**
 * Author: Ludo Pulles, chilli, Simon Lindholm
 * Date: 2019-01-09
 * License: CC0
 * Source: http://neerc.ifmo.ru/trains/toulouse/2017/fft2.pdf (do read, it's excellent)
   Accuracy bound from http://www.daemonology.net/papers/fft.pdf
 * Description: fft(a) computes $\hat f(k) = \sum_x a[x] \exp(2\pi i \cdot k x / N)$ for all $k$. N must be a power of 2.
   Useful for convolution:
   \texttt{conv(a, b) = c}, where $c[x] = \sum a[i]b[x-i]$.
   For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back.
   Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2{N} < 9\cdot10^{14}$ (in practice $10^{16}$; higher for random inputs).
   Otherwise, use NTT/FFTMod.
 * Time: O(N \log N) with $N = |A|+|B|$ ($\tilde 1s$ for $N=2^{22}$)
 * Status: somewhat tested
 * Details: An in-depth examination of precision for both FFT and FFTMod can be found
 * here (https://github.com/simonlindholm/fft-precision/blob/master/fft-precision.md)
```

```
 */
#pragma once

typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)  /// include-line
      auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];        /// exclude-line
      C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);           /// exclude-line
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

6.31 Convolution modulo arbitrary integers

```
/**
 * Author: chilli
 * Date: 2019-04-25
 * License: CC0
 * Source: http://neerc.ifmo.ru/trains/toulouse/2017/fft2.pdf
 * Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers
 * as long as $N\log_2N\cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice $10^{16}$ or higher).
 * Inputs must be in $[0, \text{mod})$.
 * Time: O(N \log N), where $N = |A|+|B|$ (twice as slow as NTT or FFT)
 * Status: stress-tested
 * Details: An in-depth examination of precision for both FFT and FFTMod can be found
 * here (https://github.com/simonlindholm/fft-precision/blob/master/fft-precision.md)
 */
#pragma once

#include "FastFourierTransform.h"

typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

6.32 AND, OR and XOR convolutions

The active code computes AND convolution. The commented alternatives select OR or XOR; XOR also needs the inverse normalization.

```
/**
 * Author: Lucian Bicsi
 * Date: 2015-06-25
 * License: GNU Free Documentation License 1.2
 * Source: csacademy
 * Description: Transform to a basis with fast convolutions of the form
 * $\displaystyle c[z] = \sum\nolimits_{z = x \oplus y} a[x] \cdot b[y]$,
 * where $\oplus$ is one of AND, OR, XOR. The size of $a$ must be a power of two
    .
 * Time: O(N \log N)
 * Status: stress-tested
 */
#pragma once

void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        // inv ? pii(v, u - v) : pii(u + v, u); // OR /// include-line
        // pii(u + v, u - v);                   // XOR /// include-line
    }
  }
  // if (inv) for (int& x : a) x /= sz(a); // XOR only /// include-line
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

6.33 Golden-section search

```
/**
 * Author: Ulf Lundstrom
 * Date: 2009-04-17
 * License: CC0
 * Source: Numeriska algoritmer med matlab, Gerd Eriksson, NADA, KTH
 * Description: Finds the argument minimizing the function $f$ in the interval $
    [a,b]$
 * assuming $f$ is unimodal on the interval, i.e. has only one local minimum and
     no local
 * maximum. The maximum error in the result is $eps$. Works equally well for
    maximization
 * with a small change in the code. See TernarySearch.h in the Various chapter
    for a
 * discrete version.
 * Usage:
  double func(double x) { return 4+x+.3*x*x; }
  double xmin = gss(-1000,1000,func);
 * Time: O(\log((b-a) / \epsilon))
 * Status: tested
 */
#pragma once

/// It is important for r to be precise, otherwise we don't necessarily maintain
     the inequality a < x1 < x2 < b.
double gss(double a, double b, double (*f)(double)) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    }
  return a;
}
```

6.34 Two-dimensional local optimization

```
/**
 * Author: Simon Lindholm
 * Date: 2015-02-04
 * License: CC0
 * Source: Johan Sannemo
 * Description: Poor man's optimization for unimodal functions.
 * Status: used with great success
 */
#pragma once

typedef array<double, 2> P;

template<class F> pair<double, P> hillClimb(P start, F f) {
  pair<double, P> cur(f(start), start);
  for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
    rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
      P p = cur.second;
      p[0] += dx*jmp;
      p[1] += dy*jmp;
      cur = min(cur, make_pair(f(p), p));
```

```
    }
  }
  return cur;
}
```

6.35 Adaptive numerical integration

```
/**
 * Author: Simon Lindholm
 * Date: 2015-02-11
 * License: CC0
 * Source: Wikipedia
 * Description: Fast integration using an adaptive Simpson's rule.
 * Usage:
  double sphereVolume = quad(-1, 1, [](double x) {
  return quad(-1, 1, [\&](double y) {
  return quad(-1, 1, [\&](double z) {
  return x*x + y*y + z*z < 1; });});});
 * Status: mostly untested
 */
#pragma once

typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}
```

6.36 Determinant modulo a composite integer

```
/**
 * Author: Unknown
 * Date: 2014-11-27
 * Source: somewhere on github
 * Description: Calculates determinant using modular arithmetics.
 * Modulos can also be removed to get a pure-integer version.
 * Time: $O(N^3)$
 * Status: bruteforce-tested for N <= 3, mod <= 7
 */
#pragma once

const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

6.37 Real matrix inverse

```
/**
 * Author: Max Bennedich
 * Date: 2004-02-08
 * Description: Invert matrix $A$. Returns rank; result is stored in $A$ unless
    singular (rank < n).
 * Can easily be extended to prime moduli; for prime powers, repeatedly
 * set $A^{-1} = A^{-1} (2I - AA^{-1})\  (\text{mod }p^k)$ where $A^{-1}$ starts
    as
 * the inverse of A mod p, and k is doubled in each step.
 * Time: O(n^3)
 * Status: Slightly tested
 */
#pragma once

int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
```

```
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  /// forget A at this point, just eliminate tmp backward
  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }

  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

6.38 Matrix inverse modulo a prime

```
/**
 * Author: Simon Lindholm
 * Date: 2016-12-08
 * Source: The regular matrix inverse code
 * Description: Invert matrix $A$ modulo a prime.
 * Returns rank; result is stored in $A$ unless singular (rank < n).
 * For prime powers, repeatedly set $A^{-1} = A^{-1} (2I - AA^{-1})\  (\text{mod }p^k)$ where $A^{-1}$ starts as
 * the inverse of A mod p, and k is doubled in each step.
 * Time: O(n^3)
 * Status: Slightly tested
 */
#pragma once

#include "../number-theory/ModPow.h"

int matInv(vector<vector<ll>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<ll>> tmp(n, vector<ll>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n) if (A[j][k]) {
      r = j; c = k; goto found;
    }
    return i;
found:
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    ll v = modpow(A[i][i], mod - 2);
    rep(j,i+1,n) {
      ll f = A[j][i] * v % mod;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
      rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod;
    }
    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    ll v = A[j][i];
    rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
  }

  rep(i,0,n) rep(j,0,n)
    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*mod;
  return n;
}
```

6.39 Uniquely determined variables in singular systems

The original file is a patch to SolveLinear.h, not a standalone implementation; apply the displayed changes to that dependency.

```
/**
 * Author: Simon Lindholm
```

```
 * Date: 2016-09-06
 * License: CC0
 * Source: me
 * Description: To get all uniquely determined values of $x$ back from
    SolveLinear, make the following changes:
 * Status: tested on kattis:equationsolverplus, stress-tested
 */
#pragma once

#include "SolveLinear.h"

rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; }
```

6.40 Linear systems over F2

```
/**
 * Author: Simon Lindholm
 * Date: 2016-08-27
 * License: CC0
 * Source: own work
 * Description: Solves $Ax = b$ over $\mathbb F_2$. If there are multiple
    solutions, one is returned arbitrarily.
 *  Returns rank, or -1 if no solutions. Destroys $A$ and $b$.
 * Time: O(n^2 m)
 * Status: bruteforce-tested for n, m <= 4
 */
#pragma once

typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

6.41 Linear programming: simplex

```
/**
 * Author: Stanford
 * Source: Stanford Notebook
 * License: MIT
 * Description: Solves a general linear maximization problem: maximize $c^T x$
    subject to $Ax \le b$, $x \ge 0$.
 * Returns -inf if there is no solution, inf if there are arbitrarily good
    solutions, or the maximum value of $c^T x$ otherwise.
 * The input vector is set to an optimal $x$ (or in the unbounded case, an
    arbitrary solution fulfilling the constraints).
 * Numerical stability is not guaranteed. For better performance, define
    variables such that $x = 0$ is viable.
 * Usage:
 * vvd A = {{1,-1}, {-1,1}, {-1,-2}};
 * vd b = {1,1,-4}, c = {-1,-1}, x;
 * T val = LPSolver(A, b, c).solve(x);
 * Time: O(NM * \#pivots), where a pivot may be e.g. an edge relaxation. O(2^n)
    in the general case.
 * Status: seems to work?
 */
#pragma once

typedef double T; // long double, Rational, double + mod<P>...
```

```
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver {
  int m, n;
  vi N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }

  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```

6.42 Permutation indexing

```
/**
 * Author: Simon Lindholm
 * Date: 2018-07-06
 * License: CC0
 * Description: Permutation -> integer conversion. (Not order preserving.)
 * Integer -> permutation can use a lookup table.
 * Time: O(n)
 */
#pragma once

int permToInt(vi& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}
```

7. Geometría

7.1 Geometría 2D entera

Cuándo usar: Orientación, colinealidad, intersección de segmentos y distancias al cuadrado con coordenadas enteras y decisiones exactas.

Complejidad: cada operación $O(1)$ tiempo y memoria.

Para coordenadas enteras, usar producto cruzado en `__int128`.

```
struct Point {
    ll x, y;
    Point operator+(Point other) const { return {x + other.x, y + other.y}; }
    Point operator-(Point other) const { return {x - other.x, y - other.y}; }
    bool operator<(Point other) const { return tie(x, y) < tie(other.x, other.y)
    ; }
    bool operator==(Point other) const { return x == other.x && y == other.y; }
};

i128 cross(Point a, Point b) { return (i128)a.x * b.y - (i128)a.y * b.x; }
i128 cross(Point a, Point b, Point c) { return cross(b - a, c - a); }
i128 dot(Point a, Point b) { return (i128)a.x * b.x + (i128)a.y * b.y; }
i128 squared_distance(Point a, Point b) { return dot(b - a, b - a); }

int orientation(Point a, Point b, Point c) {
    i128 value = cross(a, b, c);
    return (value > 0) - (value < 0); // 1 CCW, -1 CW, 0 collinear
}

bool on_segment(Point a, Point b, Point p) {
    return cross(a, b, p) == 0 && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
        && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

bool segments_intersect(Point a, Point b, Point c, Point d) {
    int ab_c = orientation(a, b, c), ab_d = orientation(a, b, d);
    int cd_a = orientation(c, d, a), cd_b = orientation(c, d, b);
    if (ab_c == 0 && on_segment(a, b, c)) return true;
    if (ab_d == 0 && on_segment(a, b, d)) return true;
    if (cd_a == 0 && on_segment(c, d, a)) return true;
    if (cd_b == 0 && on_segment(c, d, b)) return true;
    return ab_c != ab_d && cd_a != cd_b;
}
```

```
Point a{0,0},b{4,0},c{2,2},d{2,-2},p{2,0};
cout<<orientation(a,b,c)<<'\n';       // 1: c esta a la izquierda de a->b
cout<<on_segment(a,b,p)<<'\n';        // 1: p pertenece al segmento [a,b]
cout<<segments_intersect(a,b,c,d)<<'\n'; // 1: ambos segmentos se cruzan
cout<<(ll)squared_distance(a,c)<<'\n';   // 8: (2-0)^2+(2-0)^2
// cross(u,v): signo del giro/area; dot(u,v)=0 si son perpendiculares.
```

7.2 Área y convex hull

Cuándo usar: Calcular àrea firmada o obtener el menor polígono convexo que contiene un conjunto de puntos antes de aplicar calipers u otras consultas convexas.

```
i128 polygon_double_area(const vector<Point> &polygon) {
    if (polygon.empty()) return 0;
    i128 area = 0;
    int n = polygon.size();
    for (int i = 0; i < n; ++i)
        area += cross(polygon[i], polygon[(i + 1) % n]);
    return area >= 0 ? area : -area;
}

vector<Point> convex_hull(vector<Point> points) {
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());
    if (points.size() <= 1) return points;
    vector<Point> hull;
    for (Point p : points) {
        while (hull.size() >= 2 &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    size_t lower_size = hull.size();
    for (int i = (int)points.size() - 2; i >= 0; --i) {
        Point p = points[i];
        while (hull.size() > lower_size &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    hull.pop_back();
    return hull; // sin puntos collineales intermedios
}
```

```
vector<Point> points={{0,0},{2,0},{2,2},{0,2},{1,1}};
vector<Point> hull=convex_hull(points);
// hull contiene las 4 esquinas en orden antihorario; descarta el punto interior
    .
i128 double_area=polygon_double_area(hull);
cout<<hull.size()<<' '<<(ll)double_area<<'\n'; // 4 8; area real = 8/2 = 4
```

Complejidad: convex hull $O(n \log n)$ tiempo y $O(n)$ memoria; área $O(n)$ tiempo y $O(1)$ memoria adicional.

Las conversiones, normalización de ángulos, `atan2` y funciones trigonométricas están en

"Matemática cotidiana en C++".

7.3 Rectas con punto flotante

Cuándo usar: Proyección, distancia e intersección de rectas cuando el resultado no necesita ser entero y se controla el error con epsilon.

Complejidad: cada operación de rectas $O(1)$ tiempo y memoria.

```cpp
const long double GEOMETRY_EPS = 1e-12L;

struct DPoint {
    long double x, y;
    DPoint operator+(DPoint other) const { return {x + other.x, y + other.y}; }
    DPoint operator-(DPoint other) const { return {x - other.x, y - other.y}; }
    DPoint operator*(long double scale) const { return {x * scale, y * scale}; }
    DPoint operator/(long double scale) const { return {x / scale, y / scale}; }
};

long double dot(DPoint a, DPoint b) { return a.x * b.x + a.y * b.y; }
long double cross(DPoint a, DPoint b) { return a.x * b.y - a.y * b.x; }
long double norm(DPoint a) { return sqrtl(dot(a, a)); }

optional<DPoint> line_intersection(DPoint a, DPoint b,
                                   DPoint c, DPoint d) {
    DPoint first = b - a, second = d - c;
    long double denominator = cross(first, second);
    if (fabsl(denominator) <= GEOMETRY_EPS) return nullopt;
    long double parameter = cross(c - a, second) / denominator;
    return a + first * parameter;
}

DPoint projection_on_line(DPoint point, DPoint a, DPoint b) {
    DPoint direction = b - a;
    assert(norm(direction) > GEOMETRY_EPS);
    return a + direction * (dot(point - a, direction) / dot(direction, direction
    ));
}

long double distance_to_line(DPoint point, DPoint a, DPoint b) {
    DPoint direction = b - a;
    assert(norm(direction) > GEOMETRY_EPS);
    return fabsl(cross(direction, point - a)) / norm(direction);
}
```

```cpp
DPoint a{0,0},b{4,0},c{2,-2},d{2,2},p{3,3};
auto intersection=line_intersection(a,b,c,d);
cout<<(double)intersection->x<<' '<<(double)intersection->y<<'\n'; // 2 0
DPoint projected=projection_on_line(p,a,b);
cout<<(double)projected.x<<' '<<(double)projected.y<<'\n'; // 3 0
cout<<(double)distance_to_line(p,a,b)<<'\n'; // 3
// nullopt en line_intersection significa rectas paralelas o coincidentes.
```

7.4 Círculos

Cuándo usar: Intersecciones recta–círculo o círculo–círculo, tangencias y distancias asociadas a discos.

Complejidad: cada operación de círculos $O(1)$ tiempo; memoria $O(1)$ más la salida.

```cpp
struct Circle { DPoint center; long double radius; };

// nullopt: circulos coincidentes (infinitas intersecciones).
optional<vector<DPoint>> circle_intersections(Circle first, Circle second) {
    DPoint difference = second.center - first.center;
    long double distance = norm(difference);
    if (distance <= GEOMETRY_EPS &&
        fabsl(first.radius - second.radius) <= GEOMETRY_EPS) return nullopt;
    if (distance > first.radius + second.radius + GEOMETRY_EPS ||
        distance < fabsl(first.radius - second.radius) - GEOMETRY_EPS)
        return vector<DPoint>{};

    long double along = (first.radius * first.radius
                         - second.radius * second.radius
                         + distance * distance) / (2 * distance);
    long double height_squared = first.radius * first.radius - along * along;
    if (height_squared < 0 && height_squared > -GEOMETRY_EPS) height_squared =
    0;
    if (height_squared < 0) return vector<DPoint>{};

    DPoint unit = difference / distance;
    DPoint base = first.center + unit * along;
    long double height = sqrtl(height_squared);
    DPoint perpendicular{-unit.y, unit.x};
    if (height <= GEOMETRY_EPS) return vector<DPoint>{base};
    return vector<DPoint>{base + perpendicular * height,
                          base - perpendicular * height};
}
```

```cpp
Circle first{{0,0},2},second{{2,0},2};
auto intersections=circle_intersections(first,second);
if(!intersections) cout<<"Coincidentes\n"; // Infinitos puntos comunes.
else cout<<intersections->size()<<'\n';     // 2: (1,+sqrt(3)), (1,-sqrt(3))
// size 0: separados/contenidos; size 1: tangentes; size 2: se cortan.
```

7.5 Punto en polígono

Cuándo usar: Decidir si un punto está dentro, fuera o sobre el borde de un polígono simple, incluso no convexo.

Complejidad: $O(n)$ tiempo y $O(1)$ memoria adicional.

Devuelve 0 afuera, 1 adentro y 2 sobre el borde. Funciona para polígonos simples en cualquier orientación.

```
bool on_segment(DPoint a, DPoint b, DPoint point) {
    if (fabsl(cross(b - a, point - a)) > GEOMETRY_EPS) return false;
    return dot(point - a, point - b) <= GEOMETRY_EPS;
}

int point_in_polygon(const vector<DPoint> &polygon, DPoint point) {
    bool inside = false;
    int n = polygon.size();
    for (int i = 0; i < n; ++i) {
        DPoint a = polygon[i], b = polygon[(i + 1) % n];
        if (on_segment(a, b, point)) return 2;
        bool crosses = (a.y > point.y) != (b.y > point.y);
        if (crosses) {
            long double intersection_x = a.x
                + (b.x - a.x) * (point.y - a.y) / (b.y - a.y);
            if (point.x < intersection_x) inside = !inside;
        }
    }
    return inside ? 1 : 0;
}
```

```
vector<DPoint> polygon={{0,0},{4,0},{4,4},{0,4}};
cout<<point_in_polygon(polygon,{2,2})<<'\n'; // 1: adentro
cout<<point_in_polygon(polygon,{0,2})<<'\n'; // 2: sobre el borde
cout<<point_in_polygon(polygon,{5,2})<<'\n'; // 0: afuera
```

7.6 Rotating calipers: diámetro convexo

Cuándo usar: Obtener el par de puntos más alejados de un polígono convexo; aplicar primero convex hull si la entrada es un conjunto desordenado.

Complejidad: $O(n)$ tiempo y $O(1)$ memoria adicional.

El polígono debe ser convexo, sin puntos repetidos y con sus vértices en orden sobre el borde, horario o antihorario. Se detecta el sentido una sola vez y se elige la comparación correspondiente, sin copiar ni invertir el polígono. Eliminar vértices colineales intermedios con `convex_hull` antes de usar calipers. Si todos son colineales, se toman los extremos.

```
template<bool Clockwise>
i128 convex_diameter_oriented(const vector<Point> &polygon) {
    int n = polygon.size();
    i128 answer = 0;
    int opposite = 1;
    for (int i = 0; i < n; ++i) {
        int next = (i + 1) % n;
        while (true) {
            int candidate = (opposite + 1) % n;
            i128 current_area = cross(polygon[next] - polygon[i],
                                      polygon[opposite] - polygon[i]);
            i128 next_area = cross(polygon[next] - polygon[i],
                                   polygon[candidate] - polygon[i]);
            if constexpr (Clockwise) {
                if (next_area >= current_area) break;
            } else {
                if (next_area <= current_area) break;
            }
            opposite = candidate;
        }
        answer = max(answer, squared_distance(polygon[i], polygon[opposite]));
        answer = max(answer, squared_distance(polygon[next], polygon[opposite]))
    ;
    }
    return answer;
}

i128 convex_diameter_squared(const vector<Point> &polygon) {
    int n = polygon.size();
    if (n <= 1) return 0;
    if (n == 2) return squared_distance(polygon[0], polygon[1]);
    for (int i = 0; i < n; ++i) {
        i128 turn = cross(polygon[i], polygon[(i + 1) % n],
                          polygon[(i + 2) % n]);
        if (turn > 0) return convex_diameter_oriented<false>(polygon);
        if (turn < 0) return convex_diameter_oriented<true>(polygon);
    }
    // Todos colineales: los extremos lexicograficos son los mas lejanos.
    auto [first, last] = minmax_element(polygon.begin(), polygon.end());
    return squared_distance(*first, *last);
}
```

```
vector<Point> points={{0,0},{2,0},{2,2},{0,2},{1,1}};
vector<Point> hull=convex_hull(points);
i128 diameter_squared=convex_diameter_squared(hull);
cout<<(ll)diameter_squared<<'\n'; // 8: distancia entre esquinas opuestas
reverse(hull.begin(),hull.end()); // El mismo poligono en sentido horario.
cout<<(ll)convex_diameter_squared(hull)<<'\n'; // Tambien 8
// La distancia real es sqrt(8); se conserva al cuadrado para evitar decimales.
```

7.7 Geometría 3D y planos

Cuándo usar: Problemas espaciales con normales de caras, lado de un plano, proyección o intersección de una recta con un plano.

Usar para normales de caras, orientación respecto de un plano, proyecciones y distancias en el espacio. Un plano por A, B, C tiene normal $n = (B - A) \times (C - A)$ y ecuación $n \cdot (X - A) = 0$.

Complejidad: cada operación $O(1)$ tiempo y memoria.

```
struct Point3D {
    long double x = 0, y = 0, z = 0;
    Point3D operator+(Point3D other) const { return {x + other.x, y + other.y, z + other.z}; }
    Point3D operator-(Point3D other) const { return {x - other.x, y - other.y, z - other.z}; }
    Point3D operator*(long double scalar) const { return {x * scalar, y * scalar, z * scalar}; }
    Point3D operator/(long double scalar) const { return {x / scalar, y / scalar, z / scalar}; }
};

long double dot3d(Point3D a, Point3D b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
Point3D cross3d(Point3D a, Point3D b) {
    return {a.y * b.z - a.z * b.y,
            a.z * b.x - a.x * b.z,
            a.x * b.y - a.y * b.x};
}
long double norm3d(Point3D vector) { return sqrtl(dot3d(vector, vector)); }
long double signed_plane_value(Point3D point, Point3D plane_point, Point3D normal) {
    return dot3d(point - plane_point, normal);
}
Point3D project_on_plane(Point3D point, Point3D plane_point, Point3D normal) {
    long double squared_norm = dot3d(normal, normal);
    assert(squared_norm > 1e-24L);
    return point - normal * (signed_plane_value(point, plane_point, normal) / squared_norm);
}
long double distance_to_plane(Point3D point, Point3D plane_point, Point3D normal) {
    long double normal_length = norm3d(normal);
    assert(normal_length > 1e-12L);
    return fabsl(signed_plane_value(point, plane_point, normal)) / normal_length;
}
optional<Point3D> line_plane_intersection(Point3D point, Point3D direction,
                                          Point3D plane_point, Point3D normal) {
    assert(norm3d(direction) > 1e-12L && norm3d(normal) > 1e-12L);
    long double denominator = dot3d(direction, normal);
    if (fabsl(denominator) < 1e-12L) return nullopt;
    long double parameter = dot3d(plane_point - point, normal) / denominator;
    return point + direction * parameter;
}
```

```
Point3D a{0,0,0},b{1,0,0},c{0,1,0};
Point3D normal=cross3d(b-a,c-a); // (0,0,1): plano z=0
Point3D point{2,3,5};
cout<<(double)signed_plane_value(point,a,normal)<<'\n'; // 5: lado positivo
Point3D projected=project_on_plane(point,a,normal);
cout<<(double)projected.x<<' '<<(double)projected.y<<' '
    <<(double)projected.z<<'\n'; // 2 3 0
cout<<(double)distance_to_plane(point,a,normal)<<'\n'; // 5
auto hit=line_plane_intersection({0,0,5},{0,0,-1},a,normal);
cout<<(double)hit->x<<' '<<(double)hit->y<<' '
    <<(double)hit->z<<'\n'; // 0 0 0
// nullopt: la recta es paralela al plano; puede estar fuera o contenida.
```

Cuidado: La orientación de la normal depende del orden A, B, C. Normalizar sólo si se necesita una distancia; para signos, evitar la raíz mejora estabilidad.

7.8 Camino mínimo rodeando un polígono convexo

Cuándo usar: El segmento directo está bloqueado por un ùnico obstáculo convexo y se permite recorrer su borde entre puntos tangentes.

Usar cuando el segmento directo entre A y B atraviesa el interior de un obstáculo convexo. Si no lo atraviesa, la respuesta es $|AB|$. Si lo atraviesa, un camino òptimo toca dos vértices tangentes y recorre una de las dos cadenas del borde entre ellos.

Procedimiento:

1. hallar desde A y B los dos vértices tangentes al polígono;
2. precomputar longitudes prefijas del perímetro;
3. probar las combinaciones compatibles de tangencias y ambas cadenas;
4. minimizar $|A - u| + \text{borde}(u, v) + |v - B|$.

Complejidad: tangencias $O(\log n)$ con búsqueda binaria, o $O(n)$ en una implementación simple; memoria $O(n)$ para prefijos.

Cuidado: Si se permite tocar el borde, distinguir intersección con el interior de un simple contacto tangencial. Para varios obstáculos construir un grafo de visibilidad; esta receta aislada ya no alcanza.

7.9 Angular ordering

```
/**
```

```
 * Author: Simon Lindholm
 * Date: 2015-01-31
 * License: CC0
 * Source: me
 * Description: A class for ordering angles (as represented by int points and
 *  a number of rotations around the origin). Useful for rotational sweeping.
 *  Sometimes also represents points or vectors.
 * Usage:
 *  vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
 *  int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
 *  // sweeps j such that (j-i) represents the number of positively oriented
    triangles with vertices at 0 and i
 * Status: Used, works well
 */
#pragma once

struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

7.10 Distance to a segment

```
/**
 * Author: Ulf Lundstrom
 * Date: 2009-03-21
 * License: CC0
 * Source:
 * Description:\\
\begin{minipage}{75mm}
Returns the shortest distance between point p and the line segment from point s
    to e.
\end{minipage}
\begin{minipage}{15mm}
\vspace{-10mm}
\includegraphics[width=\textwidth]{content/geometry/SegmentDistance}
\end{minipage}
 * Usage:
 *  Point<double> a, b(2,2), p(1,1);
 *  bool onSegment = segDist(a,b,p) < 1e-10;
 * Status: tested
 */
#pragma once

#include "Point.h"

typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

7.11 Intersection points of segments

Our existing helper tests whether segments intersect; this source also returns intersection points.

```
/**
 * Author: Victor Lecomte, chilli
 * Date: 2019-04-27
 * License: CC0
 * Source: https://vlecomte.github.io/cp-geo.pdf
 * Description:\\
```

```
\begin{minipage}{75mm}
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned.
If no intersection point exists an empty vector is returned.
If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment.
The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates.
Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.
\end{minipage}
\begin{minipage}{15mm}
\includegraphics[width=\textwidth]{content/geometry/SegmentIntersection}
\end{minipage}
 * Usage:
 * vector<P> inter = segInter(s1,e1,s2,e2);
 * if (sz(inter)==1)
 *   cout << "segments intersect at " << inter[0] << endl;
 * Status: stress-tested, tested on kattis:intersection
 */
#pragma once

#include "Point.h"
#include "OnSegment.h"

template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

7.12 Reflection across a line

Projection is already covered; the additional capability is reflection.

```
/**
 * Author: Victor Lecomte, chilli
 * Date: 2019-10-29
 * License: CC0
 * Source: https://vlecomte.github.io/cp-geo.pdf
 * Description: Projects point p onto line ab. Set refl=true to get reflection
 * of point p across line ab instead. The wrong point will be returned if P is
 * an integer point and the desired point doesn't have integer coordinates.
 * Products of three coordinates are used in intermediate steps so watch out
 * for overflow.
 * Status: stress-tested
 */
#pragma once

#include "Point.h"

template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

7.13 Similarity between oriented segments

```
/**
 * Author: Per Austrin, Ulf Lundstrom
 * Date: 2009-04-09
 * License: CC0
 * Source:
 * Description:\\
\begin{minipage}{75mm}
 Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.
\end{minipage}
\begin{minipage}{15mm}
\vspace{-8mm}
\includegraphics[width=\textwidth]{content/geometry/linearTransformation}
\vspace{-2mm}
\end{minipage}
 * Status: not tested
 */
#pragma once

#include "Point.h"

typedef Point<double> P;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

7.14 Circle–line intersection

```cpp
/**
 * Author: Victor Lecomte, chilli
 * Date: 2019-10-29
 * License: CC0
 * Source: https://vlecomte.github.io/cp-geo.pdf
 * Description: Finds the intersection between a circle and a line.
 * Returns a vector of either 0, 1, or 2 intersection points.
 * P is intended to be Point<double>.
 * Status: unit tested
 */

#pragma once

#include "Point.h"

template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

7.15 Common circle tangents

```cpp
/**
 * Author: Victor Lecomte, chilli
 * Date: 2019-10-31
 * License: CC0
 * Source: https://vlecomte.github.io/cp-geo.pdf
 * Description: Finds the external tangents of two circles, or internal if r2 is negated.
 * Can return 0, 1, or 2 tangents -- 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same);
 * 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers).
 * .first and .second give the tangency points at circle 1 and 2 respectively.
 * To find the tangents of a circle with a point set r2 to 0.
 * Status: tested
 */
#pragma once

#include "Point.h"

template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

7.16 Circle–polygon intersection area

```cpp
/**
 * Author: chilli, Takanori MAEHARA
 * Date: 2019-10-31
 * License: CC0
 * Source: https://github.com/spaghetti-source/algorithm/blob/master/geometry/_geom.cc#L744
 * Description: Returns the area of the intersection of a circle with a ccw polygon.
 * Time: O(n)
 * Status: Tested on GNYR 2019 Gerrymandering, stress-tested
 */
#pragma once

#include "../../content/geometry/Point.h"

typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };
  auto sum = 0.0;
```

```
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```

7.17 Circumcircle

```
/**
 * Author: Ulf Lundstrom
 * Date: 2009-04-11
 * License: CC0
 * Source: http://en.wikipedia.org/wiki/Circumcircle
 * Description:\\
\begin{minipage}{75mm}
The circumcirle of a triangle is the circle intersecting all three vertices.
    ccRadius returns the radius of the circle going through points A, B and C
    and ccCenter returns the center of the same circle.
\end{minipage}
\begin{minipage}{15mm}
\vspace{-2mm}
\includegraphics[width=\textwidth]{content/geometry/circumcircle}
\end{minipage}
 * Status: tested
 */
#pragma once

#include "Point.h"

typedef Point<double> P;
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(P A, P B, P C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

7.18 Minimum enclosing circle

```
/**
 * Author: Andrew He, chilli
 * Date: 2019-05-07
 * License: CC0
 * Source: folklore
 * Description: Computes the minimum circle that encloses a set of points.
 * Time: expected O(n)
 * Status: stress-tested
 */
#pragma once

#include "circumcircle.h"

pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

7.19 Polygon centroid

```
/**
 * Author: Ulf Lundstrom
 * Date: 2009-04-08
 * License: CC0
 * Source:
 * Description: Returns the center of mass for a polygon.
 * Time: O(n)
 * Status: Tested
 */
#pragma once

#include "Point.h"

typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
```

```
  return res / A / 3;
}
```

7.20 Polygon clipping

```
/**
 * Author: Ulf Lundstrom
 * Date: 2009-03-21
 * License: CC0
 * Source:
 * Description:\\
\begin{minipage}{75mm}
 Returns a vector with the vertices of a polygon with everything to the left of
    the line going from s to e cut away.
\end{minipage}
\begin{minipage}{15mm}
\vspace{-6mm}
\includegraphics[width=\textwidth]{content/geometry/PolygonCut}
\vspace{-6mm}
\end{minipage}
 * Usage:
 *  vector<P> p = ...;
 *  p = polygonCut(p, P(0,0), P(1,0));
 * Status: tested but not extensively
 */
#pragma once

#include "Point.h"

typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```

7.21 Polygon union area

```
/**
 * Author: black_horse2014, chilli
 * Date: 2019-10-29
 * License: Unknown
 * Source: https://codeforces.com/gym/101673/submission/50481926
 * Description: Calculates the area of the union of $n$ polygons (not
    necessarily
 * convex). The points within each polygon must be given in CCW order.
 * (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be
    needed.)
 * Time: $O(N^2)$, where $N$ is the total number of points
 * Status: stress-tested, Submitted on ECNA 2017 Problem A
 */
#pragma once

#include "Point.h"
#include "sideOf.h"

typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0){
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      }
    }
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
```

```
  return ret / 2;
}
```

7.22 Line–convex hull intersection

```
/**
 * Author: Oleksandr Bacherikov, chilli
 * Date: 2019-05-07
 * License: Boost Software License
 * Source: https://github.com/AlCash07/ACTL/blob/master/include/actl/geometry/algorithm/intersect/line_convex_polygon.hpp
 * Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points.
 * lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:
 *  \begin{itemize*}
 *    \item $(-1, -1)$ if no collision,
 *    \item $(i, -1)$ if touching the corner $i$,
 *    \item $(i, i)$ if along side $(i, i+1)$,
 *    \item $(i, j)$ if crossing sides $(i, i+1)$ and $(j, j+1)$.
 *  \end{itemize*}
 *  In the last case, if a corner $i$ is crossed, this is treated as happening on side $(i, i+1)$.
 *  The points are returned in the same order as the line hits the polygon.
 * \texttt{extrVertex} returns the point of a hull with the max projection onto a line.
 * Time: O(\log n)
 * Status: stress-tested
 */
#pragma once

#include "Point.h"

#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```

7.23 Closest pair of points

```
/**
 * Author: Simon Lindholm
 * Date: 2019-04-17
 * License: CC0
 * Source: https://codeforces.com/blog/entry/58747
 * Description: Finds the closest pair of points.
 * Time: O(n \log n)
 * Status: stress-tested
 */
#pragma once

#include "Point.h"

typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
```

```
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

7.24 Nearest-neighbor queries: KD-tree

```
/**
 * Author: Stanford
 * Date: Unknown
 * Source: Stanford Notebook
 * Description: KD-tree (2d, can be extended to 3d)
 * Status: Tested on excellentengineers
 */
#pragma once

#include "Point.h"

typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(P a, P b) { return a.x < b.x; }
bool on_y(P a, P b) { return a.y < b.y; }

struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
  Node *first = 0, *second = 0;

  T distance(P p) { // min squared distance to a point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x,y) - p).dist2();
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x);
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal...)
      sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
      // divide by taking half the array for each child (not
      // best performance with many duplicates in the middle)
      int half = sz(vp)/2;
      first = new Node({vp.begin(), vp.begin() + half});
      second = new Node({vp.begin() + half, vp.end()});
    }
  }
};

struct KDTree {
  Node* root;
  KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

  pair<T, P> search(Node *node, P p) {
    if (!node->first) {
      // uncomment if we should not find the point itself:
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(), node->pt);
    }

    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed
    auto best = search(f, p);
    if (bsec < best.first)
      best = min(best, search(s, p));
    return best;
  }

  // find nearest point to a point, and its squared distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(P p) {
    return search(root, p);
  }
};
```

7.25 Manhattan MST candidate edges

```
/**
 * Author: chilli, Takanori MAEHARA
 * Date: 2019-11-02
```

```
 * License: CC0
 * Source: https://github.com/spaghetti-source/algorithm/blob/master/geometry/rectilinear_mst.cc
 * Description: Given N points, returns up to 4*N edges, which are guaranteed
 * to contain a minimum spanning tree for the graph with edge weights w(p, q) =
 * |p.x - q.x| + |p.y - q.y|. Edges are in the form (distance, src, dst). Use a
 * standard MST algorithm on the result to find the final MST.
 * Time: O(N \log N)
 * Status: Stress-tested
 */
#pragma once
#include "Point.h"

typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<int, 3>> edges;
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
  }
  return edges;
}
```

7.26 Delaunay triangulation

```
/**
 * Author: Philippe Legault
 * Date: 2016
 * License: MIT
 * Source: https://github.com/Bathlamos/delaunay-triangulation/
 * Description: Fast Delaunay triangulation.
 * Each circumcircle contains none of the input points.
 * There must be no duplicate points.
 * If all points are on a line, no triangles will be returned.
 * Should work for doubles as well, though there may be precision issues in 'circ'.
 * Returns triangles in order \{t[0][0], t[0][1], t[0][2], t[1][0], \dots\}, all counter-clockwise.
 * Time: O(n \log n)
 * Status: stress-tested
 */
#pragma once

#include "Point.h"

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}

pair<Q,Q> rec(const vector<P>& s) {
```

```
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```

7.27 Three-dimensional convex hull

```
/**
 * Author: Johan Sannemo
 * Date: 2017-04-18
 * Source: derived from https://gist.github.com/msg555/4963794 by Mark Gordon
 * Description: Computes all faces of the 3-dimension hull of a point set.
 *  *No four points must be coplanar*, or else random results will be returned.
 *  All faces will point outwards.
 * Time: O(n^2)
 * Status: tested on SPOJ CH3D
 */
#pragma once

#include "Point3D.h"

typedef Point3D<double> P3;

struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
};

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3>& A) {
  assert(sz(A) >= 4);
  vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
      q = q * -1;
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
  };
  rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);
```

```
41
42    rep(i,4,sz(A)) {
43      rep(j,0,sz(FS)) {
44        F f = FS[j];
45        if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
46          E(a,b).rem(f.c);
47          E(a,c).rem(f.b);
48          E(b,c).rem(f.a);
49          swap(FS[j--], FS.back());
50          FS.pop_back();
51        }
52      }
53      int nw = sz(FS);
54      rep(j,0,nw) {
55        F f = FS[j];
56  #define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
57        C(a, b, c); C(a, c, b); C(b, c, a);
58      }
59    }
60    for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
61      A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
62    return FS;
63  };
```

7.28 Polyhedron volume

```
1  /**
2   * Author: Mattias de Zalenski
3   * Date: 2002-11-04
4   * Description: Magic formula for the volume of a polyhedron. Faces should point
        outwards.
5   * Status: tested
6   */
7  #pragma once
8
9  template<class V, class L>
10 double signedPolyVolume(const V& p, const L& trilist) {
11   double v = 0;
12   for (auto i : trilist) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
13   return v / 6;
14 }
```

7.29 Spherical distance

```
1  /**
2   * Author: Ulf Lundstrom
3   * Date: 2009-04-07
4   * License: CC0
5   * Source: My geometric reasoning
6   * Description: Returns the shortest distance on the sphere with radius radius
        between the points
7   * with azimuthal angles (longitude) f1 ($\phi_1$) and f2 ($\phi_2$) from x axis
        and zenith angles
8   * (latitude) t1 ($\theta_1$) and t2 ($\theta_2$) from z axis (0 = north pole).
        All angles measured
9   * in radians. The algorithm starts by converting the spherical coordinates to
        cartesian coordinates
10  * so if that is what you have you can use only the two last rows. dx*radius is
        then the difference
11  * between the two points in the x direction and d*radius is the total distance
        between the points.
12  * Status: tested on kattis:airlinehub
13  */
14 #pragma once
15
16 double sphericalDistance(double f1, double t1,
17     double f2, double t2, double radius) {
18   double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
19   double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
20   double dz = cos(t2) - cos(t1);
21   double d = sqrt(dx*dx + dy*dy + dz*dz);
22   return radius*2*asin(d/2);
23 }
```

8. Programación dinámica

8.1 Longest Increasing Subsequence

Cuándo usar: Encontrar la subsecuencia estrictamente creciente más larga; aparece también tras ordenar una dimensión en problemas bidimensionales.

Devuelve una subsecuencia estrictamente creciente y no solamente su longitud.

```
vector<ll> longest_increasing_subsequence(const vector<ll> &a) {
    int n = a.size();
    vector<ll> tails;
    vector<int> tail_index, parent(n, -1);

    for (int i = 0; i < n; ++i) {
        int position = lower_bound(tails.begin(), tails.end(), a[i])
                     - tails.begin();
        if (position == (int)tails.size()) {
            tails.push_back(a[i]);
            tail_index.push_back(i);
        } else {
            tails[position] = a[i];
            tail_index[position] = i;
        }
        if (position > 0) parent[i] = tail_index[position - 1];
    }

    vector<ll> answer;
    if (!tail_index.empty())
        for (int i = tail_index.back(); i != -1; i = parent[i])
            answer.push_back(a[i]);
    reverse(answer.begin(), answer.end());
    return answer;
}
```

Complejidad: $O(n \log n)$ tiempo y $O(n)$ memoria.

Para subsecuencia no decreciente, reemplazar `lower_bound` por `upper_bound`.

8.2 LCS con reconstrucción

Cuándo usar: Encontrar una subsecuencia común de longitud máxima entre dos cadenas. No confundir subsecuencia con substring contiguo.

Complejidad: Tiempo $O(nm)$; memoria $O(nm)$ para reconstruir. Si sólo importa la longitud, usar dos filas y $O(\min(n, m))$ memoria.

```
string longest_common_subsequence(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j - 1] + 1
                     : max(dp[i - 1][j], dp[i][j - 1]);
    string answer;
    for (int i = n, j = m; i && j; ) {
        if (a[i - 1] == b[j - 1]) {
            answer.push_back(a[--i]); --j;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) --i;
        else --j;
    }
    reverse(answer.begin(), answer.end());
    return answer;
}
void lcs_example() {
    cout << longest_common_subsequence("abcde", "ace") << '\n'; // ace
}
```

Cuidado: Con $n = m = 5000$, los enteros de la tabla ocupan unos 95.4 MiB; con 10000, unos 381.5 MiB. Esta plantilla no cuenta todas las LCS ni resuelve por sí sola el problema constructivo Many LCS (SEERC 2021).

8.3 Distancia de edición con operaciones

Cuándo usar: Transformar una cadena en otra usando inserción, borrado y reemplazo, cada uno de costo uno. Devuelve costo y una secuencia òptima de operaciones.

Complejidad: Tiempo y memoria $O(nm)$. Sólo la distancia puede calcularse con dos filas; recuperar las operaciones requiere la tabla de esta versión.

Las posiciones de las operaciones son 0-indexadas sobre la cadena *tal como queda tras aplicar las operaciones anteriores.*

```
struct EditStep { char type; int position; char value; }; // I, D, R
pair<int, vector<EditStep>> edit_distance(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; ++i) dp[i][m] = n - i;
    for (int j = 0; j <= m; ++j) dp[n][j] = m - j;
    for (int i = n - 1; i >= 0; --i)
        for (int j = m - 1; j >= 0; --j)
            dp[i][j] = a[i] == b[j] ? dp[i + 1][j + 1]
                : 1 + min({dp[i + 1][j], dp[i][j + 1], dp[i + 1][j + 1]});
    vector<EditStep> steps;
    for (int i = 0, j = 0, position = 0; i < n || j < m; ) {
        if (i < n && j < m && a[i] == b[j]) {
            ++i; ++j; ++position;
        } else if (i < n && j < m && dp[i][j] == 1 + dp[i + 1][j + 1]) {
            steps.push_back({'R', position++, b[j++]}); ++i;
        } else if (i < n && dp[i][j] == 1 + dp[i + 1][j]) {
            steps.push_back({'D', position, 0}); ++i;
```

```
        } else {
            steps.push_back({'I', position++, b[j++]});
        }
    }
    return {dp[0][0], steps};
}
void edit_distance_example() {
    auto [cost, steps] = edit_distance("kitten", "sitting");
    cout << cost << '\n'; // 3
    for (auto step : steps)
        cout << step.type << ' ' << step.position << ' ' << step.value << '\n';
    // R 0 s; R 4 i; I 6 g. En D, value no se utiliza.
}
```

8.4 Distancia al palíndromo más cercano

Cuándo usar: Ediciones de costo uno para convertir una cadena en algún palíndromo, como Palindromic Distance (Taiwan Online Programming Contest 2025).

Complejidad: Tiempo y memoria $O(n^2)$; reconstrucción $O(n)$. Con suma de longitudes ≤ 3000, hay a lo sumo $9 \cdot 10^6$ celdas entre todos los casos.

```
pair<int, string> nearest_palindrome(const string &s) {
    int n = s.size();
    if (n == 0) return {0, ""};
    vector<vector<int>> dp(n, vector<int>(n));
    auto cost = [&](int l, int r) { return l >= r ? 0 : dp[l][r]; };
    for (int l = n - 1; l >= 0; --l)
        for (int r = l + 1; r < n; ++r)
            dp[l][r] = s[l] == s[r] ? cost(l + 1, r - 1)
                : 1 + min({cost(l + 1, r), cost(l, r - 1), cost(l + 1, r - 1)});
    string left, right;
    for (int l = 0, r = n - 1; l <= r; ) {
        if (l == r) { left.push_back(s[l]); break; }
        if (s[l] == s[r] || dp[l][r] == 1 + cost(l + 1, r - 1)) {
            left.push_back(s[l]); right.push_back(s[l]); ++l; --r;
        } else if (dp[l][r] == 1 + cost(l + 1, r)) ++l;
        else --r;
    }
    reverse(right.begin(), right.end());
    return {dp[0][n - 1], left + right};
}
void palindrome_edit_example() {
    auto [cost, palindrome] = nearest_palindrome("hello");
    cout << cost << '\n'; // 2; palindrome es una solucion valida.
}
```

Una inserción que empareja un extremo puede sustituirse por borrar ese extremo con igual costo. Por eso alcanza reconstruir con borrados y reemplazos. Los tres algoritmos trabajan con bytes de `string`, no con caracteres Unicode decodificados.

8.5 Knapsack 0/1

Cuándo usar: Elegir cada objeto como máximo una vez, con capacidad moderada, para maximizar valor o decidir qué sumas son alcanzables.

Cada objeto puede elegirse a lo sumo una vez; por eso la capacidad se recorre en orden descendente. Para recuperar sus *índices*, activar los comentarios: las decisiones usan $O(nW)$ bits adicionales y la recuperación cuesta $O(n)$. Admite peso cero; pesos no negativos y sumas de valores que entren en `long long`.

```
ll knapsack_01(const vector<int> &weight, const vector<ll> &value,
               int capacity
    // , vector<int>& chosen // Reconstruccion: indices elegidos.
) {
    assert(weight.size() == value.size() && capacity >= 0);
    assert(all_of(weight.begin(), weight.end(), [](int w) { return w >= 0; }));
    vector<ll> dp(capacity + 1, 0);
    // vector<vector<bool>> take(weight.size(), vector<bool>(capacity + 1, false
    ));
    for (int item = 0; item < (int)weight.size(); ++item)
        for (int current = capacity; current >= weight[item]; --current) {
            // if (dp[current - weight[item]] + value[item] > dp[current]) take[
    item][current] = true;
            dp[current] = max(dp[current],
                              dp[current - weight[item]] + value[item]);
        }
    // chosen.clear();
    // for (int item = (int)weight.size() - 1, current = capacity; item >= 0; --
    item)
    //     if (take[item][current]) { chosen.push_back(item); current -= weight[
    item]; }
    // reverse(chosen.begin(), chosen.end());
    return *max_element(dp.begin(), dp.end());
}
```

Complejidad: $O(nW)$ tiempo y $O(W)$ memoria.

Ejemplo opcional, tras activar los comentarios anteriores:

```
// void knapsack_recovery_example() {
//     vector<int> weight = {2,3,4}, chosen;
//     vector<ll> value = {4,5,7};
//     cout << knapsack_01(weight, value, 5, chosen) << '\n'; // 9
//     for (int item : chosen) cout << item << ' '; // 0 1
// }
```

8.6 Bitmask DP: camino hamiltoniano mínimo

Cuándo usar: El estado depende del subconjunto exacto de elementos ya usados y n es pequeño, normalmente $n \leq 20$.

Encuentra el costo mínimo de un camino que visita cada vértice exactamente una vez, con inicio y fin libres; no exige volver al inicio. La matriz cuadrada `cost[u][v]` representa un grafo dirigido: usar `LINF` si no existe la arista $u \to v$. Para un grafo no dirigido, cargar ambos sentidos. Admite costos negativos y devuelve `LINF` si no existe un camino completo; con un solo vértice devuelve cero. Devuelve el costo, no la secuencia de nodos. Se precalculan máscaras de aristas existentes para recorrer sólo los vecinos que aún no fueron visitados. La reconstrucción comentada reutiliza la DP: agrega $O(n)$ memoria para la secuencia y cuesta $O(n^2)$ al terminar. Si no hay camino, queda vacía.

```cpp
ll hamiltonian_path(const vector<vector<ll>> &cost
    // , vector<int>& path // Reconstruccion: vertices en orden.
) {
    int n = cost.size();
    assert(1 <= n && n < 31);
    assert(all_of(cost.begin(), cost.end(), [n](const auto &row) {
        return (int)row.size() == n;
    }));
    int states = 1 << n;
    vector<int> outgoing(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v = 0; v < n; ++v)
            if (cost[u][v] != LINF) outgoing[u] |= 1 << v;
    vector<vector<ll>> dp(states, vector<ll>(n, LINF));
    for (int v = 0; v < n; ++v) dp[1 << v][v] = 0;
    for (int mask = 1; mask < states; ++mask)
        for (int last = 0; last < n; ++last) if (dp[mask][last] != LINF)
            for (int nexts = outgoing[last] & ~mask; nexts; nexts &= nexts - 1)
    {
                int next = __builtin_ctz((unsigned)nexts); // nexts != 0
                dp[mask | (1 << next)][next] = min(
                    dp[mask | (1 << next)][next],
                    dp[mask][last] + cost[last][next]);
            }
    // path.clear();
    // int last = min_element(dp.back().begin(), dp.back().end()) - dp.back().
    begin();
    // if (dp.back()[last] != LINF) for (int mask = states - 1; ; ) {
    //     path.push_back(last);
    //     int previous_mask = mask ^ (1 << last);
    //     if (previous_mask == 0) break;
    //     int previous = -1;
    //     for (int v = 0; v < n; ++v)
    //         if ((previous_mask >> v & 1) && cost[v][last] != LINF &&
    //             dp[previous_mask][v] != LINF &&
    //             dp[previous_mask][v] + cost[v][last] == dp[mask][last]) {
    //             previous = v; break;
    //         }
    //     assert(previous != -1);
    //     last = previous; mask = previous_mask;
    // }
    // reverse(path.begin(), path.end());
    return *min_element(dp.back().begin(), dp.back().end());
}
```

Complejidad: $O(2^n n^2)$ tiempo y $O(2^n n)$ memoria; usar para n pequeño.

Cuidado: Requiere $1 \leq n < 31$ para la máscara, pero tiempo y memoria suelen limitarlo mucho antes: con $n = 20$, sólo los valores de la DP ocupan 160 MiB, más el costo de los vectores. Los costos finitos y todas las sumas parciales deben estar estrictamente entre $-$`LINF` y `LINF`. Una cota suficiente, para $n > 1$, es $(n-1)\max|w| <$ `LINF` sobre las aristas existentes. Nunca sumar la marca de arista inexistente.

```cpp
// void hamiltonian_recovery_example() {
//     vector<vector<ll>> cost = {{0,4,5}, {LINF,0,-2}, {LINF,LINF,0}};
//     vector<int> path;
//     ll answer = hamiltonian_path(cost, path);
//     if (answer == LINF) cout << "Sin camino\n";
//     else { cout << answer << '\n'; // 2
//         for (int v : path) cout << v << ' '; // 0 1 2
//     }
// }
```

8.7 Mochila ilimitada

Cuándo usar: Cada tipo de objeto puede repetirse, sus pesos son enteros positivos y se maximiza valor sin superar una capacidad W.

Complejidad: Tiempo $O(nW)$; memoria $O(W)$ más los objetos recuperados. La capacidad se recorre en orden ascendente para permitir repeticiones.

```cpp
pair<ll, vector<int>> unbounded_knapsack(const vector<int> &weight,
                                         const vector<ll> &value, int capacity) {
    assert(capacity >= 0 && weight.size() == value.size());
    vector<ll> dp(capacity + 1);
    vector<int> last(capacity + 1, -1);
    for (int i = 0; i < (int)weight.size(); ++i) {
        assert(weight[i] > 0);
        for (int c = weight[i]; c <= capacity; ++c)
            if (dp[c - weight[i]] + value[i] > dp[c]) {
                dp[c] = dp[c - weight[i]] + value[i]; last[c] = i;
            }
    }
    vector<int> chosen;
```

```
    for (int c = capacity; last[c] != -1; c -= weight[last[c]])
        chosen.push_back(last[c]);
    return {dp[capacity], chosen};
}
void unbounded_example() {
    auto [value, items] = unbounded_knapsack({3,5}, {4,7}, 10);
    cout << value << '\n'; // 14; items = {1,1}.
}
```

Cuidado: Peso cero y valor positivo darían un òptimo infinito. Los valores y sumas deben entrar en `long long`. Se permite elegir cero objetos.

8.8 Suma exacta 0/1 con mínima cantidad de objetos

Cuándo usar: Cada objeto se usa a lo sumo una vez y se necesita sumar exactamente W, minimizando la cantidad elegida. Sirve como componente de Nine Packs.

Complejidad: Tiempo $O(nW)$; memoria $O(W)$ enteros y $O(nW)$ bits de decisiones para reconstruir. Para sólo obtener el costo, omitir `take`.

```
optional<vector<int>> minimum_items_exact(const vector<int> &weight, int target,
                                          vector<int> *all_costs = nullptr) {
    assert(target >= 0);
    int n = weight.size(), infinity = n + 1;
    vector<int> dp(target + 1, infinity);
    vector<vector<bool>> take(n, vector<bool>(target + 1));
    dp[0] = 0;
    for (int i = 0; i < n; ++i) {
        assert(weight[i] >= 0);
        for (int s = target; s >= weight[i]; --s)
            if (dp[s - weight[i]] + 1 < dp[s]) {
                dp[s] = dp[s - weight[i]] + 1; take[i][s] = true;
            }
    }
    if (all_costs) *all_costs = dp;
    if (dp[target] == infinity) return nullopt;
    vector<int> chosen;
    for (int i = n - 1, s = target; i >= 0; --i)
        if (take[i][s]) { chosen.push_back(i); s -= weight[i]; }
    reverse(chosen.begin(), chosen.end());
    return chosen;
}
void exact_sum_example() {
    auto items = minimum_items_exact({8,10,12}, 20);
    if (items) for (int i : *items) cout << i << ' '; // 0 2
    // nullopt: imposible; vector vacio: target == 0.
}
```

Pasar `&costs` como tercer argumento permite consultar el mínimo para *todas* las sumas, con `weight.size()+1` como marca de imposible. En Nine Packs se calcula una tabla por lado y se busca la mejor suma positiva común; no llamar a la función de nuevo para cada suma. Copiar esa tabla agrega $O(W)$ tiempo y memoria, sin cambiar la cota total.

8.9 Subset sum con bitset

Cuándo usar: Sólo importa qué sumas son alcanzables con objetos 0/1 de peso no negativo. Procesar muchas sumas en paralelo reduce el costo de la DP booleana.

Complejidad: Para n objetos y S posiciones, $O(n\lceil S/w\rceil)$ operaciones de palabra, con w bits por palabra (usualmente 64); memoria $O(S)$ bits. La aceleración concreta depende de la implementación de bitset.

```
template<size_t S>
bitset<S> subset_sums(const vector<int> &weight) {
    static_assert(S > 0);
    bitset<S> reachable;
    reachable[0] = true;
    for (int w : weight) {
        assert(w >= 0);
        reachable |= reachable << (size_t)w;
    }
    return reachable;
}
void subset_sum_example() {
    auto reachable = subset_sums<21>({8,10,12}); // S debe ser > target.
    cout << reachable[20] << ' ' << reachable[19] << '\n'; // 1 0
}
```

Cuidado: Esta variante no devuelve los objetos ni la mínima cantidad. Una suma fuera de $[0, S)$ se descarta. Evitar bitsets locales gigantes si el juez tiene poco stack; se puede dar almacenamiento `static` a la estructura. Para muchos pesos repetidos, agrupar cantidades puede ser necesario para pasar.

8.10 Digit DP: contar enteros por sus dígitos

Cuándo usar: Contar números de un intervalo enorme cuya validez depende de los dígitos, no recorrer uno por uno todos los valores hasta 10^{18}.

Complejidad: Esta plantilla: $O(d \cdot 11 \cdot 10)$ tiempo y $O(d \cdot 11)$ memoria para d dígitos. Otros estados, como un residuo módulo k, agregan su propio factor.

Ejemplo: contar enteros no negativos sin dos dígitos adyacentes iguales. `last=10` significa que aún sólo se leyeron ceros iniciales. El cero se cuenta como un número válido.

```
ll count_no_equal_neighbors(ll upper) {
    if (upper < 0) return 0;
    string digits = to_string(upper);
    ll memo[20][11];
    for (auto &row : memo) fill(begin(row), end(row), -1);
```

```
    function<ll(int,int,bool)> dfs = [&](int position, int last, bool tight) ->
    ll {
        if (position == (int)digits.size()) return 1;
        ll &cached = memo[position][last];
        if (!tight && cached != -1) return cached;
        ll answer = 0;
        int limit = tight ? digits[position] - '0' : 9;
        for (int digit = 0; digit <= limit; ++digit) {
            if (digit == last) continue;
            int next = last == 10 && digit == 0 ? 10 : digit;
            answer += dfs(position + 1, next, tight && digit == limit);
        }
        if (!tight) cached = answer;
        return answer;
    };
    return dfs(0, 10, true);
}
void digit_dp_example() {
    ll left = 0, right = 20;
    cout << count_no_equal_neighbors(right) - count_no_equal_neighbors(left - 1)
         << '\n'; // 20: solo 11 queda excluido.
}
```

Cuidado: Requiere $0 \le L \le R \le LLONG_MAX$ para la resta de prefijos del ejemplo. Los ceros iniciales no deben confundirse con un dígito real. Adaptar el estado y el caso base al problema: Score Values (World Finals 2025) necesita residuos y maximizar frecuencias; esta función no lo resuelve directamente.

8.11 Divide and Conquer DP Optimization

Cuándo usar: DP por particiones $dp[k][i] = \min_{j<i}(dp[k-1][j] + cost(j,i))$ cuando el punto òptimo es monótono en i.

Para transiciones

$$dp_{new}[m] = \min_{0 \le k \le m} \{dp_{old}[k] + cost(k,m)\},$$

cuando el índice óptimo es monótono: $opt[m] \le opt[m+1]$.

```
template<class Cost>
void compute_divide_conquer(int left, int right,
                            int option_left, int option_right,
                            const vector<ll> &previous,
                            vector<ll> &current, Cost cost) {
    if (left > right) return;
    while (option_left <= option_right && previous[option_left] == LINF)
        ++option_left;
    while (option_left <= option_right && previous[option_right] == LINF)
        --option_right;
    if (option_left > option_right || right < option_left) {
        fill(current.begin() + left, current.begin() + right + 1, LINF);
        return;
    }
    int middle = left + (right - left) / 2;
    pair<ll,int> best{LINF, -1};
    int last_option = min(middle, option_right);
    for (int option = option_left; option <= last_option; ++option) {
        if (previous[option] == LINF) continue;
        ll transition = cost(option, middle);
        if (transition == LINF) continue;
        i128 raw = (i128)previous[option] + transition;
        ll candidate = raw > LINF ? LINF : raw < -LINF ? -LINF : (ll)raw;
        best = min(best, pair<ll,int>{candidate, option});
    }
    current[middle] = best.first;
    // Sin optimo finito no hay informacion para podar a los vecinos.
    int left_end = best.second == -1 ? option_right : best.second;
    int right_begin = best.second == -1 ? option_left : best.second;
    compute_divide_conquer(left, middle - 1, option_left, left_end,
                           previous, current, cost);
    compute_divide_conquer(middle + 1, right, right_begin, option_right,
                           previous, current, cost);
}
```

La monotonía debe demostrarse (por ejemplo con desigualdad cuadrangular o matriz Monge); no alcanza con que el costo "parezca convexo". `LINF` representa estados o transiciones imposibles; no se suma a un costo finito. Si no hay candidato finito, el estado queda en `LINF` y ambos lados conservan los límites de candidatos. Los costos y sumas finitos deben estar estrictamente entre $-$`LINF` y `LINF`.

Complejidad: $O(kn \log n)$ para k capas si evaluar `cost` cuesta $O(1)$ y cada estado procesado tiene candidato finito; memoria $O(n)$ con dos capas. Si hay estados imposibles por restricciones de `cost`, el fallback sin poda puede costar $O(kn^2)$; conviene acotar el dominio a los estados factibles cuando el problema lo permita.

8.11.1 Acotar los estados factibles

Para dividir los primeros i elementos en exactamente g segmentos no vacíos, los estados $i < g$ son imposibles. Si el último segmento es $[j,i)$, el corte cumple $g-1 \le j < i$. Inicializar los demás estados en `LINF` y calcular sólo $i \in [g,n]$:

```
template<class SegmentCost>
ll partition_nonempty(int n, int groups, SegmentCost segment_cost) {
    assert(n >= 0);
    if (groups < 0 || groups > n) return LINF;
    vector<ll> previous(n + 1, LINF), current(n + 1, LINF);
    previous[0] = 0; // Cero elementos y cero grupos.
    for (int g = 1; g <= groups; ++g) {
        fill(current.begin(), current.end(), LINF);
```

```
        auto cost = [&](int j, int i) -> ll {
            if (j >= i) return LINF; // Segmento vacio o invertido.
            return segment_cost(j, i); // Costo de [j,i).
        };
        compute_divide_conquer(g, n, g - 1, n - 1,
                               previous, current, cost);
        previous.swap(current);
    }
    return previous[n];
}
```

Si todo segmento no vacío tiene costo finito, cada estado procesado tiene al menos un candidato finito: el prefijo de longitud $g-1$ puede formar $g-1$ grupos y dejar el resto como último segmento. Con óptimos monótonos y costo $O(1)$, se conserva $O(\texttt{groups} \cdot n \log n)$ tiempo y $O(n)$ memoria. No hace falta que todas las transiciones sean válidas.

Cuidado: Que existan exactamente g grupos no garantiza la monotonía del corte óptimo; hay que demostrarla para el costo del problema. Si hay otras restricciones sobre los segmentos, pueden quedar estados imposibles dentro de $[g, n]$ y esta llamada por sí sola no elimina el fallback cuadrático.

Si se conocen estados factibles con huecos, se puede adaptar la recursión para recorrer una lista ordenada de ellos. Conservar los índices originales al evaluar costos y acotar candidatos; la posición dentro de la lista no es el extremo del segmento. Si la factibilidad sólo se descubre probando todos los candidatos, esta plantilla no garantiza evitar $O(n^2)$ por capa.

8.12 DP $O(n\sqrt{n})$ por cota triangular

Cuándo usar: Un parámetro del estado crece o decrece estrictamente en cada salto, por lo que la suma mínima de los saltos limita su cantidad a $O(\sqrt{n})$.

Usar cuando un estado guarda una longitud/salto que cambia estrictamente en cada transición. No pueden existir demasiados cambios: $1+2+\cdots+t \le n$ implica $t = O(\sqrt{n})$. Guardar sólo estados hasta ese límite suele bajar una DP aparentemente cuadrática a $O(n\sqrt{n})$.

Complejidad: tiempo $O(n\sqrt{n})$; memoria $O(n\sqrt{n})$ o $O(\sqrt{n})$ con rolling arrays.

```
int triangular_state_limit(int n) {
    int length = 0;
    while (1LL * (length + 1) * (length + 2) / 2 <= n) ++length;
    return length;
}

// Esqueleto: adaptar la transicion al problema.
vector<vector<ll>> triangular_bound_dp(int n) {
    int limit = triangular_state_limit(n);
    const ll infinity = (1LL << 60);
    vector<vector<ll>> dp(n + 1, vector<ll>(limit + 1, infinity));
    dp[0][0] = 0;
    for (int position = 0; position < n; ++position)
        for (int length = 0; length <= limit; ++length) if (dp[position][length]
     < infinity) {
            dp[position + 1][length] = min(dp[position + 1][length], dp[position
    ][length]);
            int jump = length + 1;
            if (position + jump <= n && jump <= limit)
                dp[position + jump][jump] = min(dp[position + jump][jump],
                                                dp[position][length] + 1);
        }
    return dp;
}
```

Cuidado: La cota debe demostrarse con la suma mínima de los saltos; no basta con observar que aparece $\sqrt{n}$ en los límites.

8.13 Knuth optimization

KACTL provides the recurrence and the required monotonicity conditions as a reference, without a standalone function.

```
/**
 * Author: Simon Lindholm
 * License: CC0
 * Source: http://codeforces.com/blog/entry/8219
 * Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j}(a[i][k]
    + a[k][j]) + f(i, j)$, where the (minimal) optimal $k$ increases with both
    $i$ and $j$,
 *  one can solve intervals in increasing order of length, and search $k = p[i][
    j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$.
 *  This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f
    (a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le
    d$.
 *  Consider also: LineContainer (ch. Data structures), monotone queues, ternary
     search.
 * Time: O(N^2)
 */
```

9. Algoritmos offline

9.1 Mo's Algorithm

Cuándo usar: Muchas consultas offline sobre subarreglos, sin updates o con una extensión controlada, cuando agregar/quitar un extremo es barato.

Reordena consultas estáticas de rango para mover dos punteros pocas veces. Los rangos son semiabiertos $[l, r)$. El estado debe admitir agregar y quitar un elemento en $O(1)$ o costo pequeño.

Complejidad: aproximadamente $O((n+q)\sqrt{n})$ movimientos; memoria $O(n+q)$.

```cpp
struct MoQuery { int left, right, index; };

template<class Add, class Remove, class GetAnswer>
vector<ll> mos_algorithm(int n, vector<MoQuery> queries,
                         Add add, Remove remove, GetAnswer get_answer) {
    int block_size = max(1, (int)sqrt(max(1, n)));
    sort(queries.begin(), queries.end(), [&](const MoQuery &a, const MoQuery &b)
     {
        int block_a = a.left / block_size, block_b = b.left / block_size;
        if (block_a != block_b) return block_a < block_b;
        return block_a & 1 ? a.right > b.right : a.right < b.right;
    });

    vector<ll> answers(queries.size());
    int current_left = 0, current_right = 0;
    for (const MoQuery &query : queries) {
        while (current_left > query.left) add(--current_left);
        while (current_right < query.right) add(current_right++);
        while (current_left < query.left) remove(current_left++);
        while (current_right > query.right) remove(--current_right);
        answers[query.index] = get_answer();
    }
    return answers;
}
```

Ejemplos: cantidad de valores distintos, suma de frecuencias al cuadrado y consultas offline sobre frecuencias. Para actualizaciones existe Mo con tiempo.

Ejemplo: cantidad de valores distintos en cada rango. `add(index)` agrega `values[index]` a la ventana actual, `remove(index)` lo quita y `get_answer()` devuelve el estado mantenido.

```cpp
vector<int> values={1,2,1,3,2};
vector<MoQuery> queries={{0,3,0},{2,5,1}}; // [0,3), [2,5)
unordered_map<int,int> frequency;
ll distinct=0;
auto add=[&](int index) {
    if(frequency[values[index]]++==0) ++distinct;
};
auto remove=[&](int index) {
    if(--frequency[values[index]]==0) --distinct;
};
auto get_answer=[&]() { return distinct; };
vector<ll> answer=mos_algorithm(values.size(),queries,
                                add,remove,get_answer);
cout<<answer[0]<<' '<<answer[1]<<'\n'; // 2 3
```

Las consultas se procesan reordenadas, pero `query.index` restaura el orden original de las respuestas. "Offline" significa que todas las consultas deben conocerse antes de comenzar; Mo no acelera una sola consulta ni admite updates comunes en esta versión.